FRONTIS 清華大學 Tsinghua University 2026-06-25

Self-Improving Agents in the Era of Experience: A Survey of Self- to Meta-Evolution

Che Jiang[1,2*†], Jincheng Zhong[1,2*], Yu Fu[1*], Kai Tian[1,2*], Junlin Yang[1,2*], Kaikai Zhao[1*], Yuchong Wang[1,2*], Tianwei Luo[1*], Weizhi Wang[1,2], Yuxin Zuo[1], Guoli Jia[1], Xingtai Lv[1,2], Dianqiao Lei[1,2], Sihang Zeng[1], Yuru Wang[1], Zhenzhao Yuan[1,2], Xinwei Long[1], Ermo Hua[1], Can Ren[2], Xin Jiang[2], Shulei Xie[2], Yuanchun Zheng[2], Youbang Sun[1], Biqing Qi[1]
Ning Ding[1‡], Kaiyan Zhang[2†‡], Bowen Zhou[1‡]

[1] Tsinghua University [2] Horizon Research, Frontis.AI
[†] Project Lead. [*] Core Contributors. [‡] Corresponding Authors.

HomePage GitHub

Abstract | In the Era of Experience, agentic AI is no longer defined only by what a model can infer from static data, but by how a deployed system accumulates, organizes, and reuses experience from interaction. This survey studies experience-driven improvement in deployed agentic AI systems. We focus on the runtime harness as the infrastructure that captures traces, routes actions, exposes feedback, and governs mutable state. Around this infrastructure, we review how experience becomes reusable skill, persistent memory, verifiable environment feedback, trainable model behavior, and meta-level control. We then identify the remaining barriers to reliable improvement, including longitudinal evaluation, transfer, verification, and safety governance. Making agents smarter after deployment is therefore a trace-to-capability problem: the field must learn how to capture experience, assign it to the right update surface, verify its value, and preserve control as the system changes.

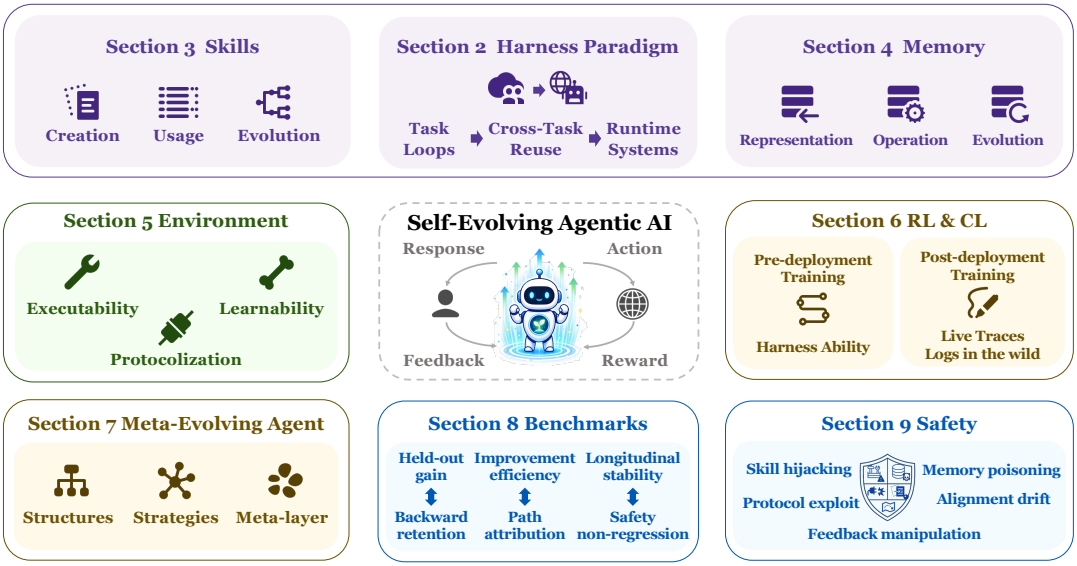

Figure 1 | Overview of the survey. The figure summarizes how the paper moves from the harness-agent definition to external runtime adaptation, parameter-side consolidation, and the evaluation and safety conditions for turning deployment experience into durable capability.

Contents

Part I. Agents in the Era of Experience

1. Introduction

Classical AI characterizes computer agents by autonomous control, environment perception, prolonged operation, adaptation to change, and goal-directedness [Russell and Norvig, 2020]. The *Era of Experience* brings these attributes back to the center of agentic AI: future progress increasingly comes from experience generated as agents interact with their environments, receive grounded rewards, and face consequences that unfold over long horizons [Silver and Sutton, 2025]. The key object is therefore a deployed agentic AI system in which accumulated experience shapes later behavior.

The question is how experience becomes reliable improvement. The Gödel Machine formalized an ideal case in which self-modification is admitted only after a proof of expected-utility improvement [Schmidhuber, 2003]. Recent self-improving coding agents replace this proof gate with empirical selection: Darwin Gödel Machine lets agents modify their own code, evaluates the resulting variants, and keeps successful descendants in an archive [Zhang et al., 2025d]. The same pattern appears at a larger scale in automated-AI-research loops, where proposed research changes are implemented, tested, checked for regressions or reward hacks, and then folded into the context for later attempts [Recursive, 2026]. Practitioner discussions have begun to describe this style of work as "loop engineering" [Osmani, 2026]. These examples point to a practical route from proof to experience: improvement becomes credible when the evidence around a change remains available to future attempts and continues to support later behavior.

This makes long-running deployment central to the study of agentic AI. Experience matters when it leaves a durable effect on the system: a retrievable skill, a memory that changes later context, a policy update, or a record that guides the next attempted improvement. Recent systems such as AlphaEvolve, SkillRL, and Agent0 illustrate this shift from isolated task execution toward engineered improvement processes [AlphaEvolve team, 2025, Xia et al., 2025, 2026a]. They differ in domain and update mechanism, but they share a dependence on persistent runtime infrastructure that can collect experience, support evaluation, and govern what is allowed to change.

The term *harness* has emerged to describe this runtime control layer. In Anthropic's managed-agent stack, it denotes the loop that invokes Claude and routes tool calls between a persistent session and an execution sandbox; related Claude Agent SDK guidance uses the same language for long-running agents that gather context, plan, and execute across extended work [Martin et al., 2026, Young, 2025]. In the wider practitioner community, *harness* is increasingly used for the control layer around the model, where planning, tool use, context, memory, permissions, verification, recovery, stop conditions, and escalation paths are specified, observed, and revised [Martin, 2026, Xu, 2026b]. In coding-agent practice, this control role can also be described through *guides* and *sensors*: guides shape what the agent is allowed or encouraged to do before action, while sensors expose feedback that lets the agent detect errors and correct its trajectory after action [Böckeler, 2026]. Building on these engineering accounts, we treat the harness as the experience infrastructure through which deployed agentic AI systems can turn interaction into later behavior.

The discussion above motivates the survey's main question: in the Era of Experience, how can long-term deployed agents use harness-mediated control to accumulate experience, self-evolve, and eventually support meta-evolution? To answer it, this survey reviews recent work on deployed agentic AI foundations, external runtime adaptation, parameter-side consolidation, meta-evolution, evaluation, safety, and open problems as follows.

- We first formalize the deployed agentic AI system as a runtime object whose behavior is shaped

by a base model, a mutable harness, a user-facing side, and an environment-facing side. We define the harness as experience infrastructure, decompose it into interaction interfaces and experience destinations, and review the development from task-bounded tool-use loops to long-running deployed runtimes in Section 2.

- We then follow deployment experience to its main update destinations: editable runtime surfaces and model-parameter consolidation. Skills, memory, and executable environments show how experience can shape later behavior without changing model weights; model-side consolidation asks when stable lessons from deployment should be internalized into parameters. These update sites are developed in Sections 3–6.
- In addition, when the improvement process itself becomes part of the evolving system, meta-layers coordinate which components, rules, budgets, evaluators, and update mechanisms should change. This meta-evolution layer is discussed in Section 7.
- Finally, we study the conditions under which deployed agentic AI systems can improve reliably. Evaluation must distinguish real longitudinal gain from overfitting, cost transfer, and hidden regression; safety must control mutable skills, memories, tools, feedback channels, and governance policies as moving attack surfaces. These issues anchor Sections 8, 9, and 10.

2. Harness as Experience Infrastructure

2.1. Runtime Adaptation Requires Experience Infrastructure

Experience-driven improvement depends on a part of the system that can change on the timescale of deployment. In contemporary agentic AI systems, this role is increasingly served by a runtime harness around the model: the layer that organizes context, tool access, permissions, execution, feedback, memory, and recovery. Its practical importance comes from a timescale asymmetry. Harness state can be inspected, revised, and governed during deployment far more frequently and cheaply than model weights, yet it directly shapes what the model sees, what actions it can take, what evidence is captured, and which parts of interaction become reusable experience.

This motivates our unit of analysis: a deployed agentic AI system whose behavior at time t is jointly determined by the base model, the mutable harness, the user-facing side through which goals, instructions, preferences, and feedback enter, and the environment-facing side through which actions, observations, tests, and operational state are exposed:

$$\mathcal{A}_t = \langle M_{\theta_t}, H_t, U_t, E_t \rangle.$$

Here t denotes wall-clock deployment time. M_{θ_t} is the base model available at time t, with parameters θ_t; H_t is the stateful mutable harness; U_t denotes the user-facing side of deployment, including chat, gateways, workflow interfaces, user protocols, and context assembly from user intent; and E_t denotes the environment-facing side, including filesystems, browsers, shells, MCP, A2A [A2A Protocol Contributors, 2026], tools, tests, logs, and operational state. This tuple defines the runtime object, while the coupling relation is carried by the harness: H_t mediates how user requests become model-visible context, how model outputs become environment actions, and how resulting traces become future updates.

Deployment is continuous, but experience need not be compiled after each individual interaction. We therefore distinguish wall-clock time t from irregular trace-window boundaries

$$0 = t_0 < t_1 < t_2 < \cdots < t_i < \cdots .$$

The boundary t_i is a harness-level segmentation decision, possibly made with model assistance. Between t_{i-1} and t_i, the system may contain many atomic exchanges, including user–agent interaction, agent–environment execution, and even direct user–environment feedback. We write

the i-th trace window as the collection of interaction records in that interval:

$$\tau_i = \{\alpha \mid \text{time}(\alpha) \in [t_{i-1}, t_i)\}.$$

When the window closes, the harness maps this trace into usable experience:

$$z_i = H_{t_i}(\tau_i).$$

Here H_{t_i} denotes the harness state that closes the window and performs this compilation, before any update based on z_i is committed. The symbol z_i is deliberately not a raw log. It denotes experience that has been filtered, compressed, attributed, or verified enough to be usable for later adaptation. Up to deployment horizon T, the experience reservoir is

$$Z_T = \{z_i : 1 \le i \le N(T)\}, \qquad N(T) = \max\{i : t_i \le T\}.$$

This is the point at which the deployed agent can become smarter after deployment. On the fast timescale, compiled experience changes the harness at trace-window boundaries:

$$H_{t_i^+} = \Phi_H(H_{t_i}, z_i).$$

On the slower timescale, accumulated experience can be consolidated into model parameters at rarer update checkpoints $0 < s_1 < s_2 < \cdots < s_k < \cdots$:

$$\theta_{s_k^+} = \Phi_M(\theta_{s_k}, Z_{s_k}).$$

In this formulation, a deployed agentic AI system acts in an environment, turns trace windows into compiled experience, and uses that experience to update its future behavior. Figure 2 summarizes this runtime object, showing how the harness mediates user-facing context, environment-facing execution, compiled experience, and mutable update surfaces.

2.2. How Did We Get Here? From Task Loops to Runtime Systems

The current runtime-centric view did not appear all at once. A useful historical compression is to distinguish three successive generations of agent systems.

Gen 1: task-bounded loops. The first generation established that language models could be coupled to external environments through explicit reasoning-and-acting loops. In this regime, a user request initialized an episode, the model produced interleaved reasoning and actions, and the loop ended when the task was complete. WebGPT showed that a language model could operate inside a browser-like environment and use references to support long-form answers [Nakano et al., 2022]. SayCan grounded language-model plans in robotic affordances, making action feasibility part of the agent loop [Ahn et al., 2022]. ReAct then made interleaved reasoning and acting a general prompting pattern across knowledge and decision-making tasks [Yao et al., 2023]. Reflexion added verbal feedback and self-reflection inside bounded episodes, foreshadowing later self-improvement work while still remaining largely episode-centered [Shinn et al., 2023]. The main contribution of this generation was interface grounding: the model could search, call tools, observe results, revise its next action, and sometimes reflect on failure inside a bounded task. Its limitation was that most useful state remained episodic. Once the task ended, the system usually had no durable runtime state through which later tasks could benefit from earlier experience.

Gen 2: persistent and reusable agent runtimes. The second generation made agent state and execution structure persist across episodes. Voyager is an important early marker of this shift: its automatic curriculum, executable skill library, and feedback-driven program repair turned prior interaction into reusable behavior [Wang et al., 2023]. In parallel, frameworks such as

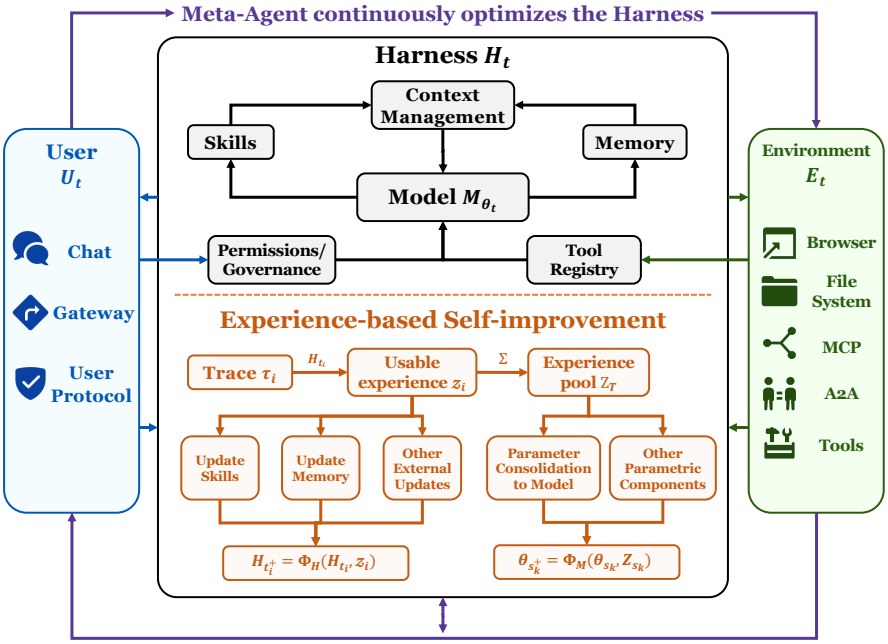

Figure 2 | Harness as experience infrastructure for deployed agents. The user side U_t supplies goals, instructions, preferences, approvals, and corrections, while the environment side E_t supplies executable state, observations, tests, and verifiable feedback. The harness H_t mediates context, action routing, permissions, and trace compilation, turning interaction traces into usable experience that can update skills, memory, other external components, or model parameters.

MetaGPT and AutoGen made multi-agent roles, conversation patterns, and coordination logic explicit runtime structure [Hong et al., 2024, Wu et al., 2024b]. LangGraph sharpened this runtime view by representing agent workflows as stateful graphs with branching, checkpointing, and recoverable execution [LangChain Team, 2024]. Software-agent systems such as SWE-agent and OpenHands further moved the field toward agent-computer interfaces, repository navigation, file editing, test execution, and sandboxed execution [Wang et al., 2025c, Yang et al., 2024b]. In this generation, the agent is no longer only a prompt loop. It becomes a runtime system with reusable procedures, persistent artifacts, execution tools, and coordination mechanisms. However, these systems are still mostly designed or configured by humans; their runtime surfaces are durable, but not yet systematically evolved after deployment.

Gen 3: productized and self-evolving runtime systems. The third generation turns the deployed harness itself into the main object of engineering and adaptation. Productized coding agents and agent workspaces such as Claude Code, Codex, and Cursor 3 show that the agent runtime is becoming a development substrate: it runs in terminals, repositories, cloud workspaces, managed execution environments, and agent-management interfaces; maintains task state; invokes tools; edits files; runs tests; and hands work back to humans through reviewable artifacts [Anthropic, 2025, Cursor, 2026, OpenAI, 2025]. OpenClaw and Hermes Agent provide open and personal-runtime counterparts to this product wave. OpenClaw exposes a persistent, editable harness surface [OpenClaw Contributors, 2026]. Hermes Agent emphasizes an always-on personal agent with durable memory, agent-managed skills, cron, MCP, and gateway-style tool integration [Nous Research, 2026]. Recent self-evolution work studies how such runtime surfaces can be updated after interaction. MetaClaw studies deployed agents that combine fast external adaptation with slower policy consolidation [Xia et al., 2026b]; OpenClaw-RL turns live next-state signals into policy-learning supervision inside an agent runtime [Wang et al., 2026l]; and Agentic Harness Engineering treats tools, middleware, skills, memory, and sub-agent configurations as editable,

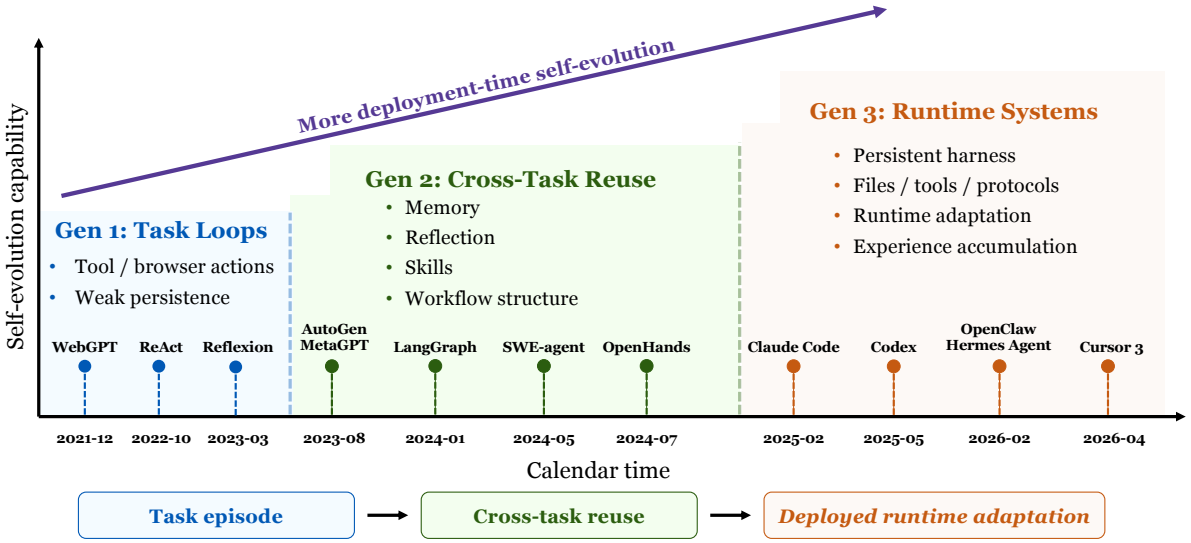

Figure 3 | Historical progression from task-bounded loops to deployed runtime systems. From Gen 1 to Gen 3, the main object shifts from episodic tool use, to cross-task memory and workflow structuring, to persistent harness-centered runtimes whose adaptation surfaces can evolve after deployment.

observable, and verifiable evolution targets [Lin et al., 2026b]. The central shift is temporal and operational: the agent is a persistent, environment-coupled runtime whose harness can collect evidence, revise itself, and govern post-deployment change.

This generational framing answers the question raised in the Introduction: how did we get here? The field arrived here by gradually externalizing more of the agent's operative state and then discovering that, once deployment-time change becomes systematic, the runtime layer itself must be treated as a first-class scientific object.

2.3. Experience-Centric Scope and Survey Organization

Building on the deployed-system definition in Section 2.1, we organize the survey around experience-based improvement in agentic AI systems. The structure follows a simple progression: experience is first externalized through mutable runtime surfaces, then selectively consolidated into model parameters, then used to schedule and coordinate improvement processes, and finally measured and bounded through evaluation and safety. Our scope foregrounds components that generate, retain, reuse, or consolidate deployment experience, without aiming to list every agent component.

Figure 2 makes this organization explicit. A deployed agentic AI system accumulates traces through the interaction between the model, the harness, users, and the environment. These traces become useful only when the runtime can turn them into durable objects that later executions can inherit. The first set of chapters therefore studies the harness-visible surfaces where experience can be written without changing model weights. Section 3 studies how experience becomes reusable procedures: skills capture ways of acting that can be retrieved, composed, verified, revised, and reused across future tasks. Section 4 studies how experience becomes persistent state: memory preserves facts, preferences, task state, failures, summaries, and abstractions that remain useful beyond the current interaction. Section 5 then studies the execution environments in which experience is produced. Environments determine whether actions are grounded, whether outcomes are observable, and whether feedback can be verified through tests, logs, protocols, or other runtime evidence.

The survey then turns to model-side consolidation. Some experience remains too local, brittle, or context-specific to justify parameter updates, while other traces become stable enough to serve as training signals. Section 6 therefore studies reinforcement learning, fine-tuning, and continual learning methods that use deployment-derived traces or harness-mediated signals to update the model itself. This path depends on the runtime system rather than replacing it: the harness selects traces, extracts training signals, preserves execution conditions, and determines which experience is suitable for parameter-side learning.

Once both external and parameter-side routes are available, improvement is no longer only a component-level problem. The system must also decide which improvement process should run, when it should run, and under what budget, evidence, and rollback conditions. Section 7 studies this meta-level problem: experience does not only update skills, memory, environments, or parameters, but can also inform the choice and coordination of future update mechanisms.

Finally, experience-based change must be measured and bounded. Section 8 studies how to evaluate longitudinal improvement rather than isolated task performance, including the need to distinguish real learning from overfitting, cost transfer, benchmark leakage, and hidden regression. Section 9 studies the corresponding control problem: mutable skills, memories, tools, feedback channels, and governance rules expand the system's attack surface as they evolve. These chapters close the loop by asking when experience-driven change is reliable enough to be trusted after deployment. Section 10 collects the remaining challenges that appear when experience-based improvement is studied as a property of deployed systems.

2.4. Related Surveys

Existing surveys can be organized into three complementary lines. The first studies agent harnesses and agent systems as model-external runtime infrastructure. The second studies individual harness surfaces such as skills, memory, context, workflow, evaluation, and safety. The third makes agent evolution itself the object of study. Table 1 compares these lines along three questions: what evolves, where evolution happens, and how improvement is driven.

2.4.1. Agent Harness and System-level Surveys

Broad agent surveys provide the natural baseline. Wang et al. and Xi et al. help establish the general LLM-agent framing and application landscape, covering core agent capabilities, tool/environment interaction, and representative deployment scenarios. [Wang et al., 2024, Xi et al., 2023]. More recent system-level surveys make the execution loop and surrounding infrastructure more explicit. Xu surveys AI agent systems across architectures, applications, and evaluation, while Chen et al. frame modern AI systems as compound compositions of models and external components rather than standalone model calls [Chen et al., 2025a, Xu, 2026a]. Meng et al. explicitly treat the agent harness as a first-class infrastructure layer around LLM agents, formalizing its execution, tool, context, state, lifecycle, and verification functions [Meng et al., 2026a]. Li et al. extend this harness-centered line with the ETCLOVG taxonomy, separating execution environment, tool interface, context management, lifecycle/orchestration, observability, verification, and governance while mapping more than 170 open-source projects onto that engineering surface [Li et al., 2026b]. Zhou et al. [Zhou et al., 2026a] frame recent agent progress as externalization, where memory, skills, protocols, and harness engineering shift cognitive burdens from model parameters into external infrastructure, while Lu et al. [Lu et al., 2026a] organize OpenClaw agents in open deployment around open policy, environment, population, and substrate.

These surveys establish that modern agents are increasingly system-level and harness-mediated. Their main purpose, however, is to characterize system composition and execution infrastructure.

Survey	Evolution Object			Application Settings		Evolution Methods		
	Model	Simple Agent	Harness-mediated Agent	Long Horizon	Open-ended	Human-aided	Self-improve	Meta-evolve
Ours	△	△	✓	✓	✓	✓	✓	✓
Agent harness and system-level surveys								
Wang et al. [2024]	△	✓	△	△	△	△	△	×
Xi et al. [2023]	△	✓	△	△	△	△	△	×
Xu [2026a]	△	✓	△	✓	△	△	△	×
Chen et al. [2025a]	×	△	✓	△	△	△	△	×
Meng et al. [2026a]	×	△	✓	✓	✓	△	△	△
Li et al. [2026b]	×	△	✓	✓	△	✓	△	△
Zhou et al. [2026a]	△	△	✓	✓	✓	△	△	△
Lu et al. [2026a]	△	△	✓	✓	✓	△	✓	△
Harness-component surveys								
Jiang et al. [2026b]	×	△	✓	✓	△	△	△	×
Xu and Yan [2026]	×	△	✓	✓	△	△	△	×
Zhang et al. [2024b]	×	△	△	✓	△	×	△	×
Du [2026]	×	△	△	✓	△	×	△	×
Yue et al. [2026]	×	△	✓	✓	△	△	△	△
Li et al. [2026a]	△	△	△	✓	△	△	△	△
Mei et al. [2025]	△	△	△	✓	△	△	△	×
Yehudai et al. [2025]	×	✓	△	△	△	×	×	×
Mohammadi et al. [2025]	×	✓	△	✓	△	×	×	×
Su et al. [2025a]	×	✓	△	✓	△	×	△	×
Evolution-oriented surveys								
Tao et al. [2024]	✓	△	×	△	△	△	✓	×
Gao et al. [2026]	✓	✓	△	✓	✓	△	✓	△
Fang et al. [2025a]	△	✓	△	✓	✓	△	✓	△
Xiang et al. [2026]	✓	✓	△	✓	✓	△	✓	△
Zhang et al. [2026c]	✓	✓	△	✓	✓	△	✓	△

Table 1 | Coverage of related surveys. ✓ denotes primary coverage, △ partial coverage, and × little or no coverage.

Our focus is the next question: once those external structures exist, which of them can change after deployment, and how do such changes make the agent system more capable over time?

2.4.2. Harness-Component Surveys

A second line surveys the individual surfaces that make up the runtime infrastructure. Skill surveys treat procedural competence as a reusable object that can be acquired, stored, composed, evaluated, governed, and updated [Jiang et al., 2026b, Xu and Yan, 2026]. Memory surveys study persistence across interactions, including memory design, write–manage–read operations, forgetting, retrieval, evaluation, and safety [Du, 2026, Zhang et al., 2024b]. Workflow and context-engineering surveys analyze how execution graphs and inference-time information payloads are constructed, optimized, and integrated into agent systems [Mei et al., 2025, Yue et al., 2026]. Environment-engineering surveys further add an execution-side view of agentic systems [Li et al., 2026a]. Evaluation and safety surveys further define the verification and risk surfaces of agent deployment, from benchmark design and long-horizon interaction to autonomy-induced security risks [Mohammadi et al., 2025, Su et al., 2025a, Yehudai et al., 2025].

These component-level views are indispensable, and later chapters build directly on them. Their limitation is not lack of depth, but fragmentation: each surface is usually studied as a separate capability layer, while deployed improvement depends on how skills, memory, context, workflow, evaluation, and safety controls interact inside one runtime loop.

2.4.3. Evolution-Oriented Surveys

A third line makes improvement itself the object of study. Tao et al. organize self-evolution around experience acquisition, refinement, updating, and evaluation at the level of large language models [Tao et al., 2024]. Gao et al. broaden the scope to self-evolving agents and ask what, when, how, and where agents evolve [Gao et al., 2026]. Fang et al. propose a feedback-loop view over system inputs, agent systems, environments, and optimizers, while Xiang et al. distinguish model-centric, environment-centric, and model–environment co-evolution [Fang et al., 2025a, Xiang et al., 2026]. Agentic reinforcement-learning surveys further connect improvement to temporally extended interaction, partially observable environments, memory, tool use, and self-improvement [Zhang et al., 2026c].

These surveys are central because they make adaptation and improvement first-class topics. They typically organize evolution at the level of models, policies, or broadly defined agents. They less systematically separate which improvements occur through editable external surfaces, which require parameter-side consolidation, and which require a meta-level process that selects, schedules, or optimizes the improvement mechanism itself.

Our position. Prior surveys have already examined agents, harnesses, and self-evolution from complementary angles. Building on these lines, this survey studies post-deployment agent improvement at their intersection: the interaction between external harness surfaces, long-horizon and open-ended environments, and multiple evolution mechanisms, including human-aided revision, self-improvement, and meta-evolution. The contribution is a focused account of where deployed agent systems can change and how those changes may become durable.

Part II. Runtime Infrastructure for Experience Reuse

3. Skills: Experience Becomes Reusable Procedure

Skills are the mechanism by which procedural experience becomes reusable runtime structure. A skill externalizes know-how as a procedure that can be persisted in a library, invoked by a harness, and revised after deployment. In current agent systems, this procedure is increasingly packaged as a `SKILL.md`-anchored folder.[1] For deployed agents that reuse experience, this makes the skill layer a native update surface: new competence can be added by expanding or editing the skill library, without modifying the base model parameters. The difficulty is that experience does not automatically become a good skill. A useful skill must be specific enough to guide future action, general enough to transfer beyond the episode that produced it, and stable enough to remain trustworthy as the runtime changes. This chapter therefore asks how skill libraries are created, used, and evolved.

Recent systems such as MUSE-Autoskill couple on-demand skill creation, skill-level memory, management, unit-test evaluation, and feedback-driven refinement in one deployed loop [Lin et al., 2026a], indicating that the current frontier is beginning to manage the skill lifecycle as a persistent adaptation layer.

3.1. Skill Formal Definition

We define the external skill layer as a time-varying bank

$$S_t = \{\sigma_{1,t}, \ldots, \sigma_{n_t,t}\}, \qquad \sigma_{i,t} = \langle M, I, R, A \rangle.$$

Each $\sigma_{i,t}$ denotes the i-th skill artifact available to the agent at time t. The tuple aligns with the folder schema around which the Agent Skills community has converged: a required `SKILL.md` file with optional supporting resources [1]. M is the manifest that lets the runtime find the right skill, corresponding to the `SKILL.md` frontmatter and related metadata; I is the instruction body that tells the model how to perform the task, corresponding to the Markdown body of `SKILL.md`; R is the reference set that supplies additional knowledge when needed, corresponding to supporting documentation, schemas, examples, tests, or audit records, often placed under `references/`; and A is the artifact bundle that supports grounded execution, corresponding to executable or static resources such as `scripts/`, `assets/`, templates, data files, or helper code. This four-part decomposition describes the converged `SKILL.md` schema rather than all historical skill systems. Earlier systems, beginning with programmatic skill libraries such as Voyager, realized the same idea in less standardized forms [Wang et al., 2023]. Canonical `SKILL.md` systems consolidate this lineage by making the manifest, instructions, references, and artifacts explicit in one portable folder. This packaging pattern has already become operationally useful in production systems. OpenAI's mounted skill bundles and OpenClaw-style registries make skills installable, routable, scoped, and versioned runtime packages [Guo, 2026, OpenClaw Contributors, 2026, Xu et al., 2026a]; Anthropic's implementation emphasizes the context side of the same design, progressively disclosing metadata, `SKILL.md`, and supporting files only as they become relevant. [2]

For an agentic AI system, using and evolving skills is one of the most direct ways to improve performance without changing M_{θ_t}. Mounting a skill gives the runtime an explicit procedure for a recurring class of tasks, represented as a reusable $\sigma \in S_t$. In the notation of Section 2, deployment evidence updates the skill bank through

$$S_{t_i^+} = \Phi_S(S_{t_i}, z_i),$$

[1] https://agentskills.io/home
[2] https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

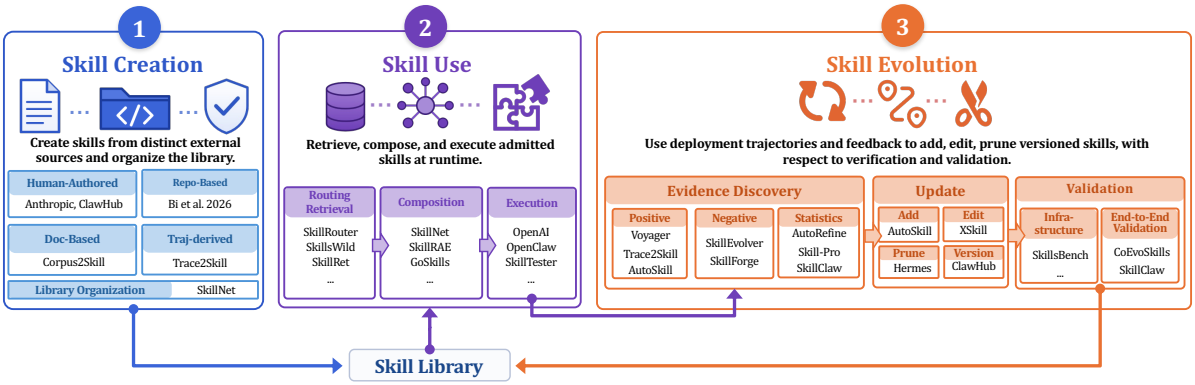

Figure 4 | External skill lifecycle used in this chapter. Skill Creation adds externally produced skills to the bank; Skill Use organizes, retrieves, composes, and executes the current bank; Skill Evolution converts compiled deployment experience into updates, producing an updated bank $S_{t_i^+}$ for the next use cycle.

where Φ_S may add or retire skills ($\pm\sigma_{n_i}$) or revise existing ones ($\sigma_{n_i,t} \to \sigma_{n_i,t^+}$) by changing their M, I, R, A components. The rest of the chapter asks how skill libraries are created, used, and evolved.

3.2. A Three-Stage Lifecycle for External Skills

For this chapter, the lifecycle of an external skill library is a closed loop of creation, use, and evolution. Creation adds vetted external procedures to the library. Use is the runtime step equipped with skills. In this stage, the harness decides which parts of the current library should shape a task and how those parts are carried out. Evolution starts after deployment evidence has accumulated: a failed run, a repeated workaround, or a successful trace can show that the next version of the library should change. The updated skill library is the natural output of evolution stage:

$$S_{t_i} \to \tau_i \xrightarrow{H_{t_i}} z_i \to S_{t_i^+} \to \tau_{i+1}.$$

Here τ_i, z_i, and $S_{t_i^+}$ follow the notation of Section 2. The skill-specific contents of z_i are unpacked only when we discuss evolution.

The three stages identify how experience enters, shapes, and revises the skill layer. The rest of the chapter follows this three-stage breakdown.

3.3. Skill Creation: From External Sources to Organized Library

Skill Creation studies how external or offline experience is converted into candidate procedures before the deployed harness begins using and updating the library. These sources include human-authored packages, repositories and workflows, documentation, marketplaces, and fixed demonstration trajectories. The stage ends when a procedure becomes a indexable skill artifact that can be checked and admitted into S_t. This boundary separates Creation from Skill Evolution: offline traces used for bootstrapping belong here, whereas traces generated by a deployed skill library and used to revise that library belong to Evolution.

Expert and product-authored skills. The most direct creation path is deliberate authoring: a domain expert or product developer writes a skill folder with a clear name, triggering description and instructions. They are often enriched by examples and optional scripts or templates. Anthropic's document skills and OpenAI's skill bundles both instantiate this pattern in production

settings, where procedural knowledge is packaged so the agent can discover it cheaply and load details only when needed [Guo, 2026, Xu et al., 2026a]. OpenClaw and ClawHub add the community distribution layer: users discover packages in a registry or marketplace, install them into a local runtime, and later load selected skills by name or version [ClawHub Marketplace, 2026, OpenClaw Contributors, 2026]. Product-level systems therefore show that skill creation has moved beyond prompt crafting toward an open-source community and package ecosystem.

Source-grounded skill construction. A second creation path grounds skills in existing external sources rather than direct expert authoring. Repository and workflow mining extracts procedural structure from tested code bases or agent workflows, where routines are already embedded in scripts, tools, and execution conventions [Bi et al., 2026]. Corpus2Skill represents the document side, clustering a corpus and materializing the resulting structure as a hierarchical `SKILL.md/INDEX.md` directory [Authors, 2026]. More specifically, SkillForge gives the enterprise version: it mines workflows and tools from historical support tickets, retrieves domain references from internal technical documentation, and synthesizes them into a domain-specific skill [Liu et al., 2026e]. OpenSkill extends this source-grounded path by using external information to synthesize transferable skills, and further extracts independently checkable anchors to build proxy evaluations for refining these skills [Yan et al., 2026]. Thus external sources can provide both the content of a skill and a creation-time signal for testing whether the skill should enter the library.

Offline trajectories and demonstrations. Trajectory-derived candidates can also appear in Creation when the trajectories are offline demonstrations rather than feedback from the currently deployed library. One line of work treats trajectories as worked examples from which the system distills reusable procedure. SkillX uses rollouts from a stronger agent to extract planning, functional, and atomic skills, so the raw trace is compressed into a multi-level skill representation [Wang et al., 2026b]. The same creation reading applies to the bootstrapping side of Trace2Skill, AutoSkill, and MIND-Skill: fixed trajectory pools, historical conversations, or successful demonstrations can be distilled into initial skill artifacts [Li et al., 2026f, Ni et al., 2026, Yang et al., 2026d]. Once such traces are produced by a deployed skill library and used to revise that library, the operation belongs to Skill Evolution.

Together, these creation paths show where an initial skill library can come from: expert packages, repositories and workflows, documents and knowledge bases, and offline demonstrations. The final stage of skill creation is library organization.

Library organization. An active library must be more than a flat pile of files. One family of systems makes searching scalable by adding explicit structure over the library. Controlled scaling experiments give a library-side reason: when semantically similar skills stay in a flat pool, they compete for the same task states and make routing less reliable [Li, 2026]. The following works differ mainly in the dimension along which they organize the library. SkillNet organizes a 200K+ skill repository as a global relation graph: skills are connected by category, similarity, dependency, and composition relations [Liang et al., 2026a]. SkillX instead organizes skills by capability granularity, separating planning, functional, and atomic skills. Corpus2Skill follows a third dimension, preserving the document-derived structure through `SKILL.md` and `INDEX.md` files [Authors, 2026, Wang et al., 2026b]. These methods show that a library can be structured through global skill relations, capability granularity, or document-derived organization. In each case, the router receives intermediate structure instead of a single undifferentiated pool.

Once the library is organized and the entries are exposed, the problem shifts from constructing artifacts to making the right ones available at runtime, which is one of the central topics in Skill Use stage.

3.4. Skill Use: Retrieval, Composition, and Execution

Skill Use begins after skills have entered S_t. It asks how the runtime makes the current library operational for a concrete task: which skills are visible, which ones are exposed, how selected skills are combined, and whether the resulting procedure can run within the harness. The following paragraphs therefore move from library organization to routing, composition, and execution.

This stage matters because many reported gains assume that the right skill is already injected. Real deployments do not have this oracle: library scale helps only when the runtime can find, combine, and execute the relevant procedures under context and latency constraints.

Production runtimes expose the same problem. OpenClaw, OpenAI's shell-skill environment, and Hermes Agent must first present the runtime with a discoverable set of installed skills: metadata is indexed, bundles are placed at known paths, and loading scopes determine which entries are visible for the current session [Guo, 2026, Nous Research, 2026, OpenClaw Contributors, 2026]. This organization operation prepares the surface on which the router can decide what to expose for a task.

Routing, retrieval, and triggering. Routing is formally prior to composition. It decides whether the right skills are selected at all. In a large library, a useful skill can remain hidden, an irrelevant one can look plausible enough to be exposed, or near-duplicates can compete for the same task state. Product runtimes often implement this step as progressive disclosure rather than as a pure external top-k selector. In the Agent Skills specification, the agent first sees lightweight metadata, then opens the full `SKILL.md` and supporting files only when the skill appears relevant.[3] OpenAI's production guidance describes the same mounted-skill pattern: the runtime exposes skill metadata such as name, description, and path in the model context, and the model can then read the full skill bundle from that path when it decides to use it.[4] MUSE-Autoskill adopts the same metadata-first runtime pattern: the agent first sees a lightweight skill catalog and loads the full `SKILL.md` only when the skill appears relevant [Lin et al., 2026a]. By contrast, many academic retrieval pipelines still use an external retriever to choose the exposed skill set before the downstream agent starts execution.

Recent retrieval studies refine this picture by showing that routing becomes a real gap when encountering a large-scale library. SkillsWild quantifies the gap between ideal injection and realistic retrieval: force-loaded curated skills can outperform retrieval from a large mixed pool, so library growth can hide useful skills instead of exposing them [Liu et al., 2026f]. SkillRouter verifies this point: Showing metadata alone first, which is standard practice now, can be insufficient at registry scale; compact retrieve-and-rerank pipelines improve matching by using fuller skill text rather than only names and descriptions [Zheng et al., 2026]. MUSE-Autoskill pushes this further by attaching a separate skill-level memory file to each skill; later invocations can surface this memory alongside the static `SKILL.md` [Lin et al., 2026a]. So the tradeoff between retrieval performance and retrieval budget naturally appears. To evaluate skill retrieval performance alone, SkillRet then isolates long-form skill matching as a specific retrieval benchmark. It shows that general retrievers do not automatically solve skill selection and that skill-specific fine-tuning helps [Cho et al., 2026]. Routing therefore cannot be a thin metadata lookup once the library becomes large.

Composition. Composition starts after retrieval and turns selected skills into a task-level procedure. Recent systems do this through different structures, including library relations, capability levels, local skill groups, workflows, and compiled execution contexts. What they share is a compatibility problem: each skill must fit the assumptions, artifacts, side effects, and stopping conditions of the surrounding procedure. The cited systems differ mainly in what structure

[3] https://agentskills.io/home
[4] https://developers.openai.com/api/docs/guides/tools-skills

they use to make this compatibility problem manageable. SkillNet records dependency and `compose_with` relations at the library level; after retrieval, these relations can be restricted to the selected subset and used to form the runtime skill graph for workflow synthesis [Liang et al., 2026a]. GoSkills makes a similar move. Its local graph starts from an anchor skill and adds supporting skills connected by dependency, artifact or other relation; in the notation above, the exposed skills become $V_{q,t}$, while these typed relations provide candidate arcs in $A_{q,t}$ before the executor sees the prompt [Zeng et al., 2026b]. SkillRAE keeps selected skills as the nodes, attaches task-relevant subunits and rescued cues back to those nodes as local guidance for execution [Meng et al., 2026b]. These systems make composition visible before execution by turning selected skills into graph, group, or compiled-context objects. Thus routing is a retrieval problem over S_t, whereas composition is a planning and compatibility problem over the smaller set $S_{q,t}$. Composition is stricter because each skill changes the state seen by the next one. The composer must therefore preserve preconditions, intermediate artifacts, side effects, and termination criteria T_σ across the whole workflow.

The scaling problem is immediate: if the retrieved set has size $|S_{q,t}|$ and a task may require depth d, naive ordered search is $O(|S_{q,t}|^d)$; without routing, it would be $O(|S_t|^d)$. Routing reduces the base of this search by exposing a smaller candidate set. Composition addresses the exponent side: it prevent the agent from trying every length-d ordering, offering a plausible execution graph far below the naive $O(|S_{q,t}|^d)$ search. Together, retrieval and composition make search smaller and add semantic constraints before execution begins [Liang et al., 2026a, Liu et al., 2025c, Meng et al., 2026b, Wang et al., 2026b, Zeng et al., 2026b].

Execution under runtime constraints. Execution is where the composed plan meets the harness. A selected skill may need files, browser state, shell commands, credentials, or scoped memory. Production runtimes therefore implement execution controls: OpenClaw exposes install/load, scope, registry, and versioning primitives [OpenClaw Contributors, 2026]; OpenAI's shell environments mount skill bundles into controlled containers [Guo, 2026, Xu et al., 2026a]. Complementarily, academic works provide more fine-grained practice. SkillSmith compiles offline skill packages into boundary-guided runtime interfaces. Instead of exposing the entire `SKILL.md` and bundled assets as runtime context, it extracts operators, input-output contracts and other elements into a compact execution contract. The result shows reduced solve-stage token use, reasoning iterations, and latency on SkillsBench [Xu et al., 2026b].

Execution closes the Use stage: the library has value only if routed and composed skills can be invoked properly and provide performance gain in the real runtime. The traces produced by these invocations are the raw material for skill evolution.

3.5. Skill Evolution: From Deployment Evidence to Library Updates

Skill Evolution is where later use changes the library itself. Skill Creation can seed candidate procedures, whose value becomes clear only after retrieval and execution on new tasks. Without revision, a growing library accumulates brittle triggers, stale assumptions, and overlapping procedures. Evolution uses deployment evidence to revise, merge, retire, or validate skills before future runs inherit them.

Here we specialize the Section 2 symbol z_i: for skill evolution, z_i is the compiled evidence consumed together with the current library S_{t_i}, not the raw log alone. The unified update remains $S_{t_i^+} = \Phi_S(S_{t_i}, z_i)$. And Validation stage checks if the updates are actually useful. SkillOpt provides a compact example of the full evolution loop: scored rollouts define the evidence, an optimizer model proposes bounded add/delete/replace edits to the skill document, and a held-out validation gate admits only edits that improve the current skill [Yang et al., 2026b]. SkillOS extends this loop from single-skill editing to learned repository-level curation [Ouyang et al., 2026]. It freezes

the agent executor and trains a separate skill curator to operate on a Markdown SkillRepo through `insert_skill`, `update_skill`, and `delete_skill`, using grouped task streams so that earlier repository edits are rewarded by their effect on later related tasks. The sections below unpack the evolution process and dig deeper into each stage.

Evidence discovery. The first step is to identify trajectory evidence that is worth turning into library updates. Successful or human-preferred trajectories are the clearest source: they show procedures that have already worked, or that users repeatedly steer the agent toward. Voyager gives the earliest concrete example in this line, accumulating executable JavaScript skills from successful Minecraft experience and retrieving them for later tasks [Wang et al., 2023]. Trace2Skill extends the same evidence logic from a single agent history to execution traces processed by sub-agents, which extract trajectory-local lessons and consolidate them into a skill directory [Ni et al., 2026]. AutoSkill adds a more user-facing source, where recurring queries and interaction traces are distilled into personalized reusable skills [Yang et al., 2026d]. Further, SkillsVote decomposes execution trajectories into fine-grained skill-linked subtasks and admits only successful, reusable, and attribution-supported discoveries to library updates [Liu et al., 2026b]. This makes success an attributed evidence source rather than a raw trajectory-level signal. Together, these works show that positive trajectories and repeated human preferences can reveal stable routines worth externalizing into the library.

Negative trajectories are equally important because they expose quality gaps in the existing library. However, failure is useful only when an analytic pipeline turns it into a specific update target. Work on negative traces therefore follows two lines: producing informative failure trajectories, and transforming those trajectories into usable evidence for skill evolution. SkillEvolver mainly strengthens the first line: fresh downstream agents load a candidate skill, and their traces expose deployment-specific failures that the authoring agent's own exploration may miss [Zhang et al., 2026b]. SkillForge operationalizes the second line through a failure analyser, a skill diagnostician, and a skill optimiser [Liu et al., 2026e]. It decomposes bad cases into knowledge, tool-use, clarification, and style dimensions, then maps recurring patterns to concrete defects in `SKILL.md`. EvoSkill connects the two lines by recording failed traces together with the current skill inventory and ground-truth answers, so the proposer can distinguish uncovered capabilities from failures of existing skills [Alzubi et al., 2026]. Together, these works show how negative trajectories become localized evidence for library updates rather than undifferentiated pass/fail labels.

A third source is evidence that appears only after skills become long-lived shared assets. This evidence is not a new kind of trajectory, since trajectories are either success/high score or failure/low score; the difference is the aggregate signal/statistics produced by repeated use. AutoRefine tracks the effectiveness, frequency, and precision of accumulated experience patterns, so recurring but low-value or redundant patterns can be pruned or merged [Qiu et al., 2026]. Memento-Skills and Skill-Pro similarly attach running utility, reward, or failure attribution signals to skills across repeated executions, making library maintenance depend on usage history rather than on a single episode [Mi et al., 2026, Zhou et al., 2026b]. SkillClaw adds the collective setting: trajectories from multiple deployed agents and users are aggregated by referenced skill, so repeated successes define stable behavior while repeated failures expose shared correction targets [Ma et al., 2026b].

At this scale, evidence discovery has three channels: successful or preferred trajectories identify routines worth preserving, failure trajectories locate defects or missing capabilities, and repeated or shared use supplies statistical evidence about which entries should keep influencing the library. These channels provide the empirical basis for the update operations discussed next.

Update operations. Once evidence is discovered, the update should be described by how it changes S_t. The first operation is addition. It addresses a coverage gap: the library lacks a procedure that appears repeatedly in successful traces, user requests, or diagnosed failures.

Trace2Skill answers this through horizontal consolidation: many trajectory-local patches are compared with one another, deduplicated, conflict-resolved, and merged into a single unified skill directory [Ni et al., 2026]. AutoSkill adds a vertical repository comparison: the candidate is compared with nearby entries in the current SkillBank, so add is proposed only when the candidate remains a distinct durable capability rather than a merge or discard case [Yang et al., 2026d].

The second operation is editing. It addresses a quality gap: the library already contains a relevant skill, but something in the entry makes it fail in use. SkillOS shows that repository evolution naturally shifts toward this operation: during training, insertion dominates early, while updates become more frequent as the curator moves from expanding the repository toward refining and consolidating existing skills [Ouyang et al., 2026]. Other systems make editing more conservative. SkillForge contributes precise and well-evidenced edits: its diagnostician maps recurring failure patterns to concrete locations inside `SKILL.md` or reference files, and each proposal carries evidence, expected impact, and risk [Liu et al., 2026e]. XSkill adds a different guardrail by separating task-level skills from context-sensitive experiences. Local experiences can be updated without immediately rewriting the skill; a modify operation on skill is proposed only when multi-path rollouts consistently show necessity [Jiang et al., 2026a]. Editing is therefore the operation for repairing an existing abstraction only after evidence is strong enough to distinguish a durable defect from a temporary experience.

The third operation is pruning and retirement. It addresses a pollution gap: a growing library can become increasingly harder to route through. AutoRefine proposed a statistic-driven stale strategy: if a pattern is retrieved often but rarely used, or used without improving success, the system treats it as a candidate for pruning [Qiu et al., 2026]. Hermes Agent represents the production side of the same operation. It exposes mature lifecycle actions such as create, update, delete, and archive-style handling, showing that industrial skill products already treat retirement as a standard library operation [Nous Research, 2026]. The two works therefore cover different sides of retirement: AutoRefine explains how stale skills can be identified, while Hermes shows which maintenance actions a deployed skill system must expose.

The fourth operation is synchronization, versioning, and rollback. It addresses a deployment gap: Since skill library becomes a shared source, an mechanism must be specified. OpenClaw and ClawHub expose registry, install/load, version, and distribution surfaces [ClawHub Marketplace, 2026, OpenClaw Contributors, 2026]. These systems show that update proposals can target library state and distribution metadata, not only the text of a single skill.

Validation and admission of updates. Validation tests whether the updated library is actually useful. SkillsBench and SWE-Skills-Bench make this requirement visible at benchmark scale: skills can improve agents, but self-generated or mismatched skills can also cause negative transfer [Han et al., 2026, Li et al., 2026c]. This has already led to basic evaluation infrastructure. SkillLens further evaluates the full trajectory-to-skill pipeline across experience generation, skill extraction, and skill consumption. It shows that generated skills often help but can still produce substantial negative transfer, and that a model may be a strong extractor yet a weak skill consumer, or vice versa. Validation therefore has to be end-to-end, testing an extracted skill in the target model and domain where it will actually be consumed [Huang et al., 2026].

SkillTester, for example, compares with-skill execution against a baseline execution [Wang et al., 2026h]. Although its original comparison is skill versus no skill, the same end-to-end idea can be reused for updates by running the old and updated libraries on the same tasks. EvoSkill implements a standard version of this validation method: candidate programs containing new or edited skills are kept only when they improve the held-out validation frontier [Alzubi et al., 2026].

Not every validation signal has to be end-to-end task score. MIND-Skill checks whether a trajectory-induced skill preserves transferable procedural knowledge without dropping key steps or memorizing instance-specific details [Li et al., 2026f]. In this lifecycle, however, the central admission question is still task-level utility after an update.

Recent systems therefore focus on making end-to-end validation more effective and reliable. SkillEvolver observes that a revised skill can look useful in the authoring context while leaking training-instance details, relying on private session context. Its fresh-session auditor checks these failure modes before the revised artifact is evaluated on the held-out split [Zhang et al., 2026b]. CoEvoSkills addresses a different problem: ground-truth task scores are authoritative but sparse, so pass/fail alone gives insufficient signal. Its Surrogate Verifier supplies denser diagnostic feedback before escalation to the ground-truth oracle, while the paper also shows the remaining surrogate–oracle gap [Zhang et al., 2026e]. SkillClaw moves validation closer to user-side deployment. Its candidates are collective updates derived from cross-user trajectories, and are evaluated in a more realistic style: evaluation occurs at night in idle user environments, with the full toolchain in use [Ma et al., 2026b]. These systems keep the evolution loop from treating any proposed update as an improvement: the update must survive a validation procedure appropriate to the setting in which the library will be used.

Summary. Across the lifecycle, skills provide the harness with an external and editable layer of procedural competence. Skill Creation turns expert knowledge, documents, demonstrations, and trajectories into initial library entries; Skill Use makes those entries available through organization, routing, composition, and execution; Skill Evolution converts deployment evidence into proposed library changes and admits only those that pass validation. This lifecycle changes agent behavior without requiring an immediate model-weight update, but they remain useful only when the library is continuously organized, tested, and maintained.

4. Memory: Experience Becomes Persistent State

4.1. Agent Memory as Context-Mediated Persistence

Agent memory keeps task-relevant information available across model calls, sessions, and environments. In deployed agents, this is inseparable from context management: the system must decide what past information enters the active context, what remains retrievable outside it, and what should be compressed, revised, or forgotten [Verma, 2026]. This distinguishes agent memory from ordinary retrieval. RAG usually retrieves from a relatively static corpus[Lewis et al., 2020], whereas agent memory is produced, edited, and reused by the agent itself. Generative Agents showed that stored observations and reflection can shape long-horizon behavior [Park et al., 2023], and xMemory further treats memory as self-authored, revisable, and action-coupled [Hu et al., 2026]. The key object is therefore a context-mediated persistence system: past information is stored outside the model, transformed into useful forms, and selectively reintroduced into context.

Memory is also different from skill. Skills specify reusable procedures for how to do something again [Jiang et al., 2026b]. Memory keeps state, evidence, and history: what happened, what changed, what failed, or what a user preferred. A trajectory can produce both. If it becomes a named procedure, it belongs to the skill layer; if it remains valuable as situated evidence, it belongs to memory.

Most current agent-memory systems are context-mediated. Information lives in files, logs, vector stores, graphs, summaries, or memory managers, then returns through retrieval or context injection. This chapter focuses on this external memory layer because it is inspectable, editable, and able to change during deployment. Figure 5 summarizes this view: memory is not a single

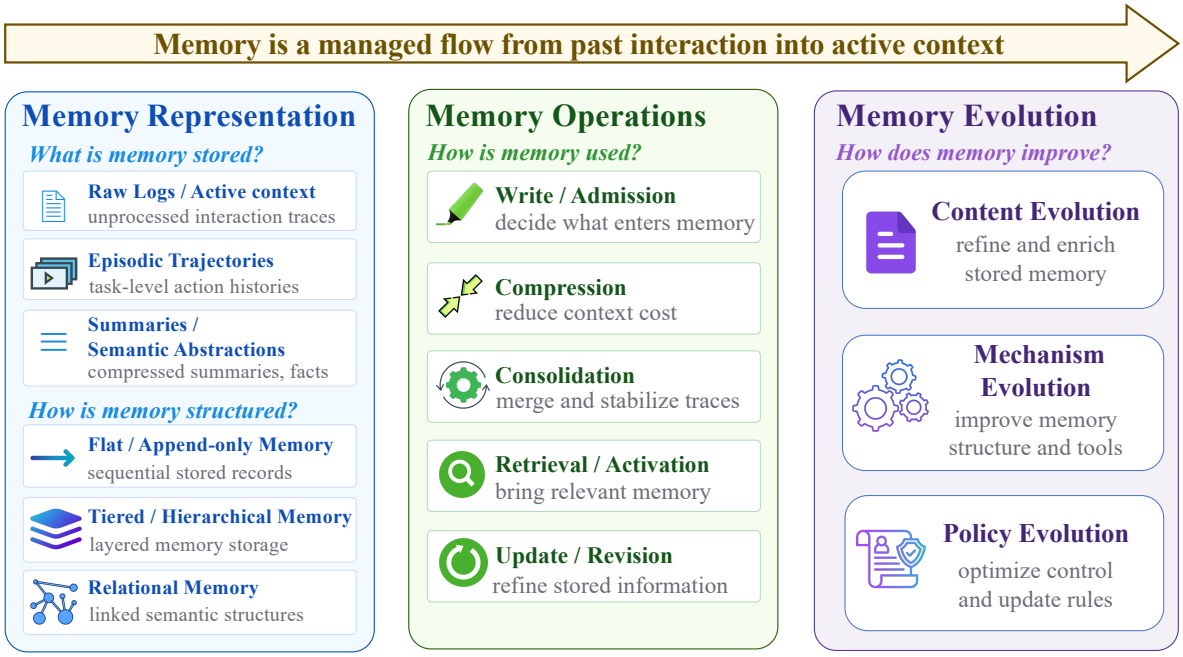

Figure 5 | Context-mediated memory management and evolution. Memory representations define what is stored, memory operations decide how stored information is written, compressed, retrieved, updated, and reintroduced into active context, and memory evolution captures how the agent improves memory content, memory policies, and meta-memory over time.

store, but a managed flow from past interaction into active context through representation, operations, and evolution. Table 2 lists representative works under this same structure, and the rest of the chapter proceeds through each aspect in turn.

Topic	Representative works
	Memory Representation
	Memory Content Units
Raw logs / active context	Empowering Working Memory [Guo et al., 2023]; Hangman [Baldelli et al., 2026]; Active Context Compression [Verma, 2026]; TierMem [Zhu et al., 2026a]
Episodic trajectories	WebCoach [Liu et al., 2025a]; Live-Evo [Zhang et al., 2026n]; CAST [Ma et al., 2026a]; MemRL [Zhang et al., 2026k]
Summaries / semantic abstractions	SimpleMem [Liu et al., 2026c]; Memori [Borro et al., 2026]; Chronos [Sen et al., 2026]
	Memory Organization Structures
Flat / append-only memory	Generative Agents [Park et al., 2023]; Reflexion [Shinn et al., 2023]; Cognis [Daftari et al., 2026]; APEX-MEM [Banerjee et al., 2026]
Tiered / hierarchical memory	MemGPT [Packer et al., 2023]; LightMem [Zhang et al., 2026h]; Agentic Memory [Yu et al., 2026b]; CLAG [Roh et al., 2026]
Relational / associative memory	PlugMem [Yang et al., 2026a]; AriadneMem [Zhu et al., 2026b]; GAAMA [Paul et al., 2026]; MemORAI [Pham Van et al., 2026]; SAGE [Wang et al., 2026g]; H-MEM [Yu et al., 2026a]
	Memory Operations
Write / admission	AMAC [Zhang et al., 2026d]; Agentic Memory [Yu et al., 2026b]; U-Mem [Wu et al., 2026c]
Compression	Active Context Compression [Verma, 2026]; SimpleMem [Liu et al., 2026c]; Memori [Borro et al., 2026]; DeMem [Zou et al., 2026]
Consolidation	WebCoach [Liu et al., 2025a]; A-MEM [Xu et al., 2025b]; LightMem [Zhang et al., 2026h]; Auto-Dreamer [Ye et al., 2026b]; GAAMA [Paul et al., 2026]; CLAG [Roh et al., 2026]
Retrieval / activation	xMemory [Hu et al., 2026]; Chronos [Sen et al., 2026]; AriadneMem [Zhu et al., 2026b]; TierMem [Zhu et al., 2026a]; MemORAI [Pham Van et al., 2026]; Cognis [Daftari et al., 2026]
Update / revision	Live-Evo [Zhang et al., 2026n]; STALE [Chao et al., 2026]; MemoRepair [Zhao et al., 2026b]; APEX-MEM [Banerjee et al., 2026]
	Memory Evolution
Content evolution	SimpleMem [Liu et al., 2026c]; Memori [Borro et al., 2026]; Chronos [Sen et al., 2026]; Live-Evo [Zhang et al., 2026n]
Mechanism evolution	MetaMem [Xin et al., 2026]; MemSkill [Zhang et al., 2026f]; Self-Evolving Memory Extraction [Yang et al., 2026c]; SAGE [Wang et al., 2026g]; CLAG [Roh et al., 2026]
Policy evolution	Agentic Memory [Yu et al., 2026b]; MemRL [Zhang et al., 2026k]; DeltaMem [Zhang et al., 2026i]; U-Mem [Wu et al., 2026c]; AMAC [Zhang et al., 2026d]

Table 2 | Representative works organized by the three-layer structure of Figure 5.

4.2. Memory Representation: What Is Stored and How Is It Organized?

Representation determines what later operations can do. It is useful to separate it into two related questions: what units are stored, and how those units are organized for later retrieval and update. The first concerns the content unit, or what the system stores as a memory item. The second concerns the organization structure, or how stored items are arranged for retention, retrieval, and update. Recent memory systems are therefore best read not only through working, episodic, or semantic-memory analogies, but through this two-part design space.

4.2.1. Memory Content Units

The first question is what unit the system stores. Raw logs and active context keep messages, tool outputs, observations, actions, and intermediate state in high-fidelity form. Their advantage is evidence preservation; their weakness is low density. Work on working memory frames this as a control problem rather than transcript accumulation [Guo et al., 2023], and Hangman-style failures show that agents may need explicit task state even when the dialogue history is available [Baldelli et al., 2026]. Recent systems also treat raw context as a substrate to be managed rather than simply retained: Active Context Compression prunes and compresses active interaction history, while TierMem keeps raw logs available as a lower-level source that can be revisited when summaries are insufficient [Verma, 2026, Zhu et al., 2026a].

Episodic trajectories store temporally grounded sequences of actions, observations, and outcomes. Their value lies in order and consequence: which attempt failed, which correction followed, and what state transition mattered. WebCoach keeps cross-session navigation experience and retrieves prior trajectories as task-specific advice [Liu et al., 2025a]. Live-Evo lets episodic records evolve under feedback [Zhang et al., 2026n], while CAST makes the unit itself more explicit by organizing memory around character-and-scene events and derived character profiles [Ma et al., 2026a]. This style is especially useful when future action depends on history rather than isolated facts; EMemBench reflects that need in evaluation [Li et al., 2026d].

Summaries and semantic abstractions both belong to the same general move away from raw history toward abstracted state. The difference is mainly in how explicit the abstraction becomes. Summaries are the freer form: at the simplest end, they are natural-language compression of long interaction history into shorter task digests, reflection blocks, or context summaries. Semantic facts, profiles, and events are a more structured form of the same idea: instead of retaining a prose summary, the system compresses history into more explicit state representations such as facts, triples, profiles, or event tuples. SimpleMem extracts structured memory units from interaction history [Liu et al., 2026c], Memori rewrites dialogue into triples plus summaries [Borro et al., 2026], and Chronos adds dated event tuples and calendars [Sen et al., 2026]. We therefore group them together because they form a natural progression from raw traces, to summarized traces, to distilled semantic state. Across these cases, the key trade-off is density versus fidelity.

4.2.2. Memory Organization Structures

The second question is how stored units are organized. Flat or append-only memory is the simplest structure: records are added over time with little explicit relation between them beyond order, recency, or metadata. This makes writing cheap and generic, but leaves later retrieval to reconstruct structure on demand. Observation streams and reflection logs often follow this pattern even when their content units differ [Park et al., 2023, Shinn et al., 2023]. Recent conversational-memory systems keep this append-only substrate while adding retrieval-time interpretation: Cognis stores full historical records with version tracking, and APEX-MEM explicitly preserves the temporal evolution of information through append-only storage [Banerjee et al., 2026, Daftari et al., 2026].

Tiered or hierarchical memory organizes records into layers with different access roles, costs, or time scales. MemGPT treats memory partly as a paging problem across prompt-visible and external tiers [Packer et al., 2023]. LightMem similarly separates active and longer-term stores under latency constraints [Zhang et al., 2026h]. Agentic Memory couples short-term and long-term stores under a learned controller [Yu et al., 2026b]. CLAG sits between flat archives and explicit graphs by partitioning memories into evolving semantic clusters rather than adding rich typed relations [Roh et al., 2026]. Here the main benefit is not richer lateral relations between items, but better control over what stays active, what is compressed, and what is archived.

Relational or associative memory gives stored units explicit links to one another. Those links may be relatively weak, as in neighborhoods or associations, or stronger, as in multi-relation graphs over facts, episodes, reflections, and concepts. PlugMem and AriadneMem use graph structure to support long-range retrieval and bridge discovery [Yang et al., 2026a, Zhu et al., 2026b]. GAAMA builds associative graphs over heterogeneous memory nodes [Paul et al., 2026], MemORAI adds provenance-aware multi-relational graph retrieval [Pham Van et al., 2026], and SAGE turns that graph organization into a self-evolving associative memory engine [Wang et al., 2026g]. The central advantage is that the system stores not only individual memories, but also how they relate. H-Mem extends this direction with a temporal-semantic tree for consolidation and an entity graph for multi-hop retrieval [Yu et al., 2026a].

4.3. Memory Operations: How Is Memory Processed and Used?

Operations determine whether stored memory affects future behavior. The same operation varies by representation: writing may mean placing task state in context, admitting an episode to a store, or extracting a graph edge. The main operations are write/admission, compression, consolidation, retrieval/activation, and update/revision.

4.3.1. Write and Admission

The write path determines memory quality before retrieval begins. Append-only storage accumulates redundant, stale, or malformed records. Adaptive Memory Admission Control treats writing as an explicit utility decision using factors such as future utility, confidence, novelty, recency, and content type [Zhang et al., 2026d]. U-Mem turns memory growth into an active acquisition-and-validation process rather than passive accumulation [Wu et al., 2026c]. Agentic Memory turns store, retrieve, update, summarize, and discard into policy actions over short-term and long-term memory [Yu et al., 2026b]. More broadly, systems such as Nemori, SimpleMem, and Memori show that write quality also depends on how candidate memory units are formed before admission [Borro et al., 2026, Liu et al., 2026c, Ma et al., 2025].

4.3.2. Compression

Compression increases context density by reducing the cost of carrying forward past interaction. In working memory, compression preserves task state while removing repetitive or oversized traces [Verma, 2026]. In semantic memory, it extracts facts, triples, profiles, or events, as in SimpleMem and Memori [Borro et al., 2026, Liu et al., 2026c]. DeMem reframes compression through a decision-theoretic rate-distortion objective rather than summary quality alone [Zou et al., 2026].

4.3.3. Consolidation

Consolidation goes beyond shortening. It merges related records, abstracts repeated patterns, and sometimes converts episodes into guidance. A-MEM treats memory as evolving over time

through linking, merging, and revision [Xu et al., 2025b]. LightMem adds an explicit offline long-term synthesis stage [Zhang et al., 2026h]. Auto-Dreamer makes this offline path explicit by learning a consolidator that decouples fast per-session memory acquisition from slower cross-session abstraction and replacement [Ye et al., 2026b]. GAAMA consolidates by synthesizing higher-order reflections over graph structure, while CLAG consolidates locally inside clusters through profile generation and neighborhood-level evolution [Paul et al., 2026, Roh et al., 2026]. The open problem is deciding which abstractions should be merged into a more stable memory and which should remain separate traces.

4.3.4. Retrieval and Activation

Retrieval decides which memory affects the next step. In active context, this can be as simple as keeping the right state visible. In external stores, retrieval requires selecting records, choosing an injected form, and budgeting context. xMemory retrieves top-down across messages, episodes, semantics, and themes rather than relying only on flat similarity [Hu et al., 2026]. Chronos scopes retrieval by time [Sen et al., 2026], AriadneMem uses graph bridge discovery when evidence is distributed [Zhu et al., 2026b], TierMem escalates from summaries to raw logs when summary evidence is insufficient [Zhu et al., 2026a], and MemORAI adapts graph traversal weights to the query [Pham Van et al., 2026].

Retrieval is also a planning and systems problem. Cognis couples retrieval with version tracking over full historical records [Daftari et al., 2026], making activation depend not only on similarity but also on the current validity of stored evidence.

4.3.5. Update and Revision

Persistent memory must handle change. Preferences shift, facts become outdated, plans fail, and summaries become misleading. A memory may be correct during one interval and wrong later, so update mechanisms must handle implicit invalidation as well as explicit correction. Live-Evo reweights memories online from feedback, STALE shows that current systems often fail to detect implicit invalidation, MemoRepair formalizes cascade repair over derived artifacts, and APEX-MEM resolves conflicting or evolving evidence at retrieval time [Banerjee et al., 2026, Chao et al., 2026, Zhang et al., 2026n, Zhao et al., 2026b].

4.4. Memory Self-Evolution and the Next Frontier

Memory itself becomes an update surface when the agent does not merely read stored records, but also changes the memory system that future runs will inherit.

Content Evolution means the memory content itself changes. The agent writes new experiences, compresses traces into summaries, and updates stale records. Representation gains come from structured semantic compression, temporal records, and online episode weighting [Borro et al., 2026, Liu et al., 2026c, Sen et al., 2026, Zhang et al., 2026n]. Operation gains come from better admission, offline consolidation, provenance-aware retrieval, and revision [Ye et al., 2026b, Zhang et al., 2026d, Zhao et al., 2026b, Zhu et al., 2026a]. This is the most direct external path for post-deployment improvement.

Mechanism Evolution means the agent improves how memory is organized, extracted, and integrated. MetaMem makes this distinction explicit by separating factual memory from meta-memory: factual memory stores task information, while meta-memory stores reusable experience about how to integrate, retrieve, and reason over memory fragments [Xin et al., 2026]. MemSkill treats memory extraction, integration, retention, and forgetting as reusable memory skills selected by a controller [Zhang et al., 2026f]. Self-Evolving LLM Memory Extraction narrows the loop to

the memory extractor itself, continuously improving prompts for extracting useful memory across heterogeneous tasks [Yang et al., 2026c]. SAGE closes the loop between a memory reader and writer in a self-evolving graph-memory engine, while CLAG shows a lighter-weight version where the organization mechanism itself adapts through clustering and local evolution [Roh et al., 2026, Wang et al., 2026g].

Policy Evolution means the system learns when and how to operate on memory. An always-on agent needs triggers for when to write, retrieve, revise, or merge; credit assignment for whether a memory actually helps later action; versioning and rollback for bad memory updates; and evaluation protocols that test memory-conditioned behavior rather than recall alone. Learned controllers such as AgeMem already turn memory operations into policy actions [Yu et al., 2026b], while MemRL, DeltaMem, U-Mem, and AMAC show complementary routes for learning retrieval utility, update rewards, active acquisition, and admission control over memory records [Wu et al., 2026c, Zhang et al., 2026d,i,k].

Taken together, these three levels suggest why memory is the main substrate for deploy-time agent improvement. Memory lets an agent adapt after deployment without changing model weights: it can accumulate experience, compress it into reusable abstractions, consolidate recurring patterns, revise stale or conflicting state, and learn better policies for what to keep active or retrieve later. In that sense, memory provides a practical online experience loop: acting produces traces, memory operations distill those traces into a better store, and later behavior is conditioned on the improved store. The promise is continuous adaptation under real deployment constraints; the risk is that poor abstraction, unstable revision, or uncontrolled accumulation can also amplify error over time.

Current evaluation only partially captures this loop. Existing frameworks such as LongMemEval, MemoryArena, Mem2ActBench, and EMemBench already test sustained recall, multi-session dependency, memory-conditioned action, and episodic continuity [He et al., 2026, Li et al., 2026d, Shen et al., 2026a, Wu et al., 2024a]. These are useful for evaluating memory-enabled agents, but they are still weak for self-evolving agents: they say relatively little about whether memory policies improve over time, whether consolidation leads to better future decisions, whether revision remains stable after repeated updates, or whether improvements persist over long deployment horizons. A key frontier is therefore evaluation for longitudinal adaptation rather than one-shot memory use.

This section therefore hands off naturally to the later meta-memory and meta-evolving-agent discussion. In this chapter, memory self-evolution mainly means improving the content and local operations of an agent's own memory. In the meta-memory setting, the object of improvement becomes the memory mechanism itself: the rules, extractors, controllers, evaluation signals, and update policies that decide how future memories will be formed and used. The central open question is whether an agent can maintain a memory that becomes more selective, better organized, and more trustworthy over time without silently amplifying outdated, biased, or poisoned experience.

5. Environment: The Boundary of What Agents Can Experience

The previous chapters argued that agentic systems improve after deployment by rewriting procedures, accumulating memory, and repeatedly adapting to the runtime they inhabit. That immediately raises a systems question: how much can such experience-driven adaptation achieve if the world exposed to the agent is itself thin? In practice, the environment is often the binding constraint on runtime adaptation. A stronger harness can only exploit opportunities that the environment actually exposes. If the action surface is narrow, feedback is sparse, and task episodes are short or weakly stateful, then even sophisticated improvements are likely to remain local.

Current systems have made substantial progress in building external environments that agents can act in and receive feedback from, but much less progress in making that feedback dense, attributed, and stable enough to train on. The chapter follows that progression: executable environments first, reusable boundaries second, and learnable environments as the bridge to the RL-centered parameter path in Section 6 [Harbor Framework, 2026, HKUDS, 2026a, Merrill et al., 2026, Xie et al., 2024, Zhou et al., 2023].

5.1. Environment as Self-Improvement Infrastructure

A useful starting point is to separate three concepts that are often collapsed in the agent literature. The **environment** is the external world in which the agent acts. The **execution harness** is the engineering layer that exposes that world through permissions, logging, replay, verification, and interfaces. In this chapter, the interface discussion is part of the harness: commands, APIs, GUI affordances, resource adapters, and verifier hooks are the environment-facing boundary of the harness rather than a separate topic. The **benchmark suite** is then the task collection and evaluation protocol built on top of a particular environment–harness pair. This distinction matters because Section 8 is mainly about measurement infrastructure, whereas the present chapter is about experience infrastructure: the conditions under which deployment yields reusable trajectories rather than isolated outcomes. The object studied here is the environment together with the environment-facing part of the harness: the layer that presents the external world to the agent as an executable and auditable system.

This is the sense in which the environment can be treated as an upper-bound variable E. The following notation is not a theorem about optimality or convexity; it is a compact way to express the constraint intuition. Let $\mathcal{A}_{LM}$ denote the model's nominal action space. An environment does not expose that full text-action space. Instead, it induces an executable subset

$$\mathcal{A}(E) \subseteq \mathcal{A}_{LM},$$

consisting only of actions that can actually be parsed, grounded, permitted, and state-changing under the interface and execution constraints of E. Let $\Pi(E)$ denote the corresponding executable-policy class. If

$$J(\pi; E) = \mathbb{E}_{\tau \sim P(\tau \mid \pi, E)}[R(\tau)],$$

where π denotes a runtime policy, τ a trajectory induced by executing that policy in environment E, $P(\tau \mid \pi, E)$ the corresponding trajectory distribution, and $R(\tau)$ the trajectory-level return, then the best reachable value under E is constrained by feasible-set inclusion:

$$J^\star(E) := \sup_{\pi \in \Pi(E)} J(\pi; E) \;\leq\; \sup_{\pi \in \Pi_{LM}} J(\pi; E).$$

The inequality should be read as a formalized intuition: prompting, memory, or harness-side control logic may improve search within the executable region, but they do not by themselves expand that region. Enlarging the practical ceiling requires changes to the environment-facing surface: action space, observability, feedback, or verification. Figure 6 summarizes this environment view of runtime adaptation: the environment-facing boundary determines what the agent can execute, observe, verify, and convert into reusable experience.

Recent executable-environment work makes three analytic axes visible, rather than establishing a fixed taxonomy. **Action diversity** concerns the breadth of grounded actions available to the agent and the extent to which those actions can alter real external state [Drouin et al., 2024, Le Sellier de Chezelles et al., 2025, Trivedi et al., 2024, Xie et al., 2024, Zhou et al., 2023]. **Feedback density** concerns whether the environment returns informative signals beyond unconstrained text output, such as execution errors, state-based tests, verifier outputs, or reward instrumentation [Harbor

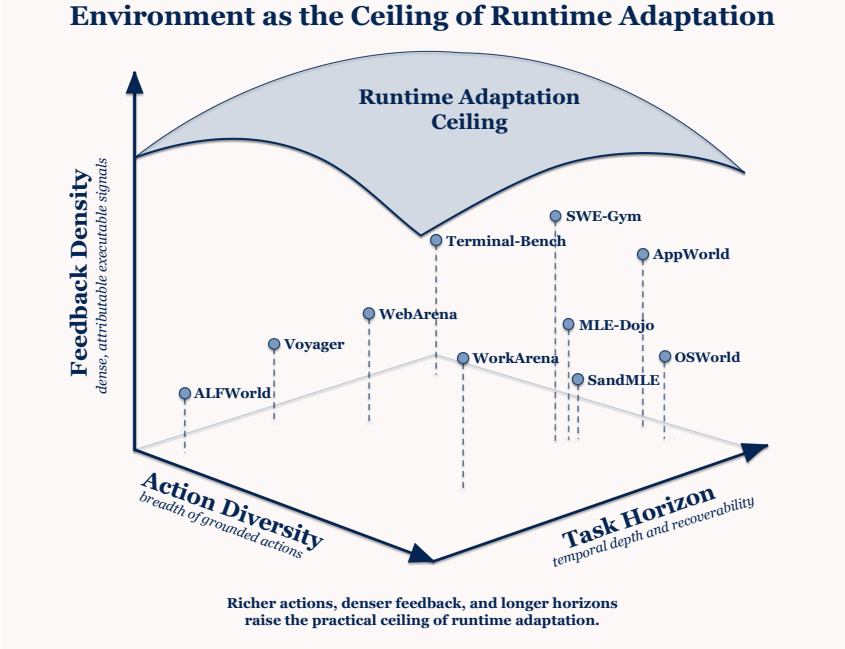

Figure 6 | Environment as the ceiling of runtime adaptation. A deployed agent can improve through prompting, memory, skills, and harness-side control only within the executable world exposed by the environment. Raising the practical ceiling therefore requires richer action surfaces, denser observations and feedback, longer stateful episodes, and verifiers that can turn interaction traces into usable experience for later adaptation or training.

Framework, 2026, Merrill et al., 2026, Trivedi et al., 2024, Wang et al., 2023]. **Task horizon** concerns the temporal depth and statefulness of the tasks supported by the environment, including whether intermediate errors are observable and recoverable [Merrill et al., 2026, Wang et al., 2023, Xie et al., 2024, Zhou et al., 2023]. Environments with narrow action surfaces, mostly final pass/fail verification, and short horizons can still support prompting or one-shot tool use, but they provide a limited substrate for accumulating reusable experience.

These dimensions are analytically separable but empirically coupled. Increasing action diversity without improving observability yields richer but harder-to-diagnose trajectories. Extending task horizon without denser feedback makes tasks more realistic while making failure attribution substantially more difficult. Conversely, artificially dense feedback in a simplified environment may ease optimization while reducing ecological validity. Thus these axes are best treated as design tensions, not as independent metrics that can be maximized in isolation.

This framing is consistent with the trajectory of recent environment research. ALFWorld established an early interactive setting with grounded action and delayed success criteria [Shridhar et al., 2021]. Voyager showed that an open-ended world can support iterative skill acquisition rather than only single-task execution [Wang et al., 2023]. WebArena and WorkArena moved toward realistic browser and knowledge-work settings, thereby increasing both action diversity and task horizon [Drouin et al., 2024, Le Sellier de Chezelles et al., 2025, Zhou et al., 2023]. OSWorld extended the action surface to general computer use, while AppWorld emphasized stateful transitions across a controllable app ecosystem [Trivedi et al., 2024, Xie et al., 2024]. Taken together, these systems show that the important question is not simply which benchmark is hardest, but which environments most enlarge the learning surface available to a deployed agent.

Seen against these requirements, world models are most relevant here when they turn executable software state into a substrate for multi-step rollout rather than only next-step prediction. This digital-world reading is the part of Agentic World Modeling that most directly connects to agentic

harnesses [Chu et al., 2026]. WorldCoder illustrates one route by constructing executable code world models through interaction with the environment [Tang et al., 2024], while world-model-augmented web agents show a related route in which an explicit world model is used to support action correction during interactive web execution [Shen et al., 2026b]. Neural Computers pushes in the complementary direction by asking whether elementary CLI and GUI interaction dynamics can be internalized into a learned runtime state from I/O traces alone, effectively amortizing part of the environment into a model of execution [Zhuge et al., 2026]. These lines differ in where the "world" resides—externally synthesized, explicitly program-induced, or partially internalized—but they reinforce the same systems point: for advanced agents, the effective environment is increasingly a constructed object that can be generated, predicted, compressed, and reused rather than a passive backdrop. The remaining question is how to construct such environments from the software and workflows agents already need to operate.

5.2. Turning Software into Executable Environments

The first construction problem is executability: how software and workflows are turned into environments that an agent can actually run. Section 5.3 then turns to portability: how the exposed boundaries of those environments are made reusable across runtimes. Recent work can be organized into three overlapping routes: CLI and terminal worlds, verifiable software environments, and scalable environment synthesis.

CLI and terminal worlds are the most direct route because shells already expose discrete commands, textual observations, filesystem state, and scriptable tests. Terminal-Bench turns realistic command-line work into executable tasks with test-based verification [Merrill et al., 2026], while CLI-Gym studies scalable command-line task generation [Lin et al., 2026c]. CLI-Anything broadens the same idea from benchmark tasks to heterogeneous software packages: `SKILL.md` describes capabilities and failure modes, `HARNESS.md` specifies installation and runtime assumptions, and `TEST.md` records partial correctness checks [HKUDS, 2026a]. The stable chapter-level numbers used here are **28 software harnesses** and **2130 automated tests**, indicating that terminal-style execution can become a broad software access layer rather than a narrow menu of isolated tools.

Verifiable software environments move beyond command access toward operational workflows whose state changes can be checked, replayed, and audited. ResearchGym packages AI research workflows, DevOps-Gym packages the software DevOps cycle, and MEnvAgent treats environment construction itself as a verifiable systems problem [Garikaparthi et al., 2026, Guo et al., 2026a, Tang et al., 2026]. At the benchmark-suite level, executable benchmarking makes the evidence contract explicit by joining workload adapters, task manifests, event schemas, replay and freeze policies, declared drivers, verifier metadata, and reporting pipelines under a single auditable protocol [Zhong et al., 2026b]. The shift is from "the agent ran the task" to "the environment admits evidence about the run under stable rules."

Scalable environment synthesis addresses a different bottleneck: if every useful environment must be manually packaged, experience remains scarce. Agent-World pushes in this direction by synthesizing scalable real-world environments and verifiable tasks, then coupling environment growth with continuous self-evolving agent training [Dong et al., 2026a]. CLI-Gym, LiteCoder-Terminal-Gen, and MEnvAgent also fit this route when read as attempts to generate or construct new executable settings rather than only evaluate agents in fixed ones [Guo et al., 2026a, Lin et al., 2026c, Peng et al., 2026a]. This route matters because long-running deployment needs not only richer environments, but a supply of environments large and varied enough to produce repeated, comparable experience.

Across the three routes, the common pattern is a shift from exposing isolated tools toward exposing executable worlds. This is the first prerequisite for long-running self-improvement: task

procedures, traces, tests, and execution assumptions can be retained and compared only after the software has been made operational for agents. Whether those artifacts become reusable across hosts is a separate boundary problem, taken up next.

5.3. Protocolizing and Standardizing the Boundary

Executability creates a local runnable world, but protocolization determines whether that world can join a larger runtime ecosystem. Without shared boundary contracts, each tool, resource, trace schema, and verifier remains tied to a bespoke integration, so reuse scales with adapter engineering. With protocolized boundaries, new tools and environments can be discovered, authorized, composed, and logged through common interfaces, and experience can travel across hosts at lower marginal cost. Section 5.2 therefore asks how software becomes executable at all, whereas the present section asks how executable environments become scalable infrastructure.

The recent literature suggests a useful three-part decomposition. MCP standardizes the agent–tool and agent–resource boundary [Model Context Protocol, 2025]. A2A standardizes the agent–agent boundary and thereby supports delegation and multi-agent interoperability [A2A Protocol Contributors, 2026]. AG-UI standardizes the boundary between agent runtimes and user-facing applications [AG-UI Protocol Contributors, 2026]. These protocols solve different systems problems, and their importance in this chapter lies precisely in that differentiation: the execution boundary of a modern agent is no longer only a tool API boundary.

From the perspective of self-improvement, standardization primarily affects the reusability of post-deployment experience. With shared interfaces, a larger fraction of runtime knowledge becomes portable: procedures can be reused, failure modes can be compared, verifier logic can be adapted, and resource access patterns can be stabilized across environments. Standardization therefore raises the practical ceiling of self-improvement indirectly but materially, not by improving the model directly, but by improving how far local experience can travel.

The same portability issue reappears at a higher layer. MCP, A2A, and AG-UI standardize runtime interfaces, but benchmark evidence has its own boundary: task definitions, trace schemas, verifier conventions, and reporting formats also have to travel. CUBE addresses this benchmark-layer fragmentation by proposing a standard for unifying heterogeneous agent benchmarks under a shared protocol perspective [Lacoste et al., 2026]. Therefore, it does not replace runtime protocols. Instead, it shows that interoperability remains multi-layer: tool calls may be portable while evaluation traces are not, or benchmark reports may be comparable while the underlying execution hosts remain incompatible. Read together, these efforts clarify why executability and protocolization must be separated. The former exposes a world to the agent; the latter determines which parts of the resulting experience can be reused elsewhere with lower integration cost.

5.4. Executable and Reusable Is Still Not Learnable

This distinction leads directly to Section 6. An environment can be executable and even standardized without yet being learnable in the reinforcement-learning sense. What RL needs is not merely access. It needs episodes, verifiers, reward structure, and enough attribution to connect outcomes back to actions. This is why the final step in the chapter's logic is not executability or interoperability, but learnability.

Terminal-Bench is a useful boundary case. It evaluates agents on **89 realistic CLI tasks**, each paired with an executable environment and test infrastructure, yet even strong frontier systems remain far from saturation. On the paper's reported results, the best model–agent combination reaches only **62.9%** overall resolution rate [Merrill et al., 2026]. The point is not only that the benchmark is hard. It is that realistic execution surfaces still expose long-horizon software work

Table 3 | Three layers in the progression from executable environments to learnable environments.

Layer	Core question	Representative objects	What is still missing
Executability	Can the agent actually run in the environment?	CLI-Anything [HKUDS, 2026a], OpenHarness [HKUDS, 2026b], workflow environments such as ResearchGym and DevOps-Gym [Garikaparthi et al., 2026, Tang et al., 2026]	Rich execution does not by itself guarantee portability or learning signal.
Protocolization	Can runtime artifacts travel across hosts, tools, and agents?	MCP [Model Context Protocol, 2025], A2A [A2A Protocol Contributors, 2026], AG-UI [AG-UI Protocol Contributors, 2026], CUBE [Lacoste et al., 2026]	Shared interfaces improve reuse, but do not yet provide reward, credit assignment, or stable update targets.
Learnability	Can interaction traces be converted into optimization-ready supervision?	Terminal-Bench-style verification [Merrill et al., 2026], Harbor-style RL environment construction [Harbor Framework, 2026]	Even executable and standardized environments may still lack dense reward, reliable verifiers, and robust episode structure.

with sparse reward, delayed verification, and difficult error recovery. To become training signal, such tests have to be attached to repeatable episodes, provenance-rich traces, and rewards or verifier labels that can be attributed to concrete command sequences, file edits, and recovery attempts. SWE-Gym illustrates this next step in software engineering by using executable tasks to train both agents and verifiers, rather than only to score completed runs [Pan et al., 2024].

The same issue appears in the MLE branch, where the raw feedback is richer but still delayed. MLE-Dojo is built from **200+ real-world Kaggle challenges** and evaluates **eight** frontier LLMs in interactive machine-learning-engineering environments, yet the authors still report significant limitations in autonomous long-horizon solution generation and complex error resolution [Qiang et al., 2025a]. Kaggle-style metrics, leaderboard ranks, and submission outcomes are useful outcome-level training signals, but they do not by themselves explain which data-processing step, feature choice, model configuration, tool call, or debugging action caused the change. Turning MLE environments into PRM- or Gym-style training substrates therefore requires denser instrumentation: intermediate experiment states, metric deltas, error fixes, and rank changes must be aligned with the multi-step trajectory. MLE-Smith and SandMLE point in this direction by scaling MLE task construction and making cheaper sandboxed optimization feasible [Qiang et al., 2025b, Zhou et al., 2026e].

Harbor makes the transition from executable evaluation to trainable environment construction explicit. It treats the harness not only as an evaluation wrapper, but as a framework for running agent evaluations and constructing RL environments [Harbor Framework, 2026]. In the terminology of this chapter, Section 5.2 enlarges the executable surface, and Section 5.3 makes that surface more reusable. The bridge to agent RL begins only when those environments can additionally provide stable episode boundaries, executable verification, and reward instrumentation.

This trainability requirement is stronger than ordinary evaluation. A benchmark can decide

whether a final state passes, while a training environment must also produce repeated episodes, preserve enough provenance to support attribution, and expose reward-bearing events at a granularity that optimization can use. For agentic systems, this turns environment design into a trace-production problem: the same execution substrate must support action, audit, replay, verification, and conversion into governed learning signals. That is why the bridge to the parameter path is not simply "more benchmarks," but environments whose feedback can be admitted into the training pipeline described in Section 6.

Robustness creates an additional learnability constraint. RobustBench-TC makes the sim-to-real gap concrete for tool-use agents: deployment noise in observations, action spaces, reward-relevant metadata, and transition dynamics can sharply reduce performance, with reward-relevant and transition perturbations producing roughly 40% and 30% accuracy drops in the reported study [Zhou et al., 2026d]. ToolRL-DR then suggests that robust learning requires perturbation-aware environment design rather than clean benchmark rollouts alone, because domain-randomized RL on a 3B backbone narrows the perturbed-performance gap. In this sense, learnability is not only a matter of reward density; it is also a matter of whether the environment exposes the noise distribution that deployed policies must survive.

Verifier construction is moving in a complementary direction. MANTRA uses natural-language manuals and tool schemas to synthesize SMT-validated, trace-level compliance checks, turning procedural rules into machine-checkable benchmark constraints [Anand et al., 2026]. ToolMisuseBench isolates a more operational failure mode by evaluating invalid arguments, interface drift, policy violations, retry behavior, and recovery under deterministic fault injection [Sigdel and Baral, 2026]. Together, these benchmarks show that execution traces need not be judged only by terminal success. They should also be checked against compliance obligations and recovery contracts, which are closer to the supervision required for policy improvement in real software environments.

Finally, the environment frontier is widening into multimodal and vertical domains. Recent examples include disaster-response geospatial workflows, clinical EHR tasks, radiology viewers, exploratory data analysis, multimedia terminal tasks, and chemistry procurement-cost reasoning [Heo et al., 2026, Liu et al., 2026d, Maksudov et al., 2026, Wang et al., 2026f, Wu et al., 2026e, Zhang et al., 2026j]. These benchmarks are best read here as a trend rather than a separate taxonomy: as environments become domain-specific, the missing pieces become less about generic tool calling and more about calibrated perception, domain-grounded state transitions, specialized verifiers, and robustness to realistic input noise.

The chapter's main conclusion is therefore narrower than "richer environments are better." Richer environments are valuable because they raise the ceiling of what a agentic system can do and experience. But executable environments, standardized boundaries, and learnable environments are three different achievements. The first makes software runnable by agents. The second makes runtime experience portable. The third makes that experience optimizable. That is the precise handoff to Section 6.

Part III. Consolidation and Coordination of Experience

6. RL and Continual Learning: Consolidating Experience into Model Parameters

6.1. Why Parameter-side Consolidation Matters

As Section 3, Section 4, and Section 5 document, useful priors often first appear outside the model: in reusable skills, in accumulated memory, and in the structure of the execution environments in which the agent acts. The parameter path begins when those repeated harness-side lessons are no longer treated as local patches, but as evidence that the underlying policy itself should change.

This shift matters for three reasons. First, it internalizes stable runtime priors instead of paying repeated context and orchestration overhead. Recent industry reports already make this direction concrete: Composer 2 aligns RL rollouts with the same coding harness used at deployment [Research et al., 2026]. Cursor's real-time RL loop then aggregates production feedback into frequent policy updates rather than treating every failure as another prompt repair [Jackson et al., 2026]. Second, weight updates generally promise broader transfer than literal context replay: a harness edit only helps when the right file, memory entry, or retrieval path fires, while a good policy update can travel across tasks, sessions, and users. Experience-driven agent training makes this distinction concrete: SEAgent turns software-interaction rollouts into optimization signal rather than only replaying them in context [Sun et al., 2025]. UI-Mem makes the same point for GUI experience memory [Xiao et al., 2026]. RetroAgent further shows that experience memory can be used for retrospective intrinsic feedback and explicit experience reuse [Zhang et al., 2026l]. EMPO^2 uses memory to improve exploration and combines on- and off-policy optimization for more robust memory-augmented agents [Liu et al., 2026g]. Third, parameter updates enable collective evolution across deployed agents: once many agents share a base model, one agent's experience can become another agent's prior. This mirrors the core premise of federated learning, where local clients compute updates from decentralized data and aggregate them into a shared model [McMahan et al., 2017]. Production federated-learning system design makes decentralized updates deployable while data remains on devices [Bonawitz et al., 2019]. This stream also raises systems, privacy, and communication challenges for practical deployment [Kairouz et al., 2021]. Update timing and system heterogeneity create the corresponding optimization problem [Li et al., 2020].

Using the survey-wide runtime definition $\mathcal{A}_t = \langle M_{\theta_t}, H_t, U_t, E_t \rangle$, the parameter path is the case in which the model component M_{θ_t} itself is updated rather than only the harness component H_t. Before any parameter update, the harness must turn the compiled deployment experience Z_{s_k} into a selected training set:

$$I_{s_k} = S_{O_{s_k}}(Z_{s_k}; H_{s_k}), \qquad \widetilde{Z}_{s_k} = \{(z_i, \widehat{y}_i) : i \in I_{s_k},\ \widehat{y}_i = a_{O_{s_k}}(z_i; H_{s_k})\}. \tag{1}$$

Eq. 1 is the *training-data selection rule*. The raw reservoir Z_{s_k} contains everything the deployed agent has compiled so far, but some of it should not be used directly for optimization. The selection operator $S_{O_{s_k}}(\cdot; H_{s_k})$ abstracts the governed choice of which reservoir traces become training examples. Its dependence on H_{s_k} records that selection is mediated by the current harness: privacy rules, attribution, verifier availability, permissions, trace quality, and task coverage can all affect whether a trace is usable. Its dependence on O_{s_k} records that the training objective also matters: SFT, preference learning, and RL may select different traces and extract different feedback or supervision objects. For example, S may be a top-K, thresholded, or otherwise filtered index set under verifier or reward scores, as in rejection- or reward-ranked fine-tuning. The objective index is written on S and a because it chooses the operator family; the harness state remains an argument because it supplies the concrete permissions, verifiers, trace metadata,

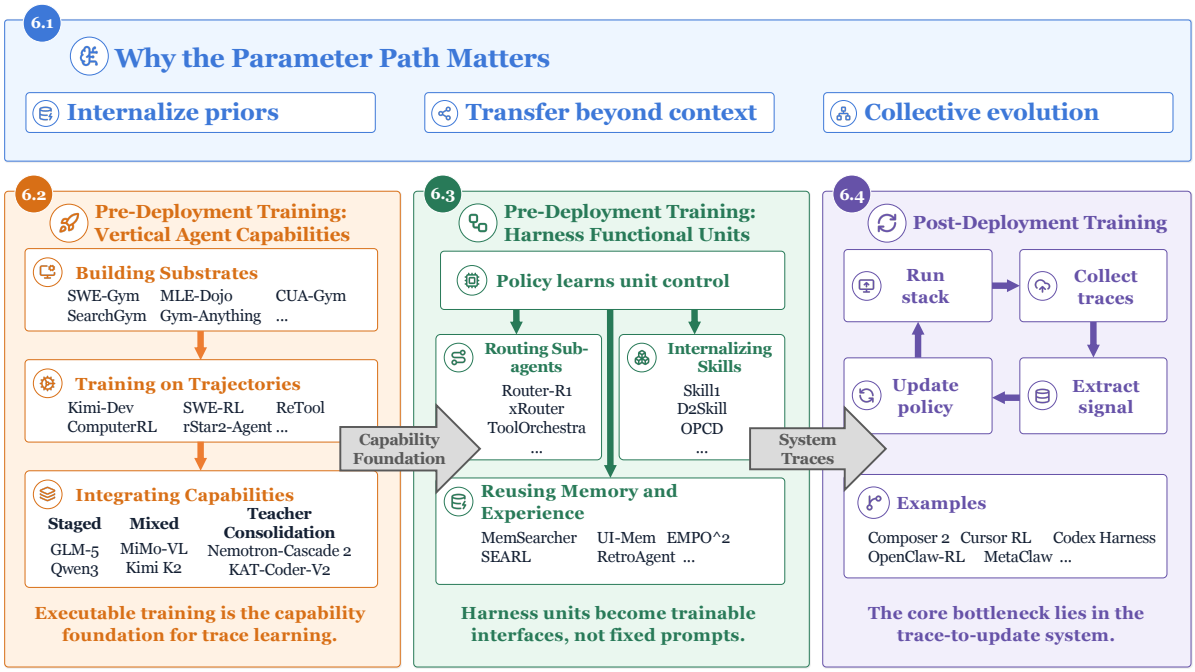

Figure 7 | Parameter-side learning for stronger agents after deployment. Stable runtime priors first accumulate in harness-side artifacts and interaction traces; the parameter path consolidates selected lessons into model updates that reduce repeated context orchestration, transfer beyond context replay, and enable collective evolution. The lower panels organize current evidence into three regimes: vertical capability learning, where executable gyms, verifiers, SFT/RL, and integration recipes build agentic abilities before release; training harness functional units, where policies learn to call and coordinate sub-agents, skills, memory, tools, and experience stores; and post-deployment trace learning, where signals come from traces generated by the running model–harness–environment stack and must pass through trace selection, signal extraction, and same-harness training infrastructure.

and extraction policies available at checkpoint s_k. For each selected trace, $a_{O_{s_k}}$ produces the corresponding $\widehat{y}_i$, a training-signal object that may be a demonstration or action label for SFT, a preference or rubric annotation, or a scalar reward used by preference learning or RL.

The parameter-side objective can then be written as a checkpointed consolidation problem:

$$\begin{aligned} \theta_{s_k^+} &= \arg\max_{\theta} J_{s_k}(\theta), \\ J_{s_k}(\theta) &= \mathbb{E}_{(z_i,\widehat{y}_i)\sim\widetilde{Z}_{s_k}} \left[\mathcal{J}_{O_{s_k}}(\pi_\theta; z_i, \widehat{y}_i) \right]. \end{aligned} \tag{2}$$

Eq. 2 is the *model-update rule* applied after selection. It says that the next deployable checkpoint is obtained by optimizing a standard training objective over trajectories and signal annotations produced by the harness. Here π_θ is the policy induced by the model parameters, and $\mathcal{J}_{O_{s_k}}$ is the post-training objective chosen for the checkpoint. For SFT, it reduces to log-likelihood on verified actions or demonstrations extracted from z_i. For preference learning or RL, $\widehat{y}_i$ specifies the feedback used by the training objective: it may come from human preferences, rubric-based evaluators, or automated verifiers, and it may be assigned at the action, segment, or trajectory level. This formulation preserves the distinction made in earlier chapters: under the external path, z_i updates skills, memory, or other harness surfaces; under the parameter path, the selected and signal-annotated experience $\widetilde{Z}_{s_k}$ updates θ.

6.2. Pre-Deployment Training for Vertical Agent Capabilities

Recent parameter-side work first improves agents by concentrating training pressure on specific vertical capabilities. The public technical reports and release notes from major model builders make this direction explicit. OpenAI's GPT-5.5 release emphasizes coding, computer use, tool use, and knowledge-work workflows [OpenAI, 2026]. Anthropic's Claude Opus 4.7 report emphasizes long-running coding and agentic tool use [Anthropic, 2026]. Google DeepMind's Gemini 3.5 Flash report foregrounds similar agentic tool-use workflows [Google DeepMind, 2026]. MiniMax M2.7 explicitly targets harness-aware self-improvement [MiniMax, 2026]. GLM-5.1 emphasizes long-horizon autonomous engineering and agentic coding workflows [Z.ai, 2026]. Qwen3-Coder-Next is a recent Qwen report specialized for coding agents trained with executable environments [Cao et al., 2026]. Beyond these frontier model families, Kimi K2.5 foregrounds visual agentic intelligence and self-directed agent swarms [Moonshot AI, 2026], while Cursor Composer 2 gives a product-facing example of coding-agent RL in a matched deployment harness [Research et al., 2026]. The common message is not that a single optimizer solves agency. It is that vertical capability learning is a stack-level problem: the domain must first be made executable enough to generate reliable trajectories and feedback, and the resulting behaviors must then be integrated through staged training, mixed-domain RL, or distillation rather than left as isolated specialists. In the Agentic AI pipeline, these methods are primarily pre-deployment mechanisms for internalizing agentic capabilities before release: they teach the base policy to act through tools, environments, verifiers, and long-horizon workflows before it is exposed to live users. This remains the mainstream path because executable gyms, synthetic tasks, and verifier-backed rewards are easier to control, scale, audit, and repeat than noisy deployment traces, while still reducing the train–test gap for later deployed agents. It also creates the usability threshold that makes post-deployment learning possible at all: if an agent cannot reliably complete useful tasks before release, users will not adopt it, the system will not accumulate meaningful usage traces, and the post-deployment trace-to-update loop has little signal to learn from.

Executable training substrates. For the parameter path, environment construction is handled upstream in Section 5; this subsection focuses on how the resulting substrates are converted into training signal. In software engineering, SWE-like worlds encode repositories, tests, issue traces, and execution logs so that feedback maps to correctness, regressions, or recovery quality [Pan et al., 2024, Yang et al., 2025a]. In machine-learning engineering, MLE-Dojo, MLE-Smith, and SandMLE build experiment-oriented sandboxes where task edits, metric deltas, and execution artifacts become explicit trajectory observables for optimization-ready traces [Qiang et al., 2025a,b, Zhou et al., 2026e]. In tool domains, substrate construction is dominated by executable API/task generation: NESTFUL and Procedural Environment Generation for Tool-Use Agents emphasize scalable, verifiable tool-call trajectories and dependency-aware task synthesis [Basu et al., 2025, Sullivan et al., 2025]. In search domains, SearchGym and DeepResearchGym build verifiable simulation/data-generation substrates that convert queries, retrieved evidence, and final answers into reward-bearing search trajectories [Coelho et al., 2025, Zhang et al., 2026m]. In computer-use domains, CUA-Gym, Gym-Anything, WebGym, and BrowserGym contribute cross-platform or GUI/browser computational substrates that turn environment state, screen observations, and action histories into repeatable episodes for training [Aggarwal et al., 2026, Bai et al., 2026, Le Sellier de Chezelles et al., 2025, Wang et al., 2026a].

SFT and RL on executable trajectories. Once the substrate exists, the methodological novelty is not simply that agents use SFT or RL, but how training is staged around executable trajectories. SFT supplies an initial policy from expert, synthetic, or teacher-generated traces; RL then optimizes recovery, tool-use, retrieval, and exploration behaviors under verifier-backed rewards. The clearest vertical is *software engineering*: Kimi-Dev, SWE-Dev, SWE-RL, long-context SWE-agent RL, and Self-Play SWE-RL all turn repositories, tests, synthetic issues, workflow traces, or execution feedback into training signal for issue-resolution agents, using different mixtures of mid-training,

SFT, rejection or execution-feedback fine-tuning, online RL, and self-play. Kimi-Dev uses agentless training to induce SWE skill priors and then adapts to SWE-agent settings with SFT on public trajectories [Yang et al., 2025b]. SWE-Dev combines synthetic test-case generation, executable patch evaluation, scaled agent trajectories, and training/inference scaling [Wang et al., 2025a]. SWE-RL uses rule-based RL rewards over open software-evolution data for real-world SWE reasoning [Wei et al., 2025b]. Long-context SWE-agent RL trains long-context, multi-turn SWE agents with execution feedback and RL [Golubev et al., 2025], while Self-Play SWE-RL studies self-play bug-injection and repair loops [Wei et al., 2025c]. *Tool* methods such as ReTool and ToolRL train when and how to invoke code interpreters, call external functions, or select among tools under outcome or tool-use rewards. ToolLLM is an early SFT-oriented pipeline for tool-use agents: it constructs large-scale API-use instructions and solution traces, then fine-tunes ToolLLaMA to internalize multi-step tool selection and invocation patterns [Qin et al., 2024]. ReTool introduces selective tool use [Feng et al., 2025], and ToolRL optimizes execution trajectories with reward-aware control [Qian et al., 2025a]. *Search* methods such as Search-R1 specialize the same SFT–RL pattern for retrieval-facing agents, training policies to decide when to query, how to use retrieved evidence, and how to fold search observations back into reasoning [Jin et al., 2025a]. *Computer-use* methods such as ComputerRL and ACuRL scale online desktop or GUI environments and adapt policies under environment shifts [Lai et al., 2025], while ACuRL provides autonomous curriculum learning for continual computer-use adaptation [Xue et al., 2026]. *Mathematical and executable reasoning* methods such as rStar2-Agent use Python execution and staged SFT–RL recipes to train multi-step agentic reasoning [Shang et al., 2025]. General multi-turn frameworks make the same point at the agent level: LOOP trains interactive digital agents directly in stateful API environments [Chen et al., 2025b]. AgentGym-RL exposes a modular RL stack for long-horizon decision-making [Xi et al., 2025], and Agent-R1 supplies a related agent-level RL recipe [Cheng et al., 2025a]. The training contribution in these papers is therefore often a trace-construction and credit-assignment contribution: decompose a domain into verifiable substeps, filter and replay rollouts, reinforce positive examples, scale synthetic or self-play environments, and transfer a workflow-trained prior into a more flexible multi-turn agent. This is why recent agentic RL surveys emphasize rollout generation, filtering, control, and replay as much as the policy-gradient formula itself. A practitioner's guide frames this as co-designing environments, rewards, and policies in multi-turn agentic RL [Wang and Ammanabrolu, 2025]. GFCR makes the same systems point through rollout filtering and control [Surana et al., 2026], while Agent Lightning pushes it into framework infrastructure by decoupling agent execution from RL training and exposing a standardized trace and credit-assignment interface for existing agents [Luo et al., 2025].

Integrating vertical capabilities into generalist agents. The next problem is how to combine many vertical capabilities without training and serving a separate specialist for every domain. Recent reports point to three integration patterns, and they should not be conflated. The first is *staged capability integration*: strengthen capabilities in a deliberate order and then consolidate them into one generalist checkpoint. GLM-4.5 is an expert-first example: its post-training first builds expert models for reasoning, agentic tasks, and general chat, then uses expert-model iteration and distillation-style post-training toward a unified hybrid-reasoning model [GLM-4.5 Team, 2025]. GLM-5 makes the post-training structure more explicit through asynchronous post-training RL and agent RL for long-horizon engineering tasks [Zeng et al., 2026a]. Qwen3 is a softer mode-fusion case, where long-CoT cold start and reasoning RL are followed by thinking-mode fusion and General RL so that one checkpoint supports thinking and non-thinking behavior, tool use, coding, and instruction following [Qwen Team, 2025].

The second pattern is *mixed or multi-domain RL*: train one policy on a mixture of domains, rewards, and environments rather than first building independent experts. M2RL directly compares mixed multi-task RLVR with separate domain RL followed by model merging across math, coding, science, instruction-following, and agent tasks [Wang et al., 2026d]. The cross-domain study

of Cheng et al. gives the useful caution behind this category: pretraining-rich domains such as math, code, and science often benefit from cross-domain RL, whereas underrepresented domains such as logic, simulation, and tabular reasoning usually need in-domain reward signals [Cheng et al., 2025b]. Frontier reports instantiate the same idea at larger scope: Kimi K2 uses large-scale agentic data synthesis and joint RL over real and synthetic environments that include agentic, coding, reasoning, and tool-use settings [Kimi Team, 2025]. MiMo-VL names its recipe Mixed On-policy RL across reasoning, perception, grounding, GUI, and other diverse reward signals [Xiaomi LLM-Core Team, 2025]. ERNIE 5.0 introduces RL for unified multimodal foundation models over text, image, video, and audio in a unified foundation model [Baidu ERNIE Team, 2026]. DeepSeek-V3.2 uses large-scale agentic task synthesis and scaled post-training RL to improve both reasoning and agentic capabilities [DeepSeek-AI, 2025].

The third pattern is *teacher-based consolidation*, especially on-policy distillation (OPD). Instead of forcing all capabilities to coexist inside one sparse-reward mixture, OPD lets the student generate from its own policy and receive dense teacher feedback on the states it actually visits. MiMo-V2-Flash introduces multi-teacher OPD, where domain-specialized teachers provide token-level supervision to one student [Xiaomi LLM-Core Team, 2026]. KAT-Coder-V2 is the clearest agentic-coding example: it follows a "Specialize-then-Unify" recipe, trains specialists for SWE, web coding, terminal, web search, and general coding, and then consolidates them into one model through on-policy distillation [KAT-Coder Team, 2026]. GLM-5 uses cross-stage OPD to preserve skills from earlier SFT, Reasoning RL, and General RL stages during later training [Zeng et al., 2026a], while Nemotron-Cascade 2 combines Cascade RL with multi-domain OPD from the strongest intermediate teachers [Yang et al., 2026e]. Recent OPD analyses distinguish general success conditions, failure modes, and open problems [Song and Zheng, 2026]. More specifically, OPD benefits from compatible teacher–student thinking patterns and genuinely new teacher capabilities, while off-policy cold start and teacher-aligned prompt selection can rescue failing OPD [Li et al., 2026e]. OPCD provides the broader on-policy context distillation formulation [Ye et al., 2026d], while CoPD studies co-evolving experts with bidirectional OPD during RLVR [Gu et al., 2026].

6.3. Pre-Deployment Training for Harness Functional Units

Vertical capability learning trains the model to be better at a domain. A separate line of work trains the model to use particular harness functions better. We include a function unit only when there is explicit training signal for the model's use of that unit; purely prompt- or workflow-engineered harness components remain part of the external path rather than this subsection. The target is inference-time control: learning when to call, adapt, and coordinate skills, memory, tools, or sub-agents.

Sub-agent orchestration and routing. Sub-agent use is the clearest newly emerging unit. Chain-of-Agents distills multi-agent systems into trajectories where one model dynamically activates tool agents and role-playing agents, then applies agentic RL on verifiable tasks; the target is no longer a hand-written multi-agent workflow, but a policy that has internalized when and how to simulate multi-agent collaboration [Li et al., 2025b]. A neighboring line trains the orchestration policy over a portfolio of models. Router-R1 formulates multi-LLM routing and answer aggregation as a sequential decision process: the router reasons, invokes candidate models, absorbs their responses into its context, and is optimized with format, outcome, and cost rewards [Zhang et al., 2025c]. xRouter instead frames orchestration as a tool-calling problem in which the learned router can answer directly or invoke one or more external LLMs under explicit cost-aware rewards [Qian et al., 2025b]. ToolOrchestra generalizes the same idea from model routing to model-and-tool routing, training an orchestrator under outcome, efficiency, and preference signals, so the choice of model or tool becomes the learned unit [Su et al., 2025b]. AgentFlow trains a planner inside a

planner–executor–verifier–generator harness, using on-policy Flow-GRPO to assign trajectory-level outcomes back to local planning decisions [Li et al., 2025c]. HiMA-Ecom makes the same issue vertical: an e-commerce master agent coordinates specialized sub-agents, with SFT samples and joint multi-agent RL used to train the hierarchy [Hu et al., 2025]. These systems are early evidence that sub-agent activation, model routing, and tool orchestration can become trainable behaviors rather than fixed prompt patterns.

Skill use and internalization. Skills are the most developed harness unit for parameter-side training. SkillRL exposes skill banks during rollout so the policy learns not only to solve the task, but to retrieve, interpret, and use task-relevant reusable skills under RL [Xia et al., 2026a]. Skill1 co-evolves skill selection, use, and distillation from a shared task-outcome signal [Shi et al., 2026]. ARISE adds a hierarchical-RL variant in which a skills manager generates, selects, and maintains a skill library from successful solution traces while a worker policy uses those skills for future rollouts [Li et al., 2026h]. D2Skill maintains a dynamic dual-granularity task/step skill bank with utility-aware updates and retrieval [Tu et al., 2026]. SKILL_0 moves from assisted use toward internalization: skills are available during training, but the learned policy is encouraged to act with less explicit skill support at inference time [Lu et al., 2026b]. Skill-SD studies a related route in which training-only skill guidance is distilled into the student policy [Wang et al., 2026c]. OPCD provides the more general context-distillation view: context-conditioned teachers can supervise the student on the student's own trajectories, internalizing historical traces or prompts into parameters [Ye et al., 2026d].

Memory access and experience reuse. Memory training asks the policy to operate over a persistent store rather than merely consume retrieved text. MemSearcher makes search and memory access part of the trainable behavior [Yuan et al., 2025]. Agentic Memory adds long-horizon store, retrieve, update, summarization, and forgetting policies [Yu et al., 2026b]. UI-Mem uses experience memory to guide exploration in GUI tasks [Xiao et al., 2026]. EMPO^2 uses memory to improve exploration and combines on- and off-policy updates so agents remain robust with or without memory [Liu et al., 2026g]. SEARL jointly optimizes a policy and a tool-graph memory so prior trajectories become reusable structure rather than flat logs [Feng et al., 2026b]. Agent-BRACE shows a related belief-state variant, where a trained belief model compresses partial observations into uncertainty-aware state claims that the policy can act on [Singh et al., 2026]. Finally, work on experience utilization makes the selection and use of prior experience itself a trainable decision, rather than assuming that more experience in context is always better [Fan et al., 2026]. Rethinking Experience provides additional evidence for selective experience use [Zhao et al., 2026a]. Traj-Evolve provides a clinical multi-agent instance of this coupling, using a verified-patient Experience Pool for retrieval while reward-ranked MARL fine-tunes worker and manager agents for longitudinal EHR risk prediction [Zeng et al., 2026c]. RetroAgent grounds the retrospective side in retrospective intrinsic feedback for online RL and explicit experience reuse [Zhang et al., 2026l].

Taken together, these papers shift harness components from passive runtime aids toward trainable interfaces. The boundary is still narrow: each work optimizes one functional unit or a small combination of units. A broader harness-training regime would likely need mechanisms for jointly governing sub-agents, skills, memory, tools, permissions, reward extraction, versioning, rollback, and evaluation across real deployments.

6.4. Post-Deployment Training from Agent Traces

Post-deployment systems satisfy two conditions simultaneously: the model must run inside a deployed 'model+harness+environment' stack, and the learning signal must come from traces generated by that running system.

These traces differ from pre-deployment benchmark rollouts. Feedback cycles are longer, trajectories often span many turns or sessions, and the learning signal is delayed, incomplete, and noisier than a curated verifier label. It may also mix several sources at once: user replies, environment state changes, tool outputs, terminal or GUI observations, runtime errors, and later downstream outcomes.

The earliest precursor appears in the chat setting rather than the agent setting. WildChat provides large-scale real user–chatbot logs [Zhao et al., 2024]. RLHI shows that natural follow-up turns in those conversations can be turned into training signal [Jin et al., 2025b]. But these systems still belong to the chatbot era: they learn from real human interaction, yet they are not agentic systems operating in richer execution environments.

The strongest public evidence comes from industry systems that align training and evaluation with product-like harnesses. Composer 2 is the clearest case: its technical report states that RL runs in realistic Cursor sessions using the same tools and harness as the deployed model [Research et al., 2026]. Cursor's real-time RL writeup states that production checkpoints are served to users, user responses are observed, and those responses are aggregated into reward signals for frequent weight updates [Jackson et al., 2026]. OpenAI's Codex is also a relevant case, as OpenAI describes codex-1 as trained with RL on real-world coding tasks across environments [OpenAI, 2025]. OpenAI's later Codex harness update clarifies the shared App Server and harness layer across Codex surfaces [Chen, 2026]. More speculative agentic systems make the same post-deployment ambition explicit. OpenClaw-RL frames live user replies, tool outputs, and terminal or GUI state changes as online RL supervision inside a deployed agentic runtime [Wang et al., 2026l]. MetaClaw sketches a related OpenClaw-like deployed loop that reuses deployed failure trajectories for skill evolution and lightweight parameter updates [Xia et al., 2026b].

Public evidence of post-deploy agent training remains sparse, and true post-deployment agent training is still at a very early stage. In practice, the bottleneck is therefore not just another policy-gradient variant, but the surrounding trace-to-update system: trajectory selection, large-scale trace collection, signal extraction, and infrastructure that can train the policy inside the same harness used at inference time.

6.5. Takeaways for Practitioners

The field has already solved more of the RL core than of the surrounding systems problem. Pre-deployment RL now works across software engineering, search, tool use, and MLE once the gym, verifier, and reward are carefully engineered. The harder deployment problem is the full trace-to-update pipeline. A deployed agent faces four concrete engineering problems: selecting which trajectories are worth training on, collecting such trajectories at sufficient scale, extracting usable learning signals from messy interaction records, and running training inside the same harness that will be used at test time. The last requirement is especially important: if training uses a simplified scaffold while inference uses the real runtime harness, the system reintroduces a train–test context mismatch exactly where agentic post-deployment learning is supposed to reduce it. Datasets such as SWE-chat indicate that the raw trace layer is becoming observable, but turning logs into trainable signals remains a separate systems problem [Baumann et al., 2026].

This is also the bridge to the next chapter. Once parameter updates become a live option, a higher-level question becomes unavoidable: who decides whether a lesson should stay as a local harness artifact or be promoted into a shared model update? The meta-evolving-agent chapter takes up that question directly. Measuring whether those updates retain capability across time is then picked up again in Section 8.

7. Meta-Evolving Agents: Who Controls What to Evolve

The preceding chapters studied how experience changes the TaskAgent, the task-facing part of the deployed system, by updating its resources and policies. This chapter asks a higher-level question: when improvement itself becomes an object of optimization, *who* controls the update process and *what* parts of that process are allowed to change? Along these two axes, post-deployment Agentic AI evolution is organized into three regimes: TaskAgent self-evolution, TaskAgent meta-learning, and meta-evolving agents, as summarized in Figure 8.

Section 7.1 defines these three regimes and their boundaries. Sections 7.2 and 7.3 review TaskAgent-level meta-learning and meta-evolving agents, respectively. The chapter closes by discussing the gap between current local meta-evolution and the ideal of general meta-evolution.

7.1. Three Regimes of Post-Deployment Agent Evolution

TaskAgent self-evolution. TaskAgent self-evolution is the regime in which the TaskAgent converts post-deployment experience into its own durable **content assets** for later use. The substantive object of evolution is the TaskAgent's reusable assets, including skill contents [Ma et al., 2026b], memory assets [Xu et al., 2025b], user preferences and experience summaries [Wang and Jiang, 2026], structured domain or task knowledge [Long, 2026, Zhang, 2026a], and subagent artifacts [Zhang et al., 2026o]. This regime has been the primary focus of the preceding surface-specific chapters.

TaskAgent meta-learning. While TaskAgent self-evolution strengthens the agent by accumulating content assets, TaskAgent meta-learning strengthens it by changing its execution mechanisms or improvement strategies. In this sense, it instantiates the classical meta-learning idea of "learning how to learn" in a deployed agent setting. The substantive object of evolution shifts from what the TaskAgent knows to how it acts or how it improves. One form improves **execution mechanisms or structures**, including memory retrieval, skill use, context construction, workflow or routing, and policy parameters. The other modifies the **improvement strategy**: how failures are abstracted, updates are triggered and validated, and useful changes are retained. In both forms, the control loop remains inside the TaskAgent or its task-facing harness.

Meta-evolving agent. Compared with TaskAgent meta-learning, a meta-evolving agent differs in two ways. First, the evolution process is organized as an ongoing, dedicated optimization task under the control of a **functionally distinct meta-layer**. This meta-layer is not primarily responsible for solving the current user-facing task. Second, a meta-evolving agent extends the scope of what can evolve. It retains the TaskAgent's execution mechanisms or structures and its improvement strategies as evolution objects, while also making the **meta-layer itself evolvable**, including its state, rules, evaluation protocols, selection policies, or control mechanisms.

7.2. TaskAgent Meta-Learning

TaskAgent meta-learning sits between content-level TaskAgent self-evolution and meta-layer-governed meta-evolving agents.

7.2.1. TaskAgent-Internal Control over Execution Mechanisms

One group of methods improves the agent's execution mechanisms, including help-seeking, experience invocation, skill selection, and policy-level skill use. MetaAgent [Qian and Liu, 2025] starts from a minimal reason–help–answer loop and improves it through self-reflection, answer verification, and tool-use traces. Its learned rules shape later judgments about when to ask for external assistance, how to route help requests, and how to turn failed trajectories into

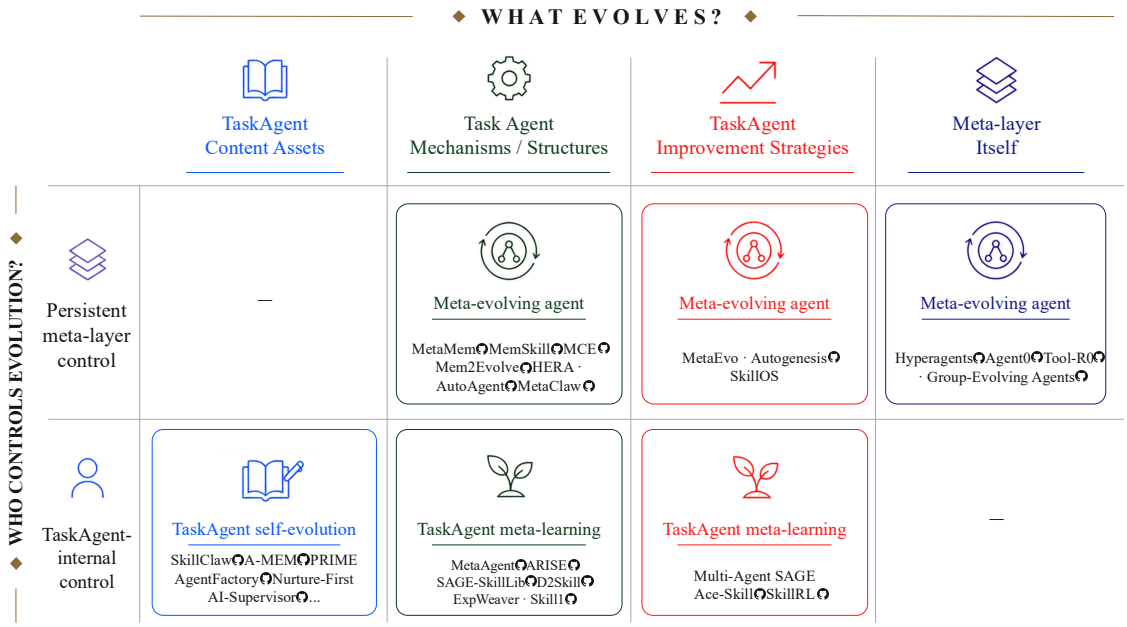

Figure 8 | Two-axis map of post-deployment agentic AI evolution regimes. Columns indicate *what evolves*: TaskAgent content assets, TaskAgent mechanisms or structures, TaskAgent improvement strategies, or the meta-layer itself. Rows indicate *who controls evolution*: TaskAgent-internal control or persistent meta-layer control.

reusable reasoning guidance. ExpWeaver [Zhao et al., 2026a] targets a different execution mechanism: when accumulated experience should be invoked during reasoning. By learning selective invocation at uncertain decision points, it improves experience utilization rather than experience content.

ARISE [Li et al., 2026h] jointly evolves the skill library and the mechanisms for selecting, generating, and using skills through a shared Skills-Manager/Worker policy. SAGE-SkillLib [Wang et al., 2025b] makes this execution-mechanism view more explicit: Sequential Rollout exposes later tasks to skills generated, updated, or saved in earlier tasks, while skill-integrated rewards and GRPO train the policy to generate, call, update, and preserve skills more reliably. D2Skill [Tu et al., 2026] adds a dual-granularity version of the same idea. It maintains task-level and step-level skills, estimates hindsight utility from paired baseline and skill-injected rollouts, and uses those signals to update skill utilities, guide retrieval and pruning, and optimize a skill-augmented policy. Skill1 [Shi et al., 2026] further unifies skill selection, utilization, and distillation under task-outcome feedback. In these systems, the skill bank is still an evolving artifact, but the substantive mechanism-level change is how the TaskAgent selects, uses, maintains, and parameterizes skills during task execution.

7.2.2. TaskAgent-Internal Control over Improvement Strategies

A second group instead targets how the TaskAgent learns to improve. Multi-Agent SAGE [Peng et al., 2026b] trains a shared-backbone system conditioned as challenger, planner, solver, and critic. The challenger generates harder tasks, the planner turns them into intermediate reasoning plans, the critic filters questions and plans before they become training signals, and the solver updates under verifiable rewards. Its improved object is the internal improvement strategy for constructing future learning pressure, including curriculum construction, training-data selection,

and validation of self-generated learning signals. Ace-Skill [Xiong et al., 2026a] shifts this idea to multimodal tool-use bootstrapping: prioritized sampling and clustered organization allocate self-evolution effort and reduce interference. The improved object is the internal strategy for choosing learning samples and organizing reusable knowledge. SkillRL [Xia et al., 2026a] connects trajectory abstraction to parameter updates: it distills successful and failed episodes into a hierarchical SkillBank, uses adaptive retrieval and cold-start SFT to teach the model how to interpret and apply skills, and then applies GRPO while recursively adding or refining skills after validation failures. Its central meta-learning signal is the improvement strategy that converts validation failures into recursive skill revision and skill-augmented policy updates.

Both groups go beyond content accumulation: one changes how the TaskAgent acts, and the other changes how the TaskAgent learns to improve. However, the control loop still belongs to the TaskAgent's execution or training process.

7.3. Meta-Evolving Agents

In a meta-evolving agent, improving the TaskAgent is no longer only an internal side effect of acting or training; it becomes an ongoing optimization task controlled by a functionally distinct meta-layer. This meta-layer continually improves the TaskAgent or itself using evidence accumulated from execution, evaluation, feedback, and prior improvement attempts.

Before reviewing core meta-evolving-agent systems, we separate them from two adjacent uses of optimization in agent systems. The key question is whether the optimization loop remains active after deployment as a distinct meta-layer for improving the TaskAgent. **Design-time or benchmark-time search methods** optimize an agent artifact before deployment or within a bounded campaign. Some search memory designs or memory programs [Pan et al., 2026, Xiong et al., 2026b]; others search harness code [Lee et al., 2026b], workflow graphs [Legrand et al., 2026, Zhang et al., 2024a], or agent architectures [Hu et al., 2024, Zhang et al., 2025a]. RoboPhD [Borthwick et al., 2026] broadens this design-time view to open-ended agent/code artifact evolution under a fixed evaluation budget. These works show that agent surfaces can be searched, but the search loop usually stops after producing the artifact. **Learnable task-time orchestrators** optimize the control policy used to solve the current request. FlowReasoner [Gao et al., 2025], MetaGen [Wang et al., 2026k], and GraphPlanner [Feng et al., 2026a] generate query-specific workflows, roles, topologies, or routing plans. ROMA [Alzu'bi et al., 2026] emphasizes recursive decomposition and aggregation, while SkillOrchestra [Wang et al., 2026e] and AOrchestra [Ruan et al., 2026] learn skill-aware routing or sub-agent delegation. Puppeteer [Dang et al., 2025] learns scheduling, termination, and coordination policies for multi-agent collaboration. Such orchestrators enter the meta-evolving-agent regime only when a distinct meta-layer treats the orchestrator itself as an object of long-term, cross-task improvement.

We next organize meta-evolving-agent works by what the meta-layer makes evolvable.

7.3.1. Meta-Layer Control over Execution Mechanisms

The first line asks how a meta-layer can take control of the mechanisms by which a TaskAgent acts. Mechanism-level evolution concerns the rules, lifecycles, states, and update paths that govern how harness components are used. We group this literature by scope and persistence of the mechanisms that the meta-layer intervenes in the TaskAgent's execution mechanisms.

Evolvable local control policies. The most localized form of mechanism evolution turns a control rule on one harness surface into an optimizable object. MetaMem [Xin et al., 2026] uses self-reflective meta-memory optimization to separate factual memory from meta-memory: factual memory stores task facts, whereas meta-memory stores rules for using memory in later

tasks. Reflection and judge feedback revise these meta-memory rules rather than merely adding more records. MemSkill [Zhang et al., 2026f] exposes memory-skill design to an outer Designer, organizing extraction, integration, retention, and forgetting as an evolvable memory skill bank controlled by a selector. CluE [Yang et al., 2026c] narrows the same idea to a prompt-evolution loop around memory extraction, maintaining cluster pools, candidate prompts, and a best prompt under heterogeneous task feedback. Meta Context Engineering [Ye et al., 2026c] applies the same pattern to context construction: an outer evolver rewrites a base agent's context-learning skill from rollout feedback, so the object of evolution is the mechanism for representing, updating, and organizing context. At this granularity, meta-layer control first appears as editable policies for extracting, selecting, constructing, and invoking information.

Governed capability-resource lifecycles. Mem^{2}Evolve [Cheng et al., 2026] combines Asset Memory and Experience Memory so that tool code, expert-agent specifications, and tool or agent experience can be created during forward inference, then verified, self-corrected, distilled, and written back by a backward evolution layer. The evolving object is therefore the creation-and-reuse mechanism for tools and expert agents, not merely the accumulated asset list. MetaClaw [Xia et al., 2026b] extends lifecycle control into real deployment through a proxy-based evolution layer with two paths: a fast skill-evolution path that turns failure trajectories into immediately usable external skills, and a slower policy-consolidation path that feeds post-adaptation trajectories into LoRA or GRPO updates of the base policy. Continual Harness [Karten et al., 2026] brings the same lifecycle view to embodied agents. During a single continuous episode, a Refiner reads recent trajectory windows and applies CRUD edits to the system prompt, sub-agents, skills, and memory; its co-learning variant also updates model weights from relabeled rollouts. Agentic Harness Engineering [Lin et al., 2026b] generalizes this lifecycle view to coding-agent harness components. It decomposes the harness into editable file-level resources such as tools, middleware, skills, sub-agent configurations, and long-term memory; an Agent Debugger distills long trajectories into evidence, and an Evolve Agent edits the harness with predictions that are checked against later task outcomes. Across these works, the meta-layer governs the lifecycle of capability resources, from creation and reuse to revision and consolidation.

Persistent organizational and cognitive state. HERA [Li and Ramakrishnan, 2026] uses an Orchestrator and an Experience Library as a persistent control layer over a multi-agent RAG system. The Orchestrator generates query-specific topologies from the query, agent pool, and experience library; the Experience Library maintains trajectory-reflection insights by utility; and Execution Agents' prompts evolve under feedback. AutoAgent [Wang et al., 2026i] similarly places an Evolution Cycle around explicit, updateable cognition state. Its Execution Cycle advances the current task, while its Evolution Cycle summarizes trajectories, aligns intended actions with actual outcomes, and writes revisions into tool prerequisites, peer profiles, skill templates, composite actions, or memory-orchestration strategies. These systems move meta-layer control from local rules and resources to agent organization, cognition state, and cross-agent experience that persist across future tasks.

Evidence-driven domain and parameter consolidation. One branch consolidates execution evidence into domain-specific state. Milkyway [Wei et al., 2026] applies this pattern to future prediction, where a persistent harness tracks evolving evidence and revises unresolved predictions through temporal contrast and retrospective feedback. KernelBlaster [Dong et al., 2026b] applies the same idea to CUDA kernel optimization, maintaining a profiler-conditioned memory of optimization strategies and observed gains for later kernels.

A second branch connects execution evidence to workflow memory and parametric or policy updates. Memory Intelligence Agent [Qiao et al., 2026] uses a Memory Manager, Judger, and TTRL update channel to compress trajectories into workflow memory and contrastive plans while updating Planner parameters [Zuo et al., 2025]. OpenClaw-RL [Wang et al., 2026l] pushes this

consolidation path into policy learning: next-state signals after actions are converted into process rewards or token-level supervision for asynchronous policy updates. Together, these systems show how a meta-layer can turn execution evidence into persistent domain state, workflow memory, reward signals, or policy checkpoints.

7.3.2. Meta-Layer Control over Improvement Strategies

The second line shifts meta-layer control from execution mechanisms to the TaskAgent's improvement strategies. Here, the meta-layer governs how evidence from prior improvement attempts is turned into candidate changes: how failures are abstracted, intervention surfaces are chosen, repairs are generated, and proposed edits are validated, retained, or rolled back.

MetaEvo [Ren et al., 2025] places a meta-optimization loop around principle-based self-correction. The loop compares stronger and weaker correction principles, improves the model's ability to abstract high-quality principles from failure trajectories, and stores the resulting principles for later retrieval. The evolving object is the TaskAgent's improvement strategy: how it learns from failures, abstracts reusable correction principles, and applies them to future repairs.

SkillOS [Ouyang et al., 2026] provides a concrete skill-curation instance of this regime. It keeps an Agent Executor frozen as the task-solving side and trains a separate Skill Curator as the meta-layer above a persistent SkillRepo. Across grouped task streams, the executor uses the repository to solve tasks, while executor-grounded composite rewards assign downstream task outcomes, operation validity, compression, and content quality to the curator's insert, update, and delete decisions. The evolving strategy is skill curation itself: how the curator learns from executor feedback to decide which skills should be inserted, revised, removed, or preserved for future tasks.

Autogenesis [Zhang, 2026b] formulates TaskAgent improvement as a protocol-governed evolution loop. RSPL represents prompts, agents, tools, environments, and memory as versioned resources, while SEPL defines the reflect-select-improve-evaluate-commit loop by which they are updated. Thus, the meta-layer is the AGS loop governed by SEPL over RSPL resources. The evolving object is the improvement procedure over TaskAgent resources: how heterogeneous states are exposed for editing, how candidate modifications are routed through evaluation, and how accepted changes become the next TaskAgent version.

7.3.3. Improving the Meta-Layer Itself

The final line turns the meta-layer itself into the object of evolution. Its state, policies, or control rules are updated so that the improvement process itself can become more effective over time.

Agent0 [Xia et al., 2025] and Tool-R0 [Acikgoz et al., 2026] instantiate meta-layer self-improvement through curriculum and task synthesis. In Agent0, a Curriculum Agent serves as the meta-layer above an Executor Agent and generates training tasks near the executor's capability frontier. Tool-R0 gives the tool-use counterpart, where a task generator, verifier, and filtering rules form the meta-layer around a tool-use Solver. In both systems, the evolving meta-layer object is the curriculum or task-synthesis policy: uncertainty estimates, tool-use frequency, repetition penalties, cross-verification, and difficulty filters determine which frontier tasks are retained for later training. Evidence from previous learning rounds therefore changes how the meta-layer produces future improvement opportunities for the TaskAgent.

Group-Evolving Agents [Weng et al., 2026] shift this self-reference from task synthesis to population-level improvement state. The TaskAgents are evolving agent groups that receive workflow, tool-use, prompt, or code-level edits. Above them, the meta-layer is the population-level improvement loop formed by the archive, parent-selection procedure, shared experience

pool, reflection modules, and evaluation-retention logic. As code patches, task-success vectors, archive lineages, and shared parent experiences accumulate, they reshape the meta-layer state that governs which parents and experiences seed later self-improvement. Although the direct edits still land on TaskAgent components, the distribution of future improvement attempts is itself being updated.

Hyperagents [Zhang et al., 2026g] pushes the idea further and is one of the central current examples of self-referential meta-evolution. Its premise is that a system remains bounded if the TaskAgent is editable but the improvement procedure is fixed by hand. Hyperagents therefore places the task-solving agent and the self-improving agent inside a single editable program space. In DGM-Hyperagents, an archive-based outer loop selects parents, generates children, evaluates them, and returns them to the archive, while the inner hyperagent can modify its own codebase through bash and editor tools. Hyperagents expands the editable program space beyond `task_agent.py`: `meta_agent.py`, prompt templates, memory, performance tracking, evaluation analysis, bias detection, and parent-selection logic can also be searched and rewritten. Its improvement-at-k metric likewise shifts evaluation from the current TaskAgent alone to the meta-layer's ability to produce better descendants under a fixed modification budget.

Taken together, these systems extend the evolvable boundary from the TaskAgent to the meta-layer that produces future improvements, yet current systems still tend to optimize within one dominant channel. An ideal meta-evolving agent would treat evolution across TaskAgent content assets, execution mechanisms or structures, improvement strategies, and meta-layer mechanisms as a unified learnable control problem. Its meta-layer would decide when and where to intervene, then evaluate and govern the resulting improvement. More importantly, the history of prior improvement attempts would refine these decisions, allowing the improvement process itself to become more effective over time. This ideal remains difficult because heterogeneous agent states, intervention actions, evidence signals, authority boundaries, and update timescales must first be made learnable and governable within one system. The harder issue is self-reference: once the meta-layer itself can change, the system gains adaptivity but loses the stable reference normally provided by a fixed optimizer. The central open problem is therefore to build an autonomous meta-evolution loop in which the process of improving agents becomes progressively more effective and more efficient through evolution.

Part IV. Measurement, Safety, and Open Problems

8. Measuring Self-Improvement: What Current Benchmarks Still Miss

A higher post-adaptation score does not necessarily mean that a deployed system is better overall after updating its skills, memory, tools, or weights. Most current benchmarks are still better at measuring one-off task success than durable self-improvement. Here, self-improvement refers to the measurable longitudinal effect of accumulated experience on the same deployed agent system. The chapter therefore defines the minimum targets of self-improvement (SI) evaluation, reviews where current benchmarks still fall short, and proposes a compact reporting standard.

8.1. What Self-Improvement Evaluation Must Measure

8.1.1. Why Self-improvement Itself Must Be Evaluated

The first evaluation mistake is to treat self-improvement as ordinary post-adaptation performance. Evidence that agents can improve after adaptation is already strong. SkillsBench reports an average gain of +16.2 percentage points from curated skills across 86 tasks and 11 domains [Li et al., 2026c]; CoEvoSkills and SkillNet extend that pattern at system scale [Liang et al., 2026a, Zhang et al., 2026e]. Memory-centered systems such as SimpleMem, AWM, and CER also improve long-context or web-agent performance without using the same adaptation path [Liu et al., 2026c, 2025b, Wang et al., 2025d]. Parameter-path methods matter as well: SkillRL, MetaClaw, and Absolute Zero report substantial gains on embodied, coding, and reasoning tasks [Xia et al., 2026a,b, Zhao et al., 2025].

Those results establish that local capability can change. They do not by themselves establish that the deployed agent became better as a system. SI is a temporal and distributional property: the same agent updates its skills, memory, tools, or weights, and then continues to face old tasks, new tasks, drift, cost constraints, and safety constraints. A higher post-adaptation score can therefore hide regression elsewhere. SkillsBench, for example, also reports negative transfer on 16 of 84 directly comparable tasks after skill injection [Li et al., 2026c]. That is why SI itself has to be evaluated: not just *did the agent improve somewhere*, but *did the same evolving agent improve on new tasks, retain old capability, and do so at acceptable cost and risk?*

8.1.2. From Continual-Learning Metrics to SI Targets

Continual learning provides the closest existing measurement vocabulary for this question. Its core concern is the stability–plasticity tradeoff: an adaptive system should learn new tasks without destroying old capability, and earlier learning should ideally help later learning [Lopez-Paz and Ranzato, 2017, van de Ven and Tolias, 2019]. For post-deployment agent updates—for example OpenClaw-RL, MetaClaw, or Composer-style real-time RL—that vocabulary maps naturally onto agent evaluation [Jackson et al., 2026, Wang et al., 2026l, Xia et al., 2026b].

The CL lens is necessary but not sufficient for SI. Agent improvement can come from external-path changes such as skills, memory, tools, and context management, or from parameter-path changes such as SFT, RL, and LoRA. It also has deployment-facing costs and safety consequences that classical CL metrics usually do not report. We therefore use continual learning as the starting point, then extend it into the minimum SI targets in Table 4. For this chapter, a benchmark counts as SI-relevant only when it can measure the before/after story of one evolving agent rather than merely the final score of one frozen agent.

The list above is intentionally narrow. It marks only the minimum evidence needed before a paper can plausibly claim that an agent improved over time. Without old-task replay, retention is

CL inspiration	SI target	Minimal question	Agent-specific extension
BWT / forgetting	**Backward retention**	Did old-task performance stay stable after adaptation?	Requires old-task replay for the same evolving agent
FWT / transfer	**Held-out gain**	Is the agent better on new tasks after adaptation?	Tests cross-task generalization beyond the adaptation stream
Plasticity and intransigence	**Longitudinal stability**	Does the gain survive replay and later re-testing, for example T0 → T1 → T2?	Checks whether the agent remains adaptable after repeated updates
Cost-aware CL variants	**Improvement efficiency**	How much compute, interaction, and supervision was spent per unit of gain?	Distinguishes practical SI from brute-force optimization
Not in classical CL	**Path attribution**	Did the gain come from skills, memory, tools, parameter updates, or a combination?	Makes external-path and parameter-path claims comparable
Reliability / safety evaluation	**Safety non-regression**	Did capability improve without creating a worse safety profile?	Treats SI as incomplete if capability gains hide new vulnerabilities

Table 4 | Minimum targets for self-improvement evaluation, derived from continual-learning concepts and extended to deployed agent systems. BWT, FWT, intransigence, and plasticity motivate the stability and generalization targets, while agent deployment adds cost, path attribution, and safety requirements.

invisible by construction. Without held-out tasks, generalization is invisible. Without cost logging, gains are hard to compare. Without refresh and decontamination, benchmark scores will keep decaying as static test sets saturate [Akhtar et al., 2026].

8.2. The Current Benchmark Landscape

8.2.1. From Static Evaluation to SI-Oriented Benchmarking

The evaluation paradigm has evolved alongside agent architectures. In the **first generation (2023)**, benchmarks measured single-task completion on static datasets, such as SWE-bench [Jimenez et al., 2024], WebArena [Zhou et al., 2023], and ALFWorld [Shridhar et al., 2021]. The question was straightforward: *Can the agent solve this task?*

By the **second generation (2025)**, benchmarks had started to include task sequences and evolutionary structure, as in LifelongAgentBench [Zheng et al., 2025b] and StuLife [Cai et al., 2025]. The focus then shifted to: *Can the agent accumulate capability across a sequence of tasks?* That was real progress, but evaluation still mostly happened at fixed checkpoints and rarely reported retention directly.

The **third generation (2025–2026)** starts to target SI properties more explicitly: forward transfer in EvoAgentBench [EverMind-AI, 2026], belief revision under drift in ClawArena [Ji et al., 2026], knowledge internalization in SE-Bench [Yuan et al., 2026], and memory-retention modes in AgentMemoryBench [Zihang Ma et al., 2026] and DomusMind [Rong Xu, 2026].

Benchmarks have become more dynamic, yet they are still better at answering "How strong is the agent on this benchmark?" than "Is this agent getting better over time?"

8.2.2. Benchmark Taxonomy and Coverage

The current landscape can be grouped into four families, ordered by decreasing direct relevance to SI measurement. The aim here is not exhaustiveness, but to show that even the strongest benchmarks still capture only part of the SI story.

Direct SI-Oriented Benchmarks

SkillsBench [Li et al., 2026c] remains the clearest benchmark for one-shot external-path gains. It tests 86 verifier-graded tasks across 11 domains under no-skill, curated-skill, and self-generated-skill conditions. It measures whether injected skills help, how much they help, and when they hurt; its negative-transfer results are especially valuable. It remains a one-shot protocol, though, rather than a longitudinal one, and says little about efficiency or cross-component composition.

EvoAgentBench [EverMind-AI, 2026] is the strongest current prototype for SI-aware evaluation. Clustered train/test splits, method-matched comparison, and per-result cost reporting make it useful for forward transfer and coarse efficiency analysis. It tests agents across information retrieval, mathematical reasoning, software issue repair, competitive programming, and document-grounded knowledge work. Formal BWT reporting, component attribution, and longitudinal tracking of one evolving agent across checkpoints are still missing.

SkillLearnBench [Zhong et al., 2026a] evaluates skill generation rather than skill presence, diagnosing the full generate–store–reuse cycle across 20 real-world tasks and four generation methods. Its tasks cover 15 skill-dependent subdomains, so the evaluated object is not only final task success but whether useful skills can be induced and reused. Its central finding is that the best automatic method closes only ~45% of the gap between no-skill and human-authored-skill performance, which points to an unsolved skill adoption problem distinct from skill quality. Like SkillsBench, it remains cross-sectional within each generation round; the main contribution is diagnostic depth rather than longitudinal tracking.

SE-Bench [Yuan et al., 2026] tests knowledge internalization on programming tasks built around an obfuscated NumPy API renamed into a pseudo-novel package. This isolates whether an agent can store and later use newly acquired knowledge rather than merely retrieve familiar documentation. It still measures one mechanism of self-evolution rather than global post-deployment improvement.

LifelongAgentBench [Zheng et al., 2025b] tests skill-grounded, interdependent task sequences in database, operating system, and knowledge-graph environments. It pushes evaluation toward task-sequence accumulation, but retention, efficiency, and standardized cross-paper reporting remain only partially visible.

StuLife [Cai et al., 2025] tests a simulated college-life trajectory across enrollment, academic development, and personal-growth subscenarios. Its value is that memory, skill abstraction, and internalization are evaluated as one experience-driven loop; its limitation is that the resulting metrics are not yet a shared reporting standard.

Dynamic and Deployment-Facing Benchmarks

This family makes deployment realism its explicit focus.

ClawArena [Ji et al., 2026] foregrounds information drift and belief revision across evolving information environments. It tests agents on tasks where evidence, user preferences, or world facts change over time, so success depends on updating beliefs rather than only executing a fixed plan.

AgentMemoryBench [Zihang Ma et al., 2026] tests continual agent memory across six interactive tasks adapted from agent and long-context-memory settings, with offline, online, replay, transfer,

and repair modes. It makes forgetting and memory repair measurable, but not yet under a full SI report card.

DomusMind [Rong Xu, 2026] tests persistent smart-home control under preference drift, tool drift, and mixed drift. It is useful because it turns long-lived personalization and recovery into explicit evaluation targets.

tau-bench [Yao et al., 2024] shows that repeated-run reliability is much weaker than single-run success, highlighting a deployment gap that static benchmarks miss. It tests multi-turn customer-service interactions in airline and retail domains, where agents must call APIs while obeying domain policies.

Taken together, these benchmarks move beyond static task completion. They still fall short of SI-native measurement: drift, persistence, or reliability are visible, while retention, cost-normalized gain, and component attribution are usually not.

Claw-Eval [Ye et al., 2026a] pushes this line further by making trustworthy evaluation explicit. It scores completion, safety, and robustness over full trajectories rather than only final outputs, using execution traces, audit logs, and environment snapshots across 300 human-verified tasks. Those tasks span general service orchestration, multimodal perception and generation, and multi-turn professional dialogue. That broader view is useful for deployment-facing assessment, especially because it makes trajectory quality visible. But it is still not an SI benchmark in the strict sense: it does not track held-out gain, backward retention, or one evolving agent across checkpoints.

WildClawBench [Ding et al., 2026] targets in-the-wild evaluation under realistic tool conditions. It spans 60 tasks across six categories—productivity, code intelligence, social interaction, search and retrieval, creative synthesis, and safety alignment—and reports time and cost per task for all frontier models. Like Claw-Eval, it foregrounds deployment realism; unlike Claw-Eval, it explicitly includes a safety-alignment category with 10 tasks probing prompt injection, credential leakage, and unsafe outputs. It remains one-shot, though, without longitudinal replay or held-out transfer measurement.

SWE-bench [Jimenez et al., 2024] and its variants **SWE-bench-Live** [Zhang et al., 2025e], **SWE-MERA** [Adamenko et al., 2025], and **SWE-rebench** [Badertdinov et al., 2025] take a different angle on dynamism: rather than varying the agent scenario, they vary the benchmark itself through continuous collection and decontamination. SWE-bench tests real GitHub issue resolution against repository tests; SWE-bench-Live and SWE-MERA refresh issue-based software tasks over time; SWE-rebench scales the same idea through an automated collection pipeline. This proves that benchmark renewal is feasible at scale, even if it does not by itself constitute SI-native evaluation.

Agentified Benchmark Platforms and Interoperability

A separate 2025–2026 development is the rise of **agentified assessment** as an infrastructure layer. Under the AAA (Agentified Agent Assessment) paradigm, the benchmark itself is packaged as a **green agent** that defines tasks, environment, and scoring, while the evaluated system is a **purple agent** that communicates through a standardized A2A interface [A2A Protocol Contributors, 2026, AgentBeats, 2026a, Model Context Protocol, 2025].

The Berkeley RDI **AgentBeats** ecosystem [AgentBeats, 2026b] exemplifies this approach at scale, with benchmark tracks spanning coding, research, computer use, and multi-agent evaluation. Its evaluated tasks are therefore supplied by packaged benchmark agents rather than by one fixed dataset. This primarily addresses interoperability and reproducibility; it does not by itself solve retention measurement, cost accounting, or longitudinal replay.

Agent World Model (AWM) [Wang et al., 2026m] synthesizes 1,000 executable, SQL database-

backed tool-use environments exposed via a unified MCP interface, each with auto-generated tasks, schemas, and verification code. The tasks are multi-turn tool-use problems whose rewards can be checked against executable database state. It makes large-scale agentic RL feasible without manual environment engineering, but addresses environment scalability rather than SI-native measurement.

Classical Benchmarks as Control Baselines Static single-shot benchmarks remain useful as controls, but they should not be mistaken for SI-native evaluation. **AgentBench** [Liu et al., 2024] and **LoCoMo** [Maharana et al., 2024] test multi-domain agent capability and long-context memory respectively. AgentBench covers operating-system, database, knowledge-graph, game, puzzle, household, shopping, and browsing tasks; LoCoMo tests long-range memory over long multi-session conversations. **WebArena** [Zhou et al., 2023] and **WorkArena** [Drouin et al., 2024] extend web-agent evaluation to realistic sites. WebArena uses self-hosted web tasks such as shopping, forum, repository, map, and content-management workflows, while WorkArena tests ServiceNow-style enterprise knowledge-work tasks. **ALFWorld** [Shridhar et al., 2021] grounds text instructions in an embodied environment by asking agents to execute household goals through text-game states aligned with ALFRED-style embodied tasks. **OSWorld** [Xie et al., 2024], **ToolBench** [Qin et al., 2024], **LiveCodeBench** [Jain et al., 2024], **MLE-bench** [Chan et al., 2024], and **TheAgentCompany** [Xu et al., 2025a] similarly extend task realism or domain breadth. OSWorld tests real desktop application workflows; ToolBench tests API-use instructions over thousands of real APIs; LiveCodeBench tests freshly collected programming-contest problems; MLE-bench tests end-to-end Kaggle-style machine-learning engineering; and TheAgentCompany tests long-horizon office-style work. What they share is that they broaden *what* is evaluated more than *how* self-improvement is evaluated.

Summary of Requirement Coverage Table 5 summarizes how current benchmarks cover the six targets defined in Table 4. Agentified platforms such as AAA / AgentBeats are discussed separately because they are evaluation protocols or ecosystems rather than single benchmarks.

A New Evidence and Reliability Layer Several recent benchmarks and meta-evaluation papers add an orthogonal lesson: SI evaluation also needs to say how trustworthy the score itself is. **AgentProp-Bench** [Gurram, 2026] tests tool-using agents on 2,000 tasks and 2,300 traces across four domains, with a 100-label human-validated subset for judge-reliability audits, error-propagation analysis, and runtime mitigation. Its result is not a new SI score; it shows that simple automatic judging can mischaracterize interactive failures. Evidence-supported score bounds [Gao and Zhou, 2026] add an outcome-evidence layer to AndroidWorld, AgentDojo, AppWorld, tau3-bench retail, and MiniWoB, forcing each run into evidence pass, evidence fail, or unknown before reporting score bounds. Trajectory-level consistency tests [Raj et al., 2026] then treat reliability as a statistical property, using output-level and trajectory-level metrics under semantically preserving perturbations on three agentic benchmarks.

The same point appears at the component level. Counterfactual trace auditing [Zhou et al., 2026c] compares with-skill and without-skill trajectories on 49 SWE-Skills-Bench software-engineering tasks with Claude, segments and aligns the traces, and emits Skill Influence Pattern annotations. The pass-rate delta is only +0.3 percentage points, but the audit finds 522 behavioral influence instances, showing that final score can miss large trajectory changes. **LongMemEval-V2** [Wu et al., 2026a] tests whether long-term memory makes a web agent behave like an experienced colleague, using 451 curated questions over histories of up to 500 trajectories and 115M tokens to probe static state recall, dynamic state tracking, workflow knowledge, environment gotchas, and premise awareness. **STALE** [Chao et al., 2026] uses 400 expert-validated conflict scenarios and 1,200 queries across more than 100 everyday topics to test whether agents can resolve outdated states, resist stale premises, and adapt downstream behavior when stored memories become invalid.

Benchmark	Year	Held-out gain	Backward retention	Improvement efficiency	Path attribution	Longitudinal stability	Safety non-regression
Direct SI-oriented (§8.2.2)							
LifelongAgentBench [Zheng et al., 2025b]	2025	○	○	×	×	○	×
StuLife [Cai et al., 2025]	2025	○	○	×	○	○	×
SkillsBench [Li et al., 2026c]	2026	○	○	×	×	×	×
SE-Bench [Yuan et al., 2026]	2026	×	○	×	×	×	×
SkillLearnBench [Zhong et al., 2026a]	2026	○	×	○	○	×	×
EvoAgentBench [EverMind-AI, 2026]	2026	○	○	○	×	×	×
Dynamic and deployment-facing (§8.2.2)							
tau-bench [Yao et al., 2024]	2024	×	×	×	×	○	×
SWE-bench [Jimenez et al., 2024]	2024	×	×	×	×	×	×
SWE-bench-Live [Zhang et al., 2025e]	2025	×	×	×	×	○	×
SWE-MERA [Adamenko et al., 2025]	2025	×	×	×	×	○	×
SWE-rebench [Badertdinov et al., 2025]	2025	×	×	×	×	○	×
ClawArena [Ji et al., 2026]	2026	×	×	×	×	○	×
Claw-Eval [Ye et al., 2026a]	2026	×	×	×	×	○	○
AgentMemoryBench [Zihang Ma et al., 2026]	2026	○	○	×	×	○	×
DomusMind [Rong Xu, 2026]	2026	×	○	×	×	○	×
WildClawBench [Ding et al., 2026]	2026	×	×	×	×	×	○
Agent World Model [Wang et al., 2026m]	2026	×	×	×	×	×	×
Evidence, reliability, and process-aware diagnostics							
Frontier-Eng [Chi et al., 2026]	2026	×	×	○	×	○	×
AgentProp-Bench [Gurram, 2026]	2026	×	×	×	×	×	○
Evidence bounds [Gao and Zhou, 2026]	2026	×	×	×	×	×	×
Consistency tests [Raj et al., 2026]	2026	×	×	×	×	○	×
CTA [Zhou et al., 2026c]	2026	×	×	○	○	×	×
LongMemEval-V2 [Wu et al., 2026a]	2026	×	○	×	×	○	×
STALE [Chao et al., 2026]	2026	×	○	×	×	○	×
RobustBench-TC [Zhou et al., 2026d]	2026	○	×	×	×	○	○
CTFusion [Lee et al., 2026a]	2026	×	×	×	×	○	×

Table 5 | Coverage of current benchmarks against the minimum targets of self-improvement evaluation, grouped by family and ordered chronologically within each group. ○ indicates partial or explicit support; × indicates that the target is largely absent from standard reporting. Evidence and reliability diagnostics are included because they strengthen score interpretation even when they are not SI-native benchmarks. Section references point to the benchmark landscape discussion in §8.2. SIP-Bench [Wang, 2026] and agentified platforms such as AAA / AgentBeats are discussed separately as protocols or ecosystems rather than single benchmarks.

Finally, evaluation environments are becoming more process-aware. **Frontier-Eng** [Chi et al., 2026] evaluates self-evolving agents on 47 human-verified engineering optimization tasks across five categories, where agents repeatedly propose artifacts, execute them, receive continuous verifier reward, and revise under a fixed interaction budget. RobustBench-TC [Zhou et al., 2026d] probes the sim-to-real gap for tool-use agents with 22 perturbation types grounded in verified GitHub issues or documented tool-calling failures, organized across observation, action-space, reward-relevant metadata, and transition dynamics. **CTFusion** [Lee et al., 2026a] evaluates cybersecurity agents through live CTF streaming: its experiments use three LLMs, two agents, and five Live CTFs while preserving per-agent independence and forwarding only the first correct flag per challenge. Together, these works suggest that a future SI benchmark should report not only gain and retention, but also the evidence boundary around each score.

8.3. SIP-Bench as a Protocol Layer

One effort in that direction is **SIP-Bench** [Wang, 2026], an open-source protocol layer designed to turn episodic benchmark runs into longitudinal, reproducible SI evaluation. It is not meant to replace task benchmarks such as SkillsBench, EvoAgentBench, or tau-bench. Instead, it processes existing benchmark tasks into a checkpointed evaluation protocol for one evolving agent.

Its difference from a standard continual-learning benchmark is the evaluation target. Continual learning mainly asks whether a learner remains stable while moving through a task sequence. SIP-Bench keeps that retention question, but makes held-out generalization the central SI test: after adaptation, does the same evolving agent improve on tasks that were not part of the adaptation stream?

The protocol therefore imposes a T0/T1/T2 structure with explicit replay/adapt/held-out partitions. Replay measures backward retention, the adapt partition records what experience the agent actually used, and the held-out partition measures cross-task generalization after self-improvement. First-class records for gain, retention, stability, cost, path type, and safety delta then make the result comparable across otherwise heterogeneous benchmark environments. Figure 9 compresses this SIP-Bench protocol.

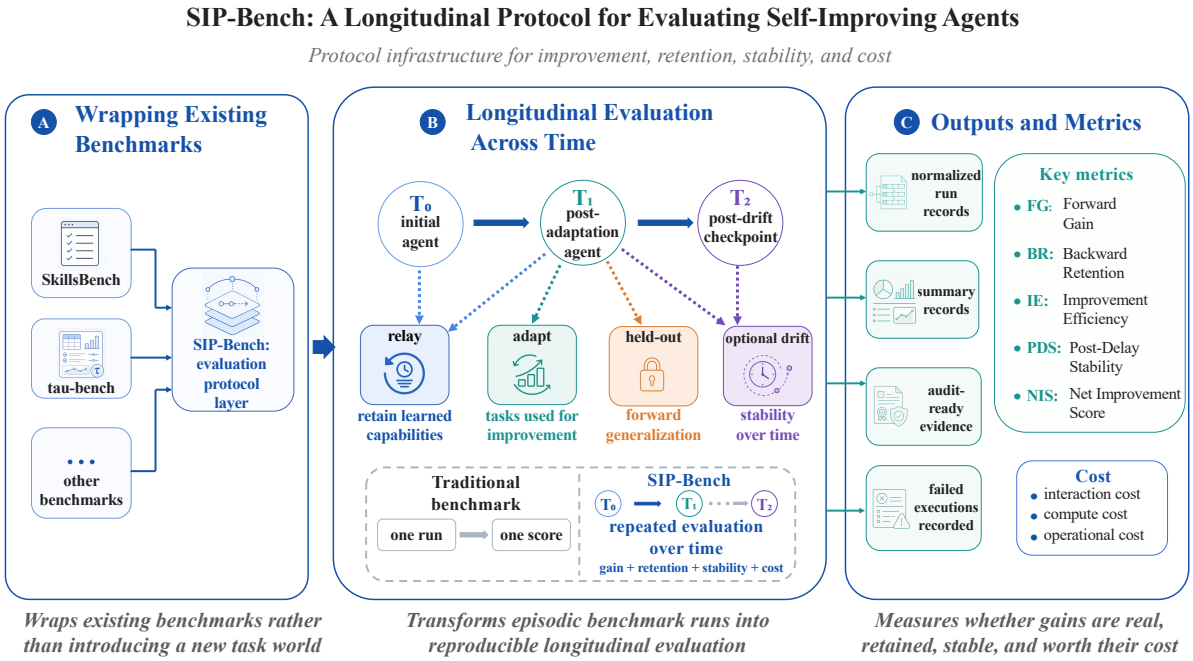

Figure 9 | SIP-Bench protocol. The key idea is to evaluate one evolving agent across T0/T1/T2 checkpoints rather than only report a final score after adaptation.

In that sense, SIP-Bench is less a competitor to existing benchmarks than a candidate measurement layer that could make their SI claims easier to compare, audit, and reproduce.

9. Safety: Self-Improvement as a Moving Attack Surface

9.1. Why Self-Improvement Changes the Safety Calculus

Most safety methods for large language models assume, at least implicitly, that the object being constrained is mostly fixed after deployment. RLHF and Constitutional AI, for example, shape a base model toward a more acceptable behavioral distribution before release [Bai et al., 2022, Ouyang et al., 2022]. They remain necessary starting points for agent safety, but self-improving agents change the unit of analysis. After deployment, the effective system is no longer just an aligned model; it is a composite of model behavior, skills, memory, tools, protocol interfaces, feedback signals, and lightweight adaptation mechanisms that can change during ordinary operation.

Safety for self-improving agents therefore cannot be reduced to the familiar claim that “more

capable agents are more dangerous." Capability matters, but the distinctive problem is post-deployment mutability. A safety claim made about the system at one moment may no longer describe the system after a skill is installed, memory is rewritten, a tool policy changes, or an adaptation update is accepted. In this sense, safety expands from aligning a snapshot to governing a process. The relevant question is whether a deployed system can preserve previously audited boundaries across multi-turn execution, updates, and cross-task transfer, rather than whether the current model avoids one unsafe output in isolation.

This is why one-time audits are structurally weak in this setting. For many conventional deployments, benchmark results, red-team findings, and safety approvals remain informative after launch because the deployed object changes only modestly. Self-improving agents weaken that assumption: the next evolved version may no longer be the system that was tested. A useful safety target must therefore include process-level properties: updates should not silently expand unsafe behavior, the sources of new behavior should remain traceable through mutable state, and a compromised or degraded agent should be recoverable to a previously certified state. ClawHavoc gives a concrete entry point into this view: once skill acquisition becomes part of the improvement loop, the same route to capability growth can also become a route to compromise [Jiang et al., 2026b]. The AI-45° Law offers a compatible diagnostic lens, arguing that capability and safety should advance together along a balanced reference line rather than letting capability accelerate while safety lags behind [Yang et al., 2024a]. For self-improving agents, the warning sign is not a single unsafe output but an update process in which new skills, memories, tools, or adaptation steps raise capability faster than the corresponding control and assurance mechanisms. High-risk states should therefore define hard stop conditions, while measurable early-warning thresholds should trigger deeper assessment before the system crosses into unacceptable risk [Shanghai AI Lab, 2025a, Yang et al., 2024a].

9.2. Threat Models Across Mutable Harness Surfaces

The threat model is therefore best organized around mutable harness components rather than around generic categories such as jailbreaks, privacy leakage, or prompt injection in isolation. Any interface that can be written to, extended, or recomposed after deployment becomes part of the attack surface. Four direct threat classes are especially salient: skill supply-chain attacks, memory poisoning and memory steering, reward or feedback manipulation, and workflow or protocol-level exploits. A fifth, cross-cutting concern is alignment drift across the adaptation loop. The attack–defense literature on LLM conversation safety already treats attacks, defenses, and evaluations as coupled objects [Dong et al., 2024]; for self-improving agents, this coupling moves from conversation outputs to the mutable harness itself.

The SafeWork-F1 framework gives a compatible operational vocabulary. Its E-T-C analysis decomposes frontier risk into deployment environment, threat source, and enabling capability, and its risk taxonomy distinguishes misuse, loss of control, accident, and systemic risk [Shanghai AI Lab, 2025a]. This is a better fit for self-improving agents than a prompt-only threat model because the same model behavior can be benign or dangerous depending on tool privileges, environmental affordances, attacker access, and the capability combinations activated by the current workflow. Table 6 therefore merges the chapter's surface-level threat map with the corresponding defense and assurance implications.

A recent cluster of safety studies further sharpens this mutable-harness threat model. Skill artifacts are no longer risky only because they may contain malicious code: `SKILL.md` text can steer discovery and selection, declared capabilities can diverge from actual behavior at registry scale, least-privilege violations can be task-conditioned, and adaptive red-teamers can iteratively mutate skills until they pass static audits [Saha et al., 2026, Wu et al., 2026b,d, Zhou, 2026]. At the memory and execution layers, graph-memory poisoning, memory misevolution, stale-state

Mutable surface	Attack mechanism	Defense and assurance implication	Residual gap
Skill library	Malicious or misdeclared skills persist across reuse [Jiang et al., 2026b, Saha et al., 2026, Wu et al., 2026b,d].	Admission tests, least privilege, registry audits, versioning, and isolation [Guo et al., 2026b, Wang et al., 2026h, Wu et al., 2024c].	Adaptive import can still bypass static vetting [Zhou, 2026].
Memory store	Poisoned, stale, or private state re-enters later reasoning [Chao et al., 2026, Cui et al., 2026, Luo et al., 2026, Patlan et al., 2025, Tian et al., 2025, Xu et al., 2026c].	Write governance, lifecycle control, invalidation, self-checking memory, and replay [Wei et al., 2025a, Wen et al., 2026].	Persistence keeps attacks alive across tasks [Xie et al., 2026].
Tool and protocol layer	Connectors and workflows compose into unintended execution chains [Fang et al., 2025b, Ferrag et al., 2025, Liang et al., 2026b, Zhang et al., 2026a].	Authorization, schema validation, injection isolation, workflow monitoring, and state verification [Li et al., 2025a, Wang et al., 2026j, Wu et al., 2024c].	Composition-level failures remain hard to certify.
Feedback and adaptation loop	Corrupted feedback makes unsafe updates look like improvements [Su et al., 2025a].	Evaluator isolation, constrained updates, risk-aware rewards, repair supervision, and early warnings [Shanghai AI Lab, 2025a, Yang et al., 2024a, Yin et al., 2026].	Few guarantees under compromised feedback.
Whole evolving harness	Successive updates move the system outside its audited boundary.	Continuous red-teaming, benchmark refresh, trajectory evaluation, hard stops, and re-certification [Li et al., 2026g, Zhang et al., 2025b, 2024c, Zhou et al., 2025].	No maintained end-to-end invariant yet.

Table 6 | A merged attack–defense map for mutable self-improving agents. The table combines the improvement surface, representative attack mechanism, and current control or assurance implication.

failures, OS-level behavioral jailbreaks, and recursive workflow amplification show that safety failures can persist, compose, and execute even when local text-level checks appear healthy [Chao et al., 2026, Liang et al., 2026b, Luo et al., 2026, Xie et al., 2026, Zhang et al., 2026a].

9.2.1. Skill Supply-Chain Attacks

ClawHavoc is the clearest skill-level case study. The *SoK: Agentic Skills* paper describes it as a supply-chain event in which nearly 1,200 malicious skills infiltrated a major agent marketplace, leading to the large-scale exfiltration of API keys, cryptocurrency wallets, and browser credentials [Jiang et al., 2026b]. The attack did not first require breaking the model. It entered through the skill acquisition interface that self-improving agents rely on to install, select, and reuse new capabilities over time. Once capability expansion depends on third-party skills, the marketplace is no longer just an ecosystem convenience; it becomes part of the agent's security boundary. This is also why skill testing and marketplace auditing matter for safety, not only utility: SkillTester frames agent skills as objects that need both utility and security evaluation, while SkillProbe treats emerging skill marketplaces as audit targets in their own right [Guo et al., 2026b, Wang et al., 2026h].

The case also illustrates a compound attack chain: malicious skills first crossed the marketplace distribution boundary, then abused skill payloads and prompt-in-skill surfaces to shape downstream execution, and finally reached sensitive assets already visible to the agent [Jiang et al., 2026b]. This makes the attack more persistent than a single malicious prompt. A prompt attack often ends with the current context window, whereas a malicious skill can be installed as a seemingly legitimate capability package, cached, invoked again, and reused across future tasks. Self-improvement makes this threat especially serious because the channel for improvement and the channel for compromise become partially identical.

Newer skill-safety work decomposes this boundary into more precise failure modes. *Under the Hood of SKILL.md* shows that natural-language skill metadata is itself operational: description-only changes can affect discovery, selection, and governance outcomes even when the underlying executable payload is unchanged [Saha et al., 2026]. *Behavioral Integrity Verification* formalizes the mismatch between declared and actual skill capabilities, reporting widespread deviations across nearly 50,000 registry skills and showing that capability comparison can support malicious-skill detection [Wu et al., 2026d]. *SkillScope* adds a complementary least-privilege view: the same skill action may be legitimate for one task and over-privileged for another, so safe admission requires task-conditioned privilege analysis rather than only global skill labels [Wu et al., 2026b]. Finally, *Proteus* shows why static vetting is insufficient: a feedback-driven attacker can repeatedly mutate

a skill until it both bypasses audit and produces runtime harm [Zhou, 2026]. Together, these results move skill safety from payload scanning toward lifecycle governance over documentation, declared behavior, privilege scope, and adaptive adversaries.

9.2.2. Memory Poisoning and Memory Steering

The second risk surface is memory. For a self-improving agent, memory is not merely a log of past interactions; it is a conditioning variable for future behavior. *From Storage to Steering* makes this point directly by showing that memory retrieval can dominate downstream control flow, and reports that more than 90% of trials remain vulnerable to such attacks even under strict safety constraints [Xu et al., 2026c]. The result reframes memory security as a control-flow problem, not only an information hygiene problem. Once observations, plans, and user preferences are written into reusable memory, they can re-enter the reasoning path of later tasks.

InjecMEM moves this threat model closer to realistic deployment. Attackers do not necessarily need direct read or edit access to the memory store; a single interaction may be enough to implant content that later reappears in responses to related downstream queries [Tian et al., 2025]. In web-agent settings, *Context Manipulation Attacks* further shows that plan injection can bypass existing prompt-injection defenses and achieve up to three times higher attack success rates than comparable prompt-based attacks [Patlan et al., 2025]. These studies point to the same trade-off: persistence increases long-horizon competence, but it also extends the lifespan of attack effects.

This dual role makes memory especially consequential in self-improving agents. The same persistence mechanism that enables cross-task improvement, experience retention, and behavioral coherence also makes agents harder to constrain and easier to manipulate over long horizons. MemoryGraft reaches the same conclusion from the angle of poisoned experience retrieval: a compromised memory can persist as an operational precedent rather than disappear with the triggering prompt [Srivastava and He, 2025]. Once memory enters the self-improvement loop, it becomes both a capability primitive and a control primitive.

Recent work further shows that memory risk is not limited to flat textual poisoning. *ShadowMerge* attacks graph-based agent memory through relation-channel conflicts, so a poisoned relation can be merged near a benign anchor and later retrieved for the victim query [Luo et al., 2026]. *MemEvoBench* frames safety degradation as memory misevolution: repeated exposure to misleading updates can gradually shift behavior even when individual interactions look plausible [Xie et al., 2026]. *Spore* adds a privacy dimension by targeting inference-time agent memory rather than training data, showing that contextual memory can become an extraction surface [Cui et al., 2026]. STALE is not an attack paper, but it matters for safety because stale memories can remain behaviorally active after the world state they encode has changed; a system that cannot invalidate implicit old states may safely retrieve evidence and still act on the wrong premise [Chao et al., 2026]. The defensive implication is that memory governance must include state invalidation, provenance, conflict repair, and privacy boundaries, not only better retrieval filters.

9.2.3. Workflow, Tool, and Protocol Exploits

The third major risk surface lies at the workflow, tool, and protocol layer. *From Prompt Injections to Protocol Exploits* broadens the threat model from simple input manipulation to host-to-tool and agent-to-agent communication, and systematizes more than thirty attack techniques spanning input manipulation, model compromise, system and privacy attacks, and protocol-level vulnerabilities [Ferrag et al., 2025]. Its value for this chapter is less any single number than the change in unit of analysis: from prompt to execution chain.

This expansion is especially relevant to self-improvement. As agents adopt new connectors, tool schemas, Model Context Protocol (MCP)-like interfaces, and multi-stage workflows, they expand

their capability boundary together with their trust boundary and failure surface. MCP-specific safety work sharpens this point: once third-party services are brought into the agent loop, the system also inherits service-provider incentives, implementation bugs, and policy gaps that sit outside the model developer's direct control [Fang et al., 2025b]. Many failures do not begin with the model producing a clearly dangerous instruction. They emerge when individually reasonable components are composed into an unintended execution path. The systemic risk of self-improvement therefore lies partly in recomposition: the more freely a deployed agent reorganizes its tools and interaction patterns, the more opportunities arise at the boundaries between those components.

The execution layer also exposes failures that semantic safety checks can miss. LITMUS evaluates behavioral jailbreaks in real operating-system environments and highlights "execution hallucination," where an agent verbally refuses a harmful request while the dangerous operation has already occurred at the OS layer [Zhang et al., 2026a]. Mobius Injection pushes the concern from single harmful actions to infrastructure-level disruption: one message can trigger recursive execution and large call amplification across agent components [Liang et al., 2026b]. In multimodal settings, visual injection work describes trust-boundary confusion: the agent must decide whether an environmental signal is a legitimate cue or an adversarial command-like artifact [Chang et al., 2026]. These cases make the boundary between reasoning and execution central to safety. A self-improving agent cannot be certified by checking text alone if it can also mutate files, call services, propagate workflows, or act on visual state.

9.2.4. Reward and Feedback Manipulation

A fourth risk surface is the feedback channel that decides which changes count as improvements. Once a deployed agent uses evaluation feedback to decide what should persist, an attacker no longer has to control the immediate action directly. Corrupting the improvement signal can make the system preserve a misleading memory, overvalue a risky skill, or treat a fragile workflow as successful. In this form, reward hacking becomes a harness-level attack: the agent is not merely tricked into one bad step, but steered into retaining the wrong lesson for future tasks. Current surveys identify reward hacking and emergent misalignment as autonomy-induced risks [Su et al., 2025a], but the agent-specific defense stack for compromised feedback loops remains less mature than defenses for memory and tool-use attacks.

9.2.5. Alignment Drift as an Organizing Lens

Together, these attack surfaces motivate a broader organizing concept: alignment drift. The current literature provides threat models, vulnerabilities, and failure modes, but not yet a unified formal theory of long-horizon drift. Recent surveys of agent security nevertheless identify memory poisoning, tool misuse, reward hacking, and emergent misalignment as qualitatively novel risks induced by agent autonomy [Su et al., 2025a]. Work on governed evolving memory makes the same point at the memory layer: stability and safety have to be maintained as the agent's stored state changes, not only checked at the moment of deployment [Lam et al., 2026]. Multi-agent settings add another reason to treat drift as process-level rather than local: *Agent Smith* shows that a single adversarial image inserted into one multimodal agent can propagate harmful behavior through agent-agent interaction, turning compromise into a population-level dynamic [Gu et al., 2024]. In this chapter, alignment drift is best treated as a synthesis lens over observed failure surfaces rather than as a fully established theorem.

9.3. Runtime Control for Self-Improving Agents

After the threat surfaces are identified, the next question is how a deployed self-improving agent can be constrained during execution. Current methods can be read as a two-layer control stack: pre-deployment alignment shapes the initial model, while runtime governance constrains the mutable harness after deployment. This distinction mirrors the structural mismatch introduced by self-improvement. The field is increasingly capable of aligning models before release, but it is less mature at constraining an agentic system that continues to change after release.

9.3.1. Pre-Deployment Alignment Remains Necessary

The first layer is pre-deployment alignment, exemplified by RLHF and Constitutional AI [Bai et al., 2022, Ouyang et al., 2022]. These methods still matter because they provide the aligned starting point from which later safeguards operate. In the present setting, their role is to establish a minimally trustworthy base model before the agent enters deployment.

The limitation is equally important. These methods constrain model behavior before deployment, but they do not directly govern the post-deployment evolution of skill sets, memory stores, tool policies, or lightweight adaptation modules. In other words, they can reduce risk at the starting point, but they do not automatically guarantee that subsequent skill installation, memory accumulation, or online adaptation remains inside the originally audited boundary.

9.3.2. Runtime Governance as the Practical Defense Layer

The second layer is runtime governance. Rather than treating these methods as new alignment paradigms, it is more useful to view them as governance primitives for deployed agents: mechanisms that constrain what can be stored, what can be executed, and which deviations can be intercepted during the execution loop. *IsolateGPT* shows that execution isolation is already engineering-feasible. Its abstract reports that the architecture mitigates multiple security, privacy, and safety issues while keeping the performance overhead below 30% for three-quarters of tested queries [Wu et al., 2024c]. Isolation is therefore a plausible systems design for limiting blast radius, not just a conceptual recommendation.

A complementary industrial design pattern is separation of concerns. In the Anthropic/Claude Code framing reported by Pachaar, "the model decides what to attempt, the tool system decides what to permit" [Pachaar, 2026]. The accompanying permission pattern has three checkpoints: trust establishment when a project is loaded, per-call authorization before tool execution, and explicit human confirmation for high-risk operations. This matters because it moves safety from an after-the-fact output filter into the harness itself: the model may propose an action, but the tool and validation layer owns the authority to execute it.

Other defenses target different mutable surfaces. *AgentSys* turns hierarchical memory management into an explicit control mechanism and reports attack success rates of 0.78% and 4.25% on AgentDojo and ASB, respectively, while slightly improving benign utility [Wen et al., 2026]. *DRIFT* represents dynamic rule validation and injection isolation: rather than relying on a fixed static policy, it derives a minimal function trajectory from the user query and monitors deviations and memory-stream contamination during execution [Li et al., 2025a]. *A-MemGuard* similarly treats memory security as a runtime problem. Its central idea is that memory should become self-checking and self-correcting, and its abstract reports more than 95% reduction in attack success rates at minimal utility cost [Wei et al., 2025a]. AgentWatcher adds a long-context monitoring pattern by attributing an agent action to influential context segments before applying explicit prompt-injection rules [Wang et al., 2026j]. SafeAgent, SARC, and Sovereign Agentic Loops push the same idea into runtime architecture: the control plane should mediate actions around the

agent loop, compile constraints into pre-action, action-time, post-action, and escalation sites, or validate structured intents against true system state before execution [Besanson, 2026, He and Yu, 2026, Liu et al., 2026a]. Table 6 summarizes how these defenses map onto the main attack vectors. Taken together, these works shift the practical safety question from how to train a safer model in isolation to how to make each execution step harder to corrupt.

The AI-45° line also suggests a positive control objective: safety should become part of the improvement process rather than an external tax imposed after capability gains. SafeWork-R1 is a useful parameter-side example of this idea. Built with the SafeLadder framework, it uses progressive safety-oriented reinforcement learning, neural and rule-based verifiers, and inference-time interventions to make safety reasoning and self-reflection part of the model's competence; the report claims a 46.54% average improvement over Qwen2.5-VL-72B on safety-related benchmarks without sacrificing general capability [Shanghai AI Lab, 2025b]. FATE gives an agent-trajectory version of the same principle by turning verifier-scored failure trajectories into repair supervision for safety alignment [Yin et al., 2026]. For self-improving agents, the important lesson is not that model-side coevolution solves harness safety by itself, but that adaptation loops need explicit safety verifiers, repair signals, and step-level intervention points if they are to stay near the 45° capability–safety trajectory.

9.3.3. Continuous Assurance and Re-Certification

Runtime controls are not the end of the safety problem. Even if the control stack in Section 9.3 reduces risk during execution, a separate assurance question remains: how can any safety claim stay valid once the system keeps changing? The issue is no longer only how to constrain a single execution, but how to maintain confidence in a system whose skills, memories, evaluators, and tool-use patterns evolve over time. Safety evaluation is therefore not just a gate before release. It becomes a governance track that runs alongside the improvement loop itself.

Benchmarking becomes important at this point because it exposes the gap between local controls and system-level assurance. *Agent Security Bench (ASB)* quantifies agent vulnerability across 10 scenarios, more than 400 tools, 23 attack and defense methods, and 8 evaluation metrics, and reports a highest average attack success rate of 84.30% [Zhang et al., 2025b]. *Agent-SafetyBench* broadens the evidence with 349 interaction environments and 2,000 test cases across 8 risk categories and 10 common failure modes; none of the evaluated agents achieves a safety score above 60% [Zhang et al., 2024c]. *ATBench* moves the unit of evaluation to full trajectories, using 1,000 audited trajectories with heterogeneous tool pools and delayed triggers to diagnose long-horizon safety failures rather than isolated unsafe responses [Li et al., 2026g]. Together, these results suggest that system-level vulnerability remains high even as the defense literature grows. Component-level mitigations have not yet accumulated into a reliable system-level safety guarantee.

AutoRedTeamer and *RedDebate* further suggest that safety evaluation itself is becoming agentic. The former emphasizes continuous attack discovery through a multi-agent architecture with memory-guided attack selection, achieving 20% higher attack success rates on HarmBench while reducing computational cost by 46% relative to prior approaches [Zhou et al., 2025]. The latter combines multi-agent debate with long-term memory and reports reductions in unsafe outputs of 17.7% from debate alone and more than 23.5% when memory is added [Asad et al., 2025]. For this chapter, their significance is not that they already provide certification, but that they move safety evaluation from one-off testing toward an adaptive process that can absorb new attack patterns over time.

Taken together, these benchmarks and red-teaming systems support a broader conclusion: self-improving agents require continuous auditing and re-certification, not deployment-time approval

alone. Without such a track, self-improvement may produce continuous capability growth without continuous controllability.

9.3.4. Limits of the Current Control Stack

These methods also share a limitation. Most address a single surface or a narrowly defined class of failure. They are better understood as strong local mitigations than as demonstrated system-level invariants. The strongest current results are still local runtime defenses rather than end-to-end safety results for self-improving agents: *A-MemGuard* and *AgentSys* reduce memory-attack success substantially, while system-level benchmarking remains sobering across ASB, Agent-SafetyBench, and ATBench [Li et al., 2026g, Wei et al., 2025a, Wen et al., 2026, Zhang et al., 2025b, 2024c]. The field can now reduce risk for particular components, but it does not yet show that a self-improvement loop can preserve a global safety boundary over long periods of operation.

An ideal post-deployment control stack would treat governance as a full-lifecycle obligation. It would certify the initial model and harness before release, monitor deployed behavior as the agent acts, govern every accepted update, support rollback and recovery after unsafe changes, and require re-certification after material drift. The scope would have to cover all mutable surfaces: skill admission and versioning, memory writing and invalidation, tool and protocol privileges, feedback integrity, and lightweight adaptation. The objective is not simply that one snapshot passed a benchmark, but that every accepted transition preserves a maintained safety boundary.

The core bottleneck is architectural rather than merely data-related. Current safety methods mostly constrain a pre-deployment snapshot, whereas self-improvement changes the post-deployment process. Once skill installation, memory updates, tool and protocol expansion, feedback selection, and lightweight adaptation occur after training, the alignment stack does not automatically extend to those changes. The open safety question is therefore whether we can define and maintain a formal safety invariant for a self-improving agent across its full sequence of skill, memory, tool, feedback, and parameter updates, rather than certifying the model only once before deployment.

10. Open Problems

10.1. Elicitation versus Acquisition

External adaptation modifies the harness while leaving the model weights unchanged; the behavior it produces is therefore bounded by the capabilities already present in the base model. Whether a measured gain reflects newly acquired competence or the elicitation of latent competence through a better skill, memory, or context is rarely tested. The two readings have opposite implications for the external path: under pure elicitation, deployed agents meet a ceiling that only parameter updates can lift, whereas genuine acquisition would let weight-frozen systems keep improving. Current protocols, which compare an adapted agent against its own unadapted baseline, do not separate the two, and the effective reach of harness-level learning is correspondingly unclear.

10.2. Consolidating External Experience into Parameters

Harness-level artifacts are cheap to revise but stay tied to the contexts that retrieve them, whereas parameter updates generalize more broadly at higher cost and with little reversibility. No account specifies when a recurrent external pattern has become stable and general enough to internalize. Continual learning frames the risk: internalization can overwrite earlier competence [Kirkpatrick et al., 2017], and its value is better read from backward and forward transfer than from target-task

accuracy [Lopez-Paz and Ranzato, 2017, van de Ven and Tolias, 2019]. A prior question is what constitutes a sufficient summary of accumulated experience, since the harness already compiles raw interaction into a compressed reservoir before any update is committed. With no promotion criterion, the decision of what to write to the weights rests on manual judgment.

10.3. Credit Assignment under Weak Feedback

Self-improvement has advanced fastest where success admits a cheap and reliable check, as in program execution or formal verification. Deployment usually offers weaker signal: objectives are underspecified, rewards are sparse, and outcomes appear only at the end of long trajectories. The resulting failures are familiar. An outcome-level signal does not indicate which step, skill, or memory was responsible, so credit assignment over long horizons remains unsolved; and optimizing against an approximate verifier rewards exploitation of its flaws as pressure on it grows. Perceptual tasks compound this, since experience arrives as observation streams that current systems neither index nor attribute with precision. Dense and trustworthy feedback for tasks without a ready verdict is a precondition for self-improvement beyond the domains where such a verdict already exists.

10.4. Stability of Self-Generated Experience

Learning from one's own interaction stream means learning largely from self-generated data: skills taken from the agent's own successes, memory written from its own judgments, rewards assigned by its own evaluators. Recursively training on self-produced data is known to narrow and distort the learned distribution, and an agent that curates its own experience is subject to the same pressure. With no periodic grounding in an external signal, a long-running agent can settle onto its own priors and lose competence it once held, a degradation that its own updates produce [Zheng et al., 2025a]. How much external grounding keeps accumulated experience informative, and how to detect this drift before it compounds, become visible only over deployment horizons that present evaluations seldom span.

10.5. Longitudinal Evaluation

Standard benchmarks report performance at a single time and can show that an agent does better on a target task without showing that it has improved as a system. No established relation links accumulated experience to capability in the way that scaling relations link data, parameters, and inference compute. A longitudinal protocol would re-evaluate the same evolving agent and report retention of earlier capabilities, cost per unit of gain, and any change in safety beside the target result; the backward and forward transfer metrics of continual learning apply directly [Lopez-Paz and Ranzato, 2017] but are rarely instrumented for deployed agents. Absent such measurement, a higher score is weak evidence of durable improvement.

10.6. Transfer across Model Versions

Skills, memory, and harness configuration are tuned to the model that produced them and often encode workarounds for that model's particular failure modes. A base-model upgrade can strand or invert this tuning, so experience accumulated over a long deployment does not carry across the transition. Which parts of accumulated experience are model-independent, and which are bound to a specific model and prompt format, has not been established. A representation that stays useful across versions, and that lets experience from many deployments be pooled, would allow durable lessons to outlast the frequent model changes that deployed systems undergo.

10.7. Coordination and Emergence in Multi-Agent Systems

Multi-agent systems are usually wired in advance, with designers fixing the roles, the communication channels, and the protocols while added agents carry out predetermined subtasks. Whether this organization can instead be acquired through the same adaptation that improves a single agent has had little study. A further question concerns emergence: whether a population can reach capability that no member holds for reasons of organization rather than of aggregate computation. Reported gains frequently reduce to added compute, and evidence for organizational effects is thin [Hu et al., 2024, Shang et al., 2024]. Learning the coordination structure and isolating genuinely collective capability are both unresolved.

10.8. Multimodal Experience Compilation

Text decomposes into discrete units whose relevance to an outcome can be isolated and scored. Visual experience has no such canonical structure. This asymmetry means that skill extraction, memory writing, and environment feedback cannot operate on visual traces with the same fidelity as on textual ones. That the systems with the most sustained experience reuse both rely on structured text interfaces [Wang et al., 2023, 2025d] confirms the gap. Imposing structure externally, for instance decomposing frames into semantic, spatial, and pixel layers aligned with environment metadata such as accessibility trees [Lu et al., 2024], could supply the missing granularity, but whether such representations transfer across tasks remains an open empirical question.

10.9. Safety under Post-Deployment Modification

Alignment is normally fixed before release, yet a self-improving agent keeps changing afterward, installing skills, writing memory, extending tools, and updating parameters outside the certified configuration [Bai et al., 2022, Ouyang et al., 2022]. Every surface that enables adaptation can also erode control, through gradual drift or deliberate manipulation. The meta-level is the sharpest case: a process that rewrites its own update rules or evaluators can void the guarantees it was meant to keep, and safe operation there presumes a calibrated estimate of the agent's own competence that a changing system does not reliably hold. Certifying a fixed model once does not establish a safety property over a sequence of post-deployment changes, and no current method supplies one. Improvement and control press in opposite directions, and their trade-off in continually evolving systems is untreated.

These problems are coupled. A criterion for internalizing experience presumes a way to verify that the internalized experience helped; longitudinal evaluation presumes that improvement can be attributed to a cause; safe self-modification presumes that the agent can estimate its own competence. Verification underlies each, and it is the capacity that scales least well as agents become more capable. The empirical object beneath all of them is still missing: over long horizons, the trajectory of improvement under repeated adaptation is not known to compound, saturate, or oscillate. Progress will likely come less from individually stronger agents than from an account of how experience becomes capability, and of when post-deployment adaptation can be measured, internalized, and trusted.

Author Contributions

We present below the contributions of all participating authors, specifying the primary responsible individual as well as the contributing participants for each section. The specific contributions of each author are detailed as follows:

- **Corresponding Author:** Ning Ding, Kaiyan Zhang, Bowen Zhou
- **Project Lead:** Che Jiang, Kaiyan Zhang
- **Introduction:** Che Jiang, Kaiyan Zhang
- **Harness as Experience Infrastructure:** Che Jiang
- **Skills: Experience Becomes Reusable Procedures:** Jincheng Zhong, Che Jiang, Kaiyan Zhang
- **Memory: Experience Becomes Persistent State:** Yu Fu, Hongyi Liu, Zhenzhao Yuan, Kaiyan Zhang
- **Environment: The Ceiling of Runtime Adaptation:** Kai Tian, Weizhi Wang, Che Jiang
- **RL and Continual Learning: Experience Becomes Parameter-Side Consolidation** Junlin Yang, Che Jiang, Kaiyan Zhang
- **Meta-Evolving Agents: Who Controls What to Evolve** Kaikai Zhao, Che Jiang, Kaiyan Zhang
- **Measuring Self-Improvement: What Current Benchmarks Still Miss** Yuchong Wang, Che Jiang
- **Safety: Self Improvement as a Moving Attack Surface** Tianwei Luo, Che Jiang
- **Open Problems:** Kaiyan Zhang, Che Jiang, Guoli Jia (Multimodal)
- **Other Contributions:**
  - Review and Editing: Yuxin Zuo, Xingtai Lv, Dianqiao Lei, Sihang Zeng, Yuru Wang, Xuekai Zhu, Ermo Hua, Yihao Liu, Youbang Sun, Biqing Qi, and all above authors

References

A2A Protocol Contributors. Agent2Agent (A2A) Protocol, 2026. URL https://github.com/a2aproject/A2A.

Emre Can Acikgoz, Cheng Qian, Jonas Hübotter, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Tool-r0: Self-evolving llm agents for tool-learning from zero data, 2026. URL https://arxiv.org/abs/2602.21320.

Pavel Adamenko et al. SWE-MERA: A dynamic benchmark for agenticly evaluating large language models on software engineering tasks. *arXiv preprint arXiv:2507.11059*, 2025.

AG-UI Protocol Contributors. AG-UI: The agent-user interaction protocol, 2026. URL https://github.com/ag-ui-protocol/ag-ui.

AgentBeats. Agentified Agent Assessment (AAA) & AgentBeats, 2026a. URL https://docs.agentbeats.dev/. Accessed 2026-04-21.

AgentBeats. AgentBeats Dashboard / Agent Registry, 2026b. URL https://agentbeats.dev/. Accessed 2026-04-21.

Pranjal Aggarwal, Graham Neubig, and Sean Welleck. Gym-anything: Turn any software into an agent environment. *arXiv preprint arXiv:2604.06126*, 2026. URL https://arxiv.org/abs/2604.06126.

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

Mubashara Akhtar et al. When AI benchmarks plateau: A systematic study of benchmark saturation. *arXiv preprint arXiv:2602.16763*, 2026. URL https://arxiv.org/abs/2602.16763.

AlphaEvolve team. AlphaEvolve: A gemini-powered coding agent for designing advanced algorithms, 2025. URL https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/. Google DeepMind blog post, published May 14, 2025.

Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems, 2026. URL https://arxiv.org/abs/2602.01848.

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026. URL https://arxiv.org/abs/2603.02766.

Ashwani Anand, Ivi Chatzi, Ritam Raha, and Anne-Kathrin Schmuck. MANTRA: Synthesizing SMT-validated compliance benchmarks for tool-using LLM agents. *arXiv preprint arXiv:2605.06334*, 2026. URL https://arxiv.org/abs/2605.06334.

Anthropic. Claude 3.7 sonnet and claude code, February 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet. Product announcement introducing Claude Code as Anthropic's first agentic coding tool in limited research preview.

Anthropic. Introducing claude opus 4.7, April 2026. URL https://www.anthropic.com/news/claude-opus-4-7. Anthropic model release and system card, accessed 2026-05-20.

Ali Asad, Stephen Obadinma, Radin Shayanfar, and Xiaodan Zhu. Reddebate: Safer responses through multi-agent red teaming debates. *arXiv preprint arXiv:2506.11083*, 2025.

Corpus2Skill Authors. Corpus2Skill: Distilling document corpora into hierarchical skill directories for agent navigation. *arXiv preprint arXiv:2604.14572*, 2026. URL https://arxiv.org/abs/2604.14572.

Ibragim Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025. URL https://arxiv.org/abs/2505.20411. NeurIPS 2025 Poster; 21,000+ tasks from 3,468 repos.

Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. Webgym: Scaling training environments for visual web agents with realistic tasks. *arXiv preprint arXiv:2601.02439*, 2026. URL https://arxiv.org/abs/2601.02439.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Baidu ERNIE Team. ERNIE 5.0 technical report. *arXiv preprint arXiv:2602.04705*, 2026. URL https://arxiv.org/abs/2602.04705.

Davide Baldelli, Ali Parviz, Amal Zouaq, and Sarath Chandar. Llms can't play hangman: On the necessity of a private working memory for language agents. *arXiv preprint arXiv:2601.06973*, 2026.

Pratyay Banerjee, Masud Moshtaghi, Shivashankar Subramanian, Amita Misra, and Ankit Chadha. Apex-mem: Agentic semi-structured memory with temporal reasoning for long-term conversational ai. *arXiv preprint arXiv:2604.14362*, 2026. URL https://arxiv.org/abs/2604.14362.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, et al. Nestful: A benchmark for evaluating llms on nested sequences of api calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33526–33535, 2025.

Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding agent interactions from real users in the wild. *arXiv preprint arXiv:2604.20779*, 2026. URL https://arxiv.org/abs/2604.20779.

Gaston Besanson. SARC: A governance-by-architecture framework for agentic AI systems. *arXiv preprint arXiv:2605.07728*, 2026. URL https://arxiv.org/abs/2605.07728.

Shuzhen Bi, Mengsong Wu, Hao Hao, Keqian Li, Wentao Liu, Siyu Song, Hongbo Zhao, and Aimin Zhou. Automating skill acquisition through large-scale mining of open-source agentic repositories: A framework for multi-agent procedural knowledge extraction. *arXiv preprint arXiv:2603.11808*, 2026. URL https://arxiv.org/abs/2603.11808.

Birgitta Böckeler. Harness engineering for coding agent users, April 2026. URL https://martinfowler.com/articles/harness-engineering.html. MartinFowler.com article, published April 2, 2026.

Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konecny, Stefano Mazzocchi, Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards federated

learning at scale: System design. In *Proceedings of Machine Learning and Systems*, volume 1, pages 374–388, 2019. URL https://proceedings.mlsys.org/paper_files/paper/2019/file/7b770da633baf74895be22a8807f1a8f-Paper.pdf.

Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A persistent memory layer for efficient, context-aware llm agents. *arXiv preprint arXiv:2603.19935*, 2026.

Andrew Borthwick, Stephen Ash, and Anthony Galczak. Robophd: Evolving diverse complex agents under tight evaluation budgets, 2026. URL https://arxiv.org/abs/2604.04347.

Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, et al. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *arXiv preprint arXiv:2508.19005*, 2025.

Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, et al. Qwen3-coder-next technical report. *arXiv preprint arXiv:2603.00729*, 2026. URL https://arxiv.org/abs/2603.00729.

Jun Shern Chan et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024. 75 Kaggle competitions.

Jiamin Chang, Minhui Xue, Ruoxi Sun, Shuchao Pang, Salil S. Kanhere, and Hammond Pearce. If you're waiting for a sign... that might not be it! mitigating trust boundary confusion from visual injections on vision-language agentic systems. *arXiv preprint arXiv:2604.19844*, 2026. URL https://arxiv.org/abs/2604.19844.

Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. STALE: Can LLM agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026. URL https://arxiv.org/abs/2605.06527.

Celia Chen. Unlocking the codex harness: how we built the app server, February 2026. URL https://openai.com/index/unlocking-the-codex-harness/. OpenAI engineering blog, accessed 2026-05-12.

Jiayi Chen, Junyi Ye, and Guiling Wang. From standalone LLMs to integrated intelligence: A survey of compound AI systems. *arXiv preprint arXiv:2506.04565*, 2025a. URL https://arxiv.org/abs/2506.04565.

Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive LLM agents. *arXiv preprint arXiv:2502.01600*, 2025b. URL https://arxiv.org/abs/2502.01600.

Mingyue Cheng, Jie Ouyang, Shuo Yu, Ruiran Yan, Yucong Luo, Zirui Liu, Daoyu Wang, Qi Liu, and Enhong Chen. Agent-R1: Training powerful LLM agents with end-to-end reinforcement learning. *arXiv preprint arXiv:2511.14460*, 2025a. URL https://arxiv.org/abs/2511.14460.

Zhoujun Cheng, Shibo Hao, Tianyang Liu, Fan Zhou, Yutao Xie, Feng Yao, Yuexin Bian, Yonghao Zhuang, Nilabjo Dey, Yuheng Zha, et al. Revisiting reinforcement learning for LLM reasoning from a cross-domain perspective. *arXiv preprint arXiv:2506.14965*, 2025b. URL https://arxiv.org/abs/2506.14965.

Zihao Cheng, Zeming Liu, Yingyu Shan, Xinyi Wang, Xiangrong Zhu, Yunpu Ma, Hongru Wang, Yuhang Guo, Wei Lin, and Yunhong Wang. Mem^{2}evolve: Towards self-evolving agents via co-evolutionary capability expansion and experience distillation, 2026. URL https://arxiv.org/abs/2604.10923.

Yizhe Chi, Deyao Hong, Dapeng Jiang, Tianwei Luo, Kaisen Yang, Boshi Zhang, Zhe Cao, Xiaoyan Fan, Bingxiang He, Han Hao, Weiyang Jin, Dianqiao Lei, Qingle Liu, Houde Qian, Bowen Wang, Situ Wang, Youjie Zheng, Yifan Zhou, Calvin Xiao, Eren Cai, and Qinhuai Na. Frontier-Eng: Benchmarking self-evolving agents on real-world engineering tasks with generative optimization. *arXiv preprint arXiv:2604.12290*, 2026. URL https://arxiv.org/abs/2604.12290.

Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. SkillRet: A large-scale benchmark for skill retrieval in LLM agents. *arXiv preprint arXiv:2605.05726*, 2026. URL https://arxiv.org/abs/2605.05726.

Meng Chu, Xuan Billy Zhang, Kevin Qinghong Lin, et al. Agentic world modeling: Foundations, capabilities, laws, and beyond. *arXiv preprint arXiv:2604.22748*, 2026. URL https://arxiv.org/abs/2604.22748.

ClawHub Marketplace. ClawHub: Open skill marketplace (audited at scale by skillprobe), 2026. URL https://clawhub.ai/ivangdavila/skills/marketplace. Use only as an open skill marketplace instance for admission/scope/versioning/cross-user distribution; audit object of SkillProbe (2603.21019). Do not write the broader commercial ecosystem.

João Coelho, Jingjie Ning, Jingyuan He, Kangrui Mao, Abhijay Paladugu, Pranav Setlur, Jiahe Jin, Jamie Callan, João Magalhães, Bruno Martins, and Chenyan Xiong. Deepresearchgym: A free, transparent, and reproducible evaluation sandbox for deep research. *arXiv preprint arXiv:2505.19253*, 2025. URL https://arxiv.org/abs/2505.19253.

Yu Cui, Ruiqing Yue, Hang Fu, Sicheng Pan, Zhuoyu Sun, Baohan Huang, Haibin Zhang, Cong Zuo, and Licheng Wang. Spore: Efficient and training-free privacy extraction attack on LLMs via inference-time hybrid probing. *arXiv preprint arXiv:2604.23711*, 2026. URL https://arxiv.org/abs/2604.23711.

Cursor. Meet the New Cursor. Cursor Blog, April 2026. URL https://cursor.com/blog/cursor-3. Official Cursor 3 announcement for the Agents Window, described as a unified workspace for building software with agents.

Parshva Daftari, Khush Patel, Shreyas Kapale, Jithin George, and Siva Surendira. Cognis: Context-aware memory for conversational ai agents. *arXiv preprint arXiv:2604.19771*, 2026. URL https://arxiv.org/abs/2604.19771.

Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, Xuantang Xiong, Lei Han, Zhiyuan Liu, and Maosong Sun. Multi-agent collaboration via evolving orchestration, 2025. URL https://arxiv.org/abs/2505.19591.

DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025. URL https://arxiv.org/abs/2512.02556.

Shuangrui Ding, Xuanlang Dai, Long Xing, Shengyuan Ding, Ziyu Liu, Jingyi Yang, Penghui Yang, Zhixiong Zhang, Xilin Wei, Yubo Ma, Haodong Duan, Jing Shao, Jiaqi Wang, Dahua Lin, Kai Chen, and Yuhang Zang. WildClawBench: An in-the-wild benchmark for AI agents in the openclaw environment, 2026. URL https://github.com/InternLM/WildClawBench. 60 tasks across 6 categories; all frontier models below 0.55.

Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026a. URL https://arxiv.org/abs/2604.18292.

Kris Shengjun Dong, Sahil Modi, Dima Nikiforov, Sana Damani, Edward Lin, Siva Kumar Sastry Hari, and Christos Kozyrakis. Kernelblaster: Continual cross-task cuda optimization via memory-augmented in-context reinforcement learning, 2026b. URL https://arxiv.org/abs/2602.14293.

Zhichen Dong, Zhanhui Zhou, Chao Yang, Jing Shao, and Yu Qiao. Attacks, defenses and evaluations for LLM conversation safety: A survey. *arXiv preprint arXiv:2402.09283*, 2024. doi: 10.48550/arXiv.2402.09283.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR, 2024. URL https://proceedings.mlr.press/v235/drouin24a.html.

Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026. URL https://arxiv.org/abs/2603.07670.

EverMind-AI. EvoAgentBench: A multi-domain benchmark for self-evolving agents, 2026. URL https://evermind-ai.github.io/EvoAgentBench/. No arXiv paper as of 2026-04; data from public leaderboard.

Zhiyuan Fan, Wenwei Jin, Feng Zhang, Bin Li, Yihong Dong, Yao Hu, and Jiawei Li. Evolving-RL: End-to-end optimization of experience-driven self-evolving capability within agents. *arXiv preprint arXiv:2605.10663*, 2026. URL https://arxiv.org/abs/2605.10663.

Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025a. URL https://arxiv.org/abs/2508.07407.

Junfeng Fang, Zijun Yao, Ruipeng Wang, Haokai Ma, Xiang Wang, and Tat-Seng Chua. We should identify and mitigate third-party safety risks in MCP-powered agent systems. *arXiv preprint arXiv:2506.13666*, 2025b.

Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. ReTool: Reinforcement learning for strategic tool use in LLMs. *arXiv preprint arXiv:2504.11536*, 2025. URL https://arxiv.org/abs/2504.11536.

Tao Feng, Haozhen Zhang, Zijie Lei, Peixuan Han, and Jiaxuan You. Graphplanner: Graph memory-augmented agentic routing for multi-agent llms. In *International Conference on Learning Representations*, 2026a. URL https://openreview.net/forum?id=ZdGB7MNQDT. ICLR 2026 Poster; code available at https://github.com/ulab-uiuc/GraphPlanner.

Xinshun Feng, Xinhao Song, Lijun Li, Gongshen Liu, and Jing Shao. SEARL: Joint optimization of policy and tool graph memory for self-evolving agents. *arXiv preprint arXiv:2604.07791*, 2026b. URL https://arxiv.org/abs/2604.07791.

Mohamed Amine Ferrag, Norbert Tihanyi, Djallel Hamouda, Leandros Maglaras, Abderrahmane Lakas, and Merouane Debbah. From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows. *arXiv preprint arXiv:2506.23260*, 2025.

Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, Bryan Hooi, Min Lin, and Tianyu Pang. Flowreasoner: Reinforcing query-level meta-agents, 2025. URL https://arxiv.org/abs/2504.15257.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026.

Shanshan Gao and Liyi Zhou. Can agent benchmarks support their scores? evidence-supported bounds for interactive-agent evaluation. *arXiv preprint arXiv:2605.10448*, 2026. URL https://arxiv.org/abs/2605.10448.

Aniketh Garikaparthi, Manasi Patwardhan, and Arman Cohan. Researchgym: Evaluating language model agents on real-world ai research. *arXiv preprint arXiv:2602.15112*, 2026. URL https://arxiv.org/abs/2602.15112.

GLM-4.5 Team. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. *arXiv preprint arXiv:2508.06471*, 2025. URL https://arxiv.org/abs/2508.06471.

Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. Training long-context, multi-turn software engineering agents with reinforcement learning. *arXiv preprint arXiv:2508.03501*, 2025. URL https://arxiv.org/abs/2508.03501.

Google DeepMind. Gemini 3.5 flash, May 2026. URL https://deepmind.google/technologies/gemini/flash/. Google DeepMind model page and release materials, accessed 2026-05-20.

Naibin Gu, Chenxu Yang, Qingyi Si, Chuanyu Qin, Dingyu Yao, Peng Fu, Zheng Lin, Weiping Wang, Nan Duan, and Jiaqi Wang. Co-evolving policy distillation. *arXiv preprint arXiv:2604.27083*, 2026. URL https://arxiv.org/abs/2604.27083.

Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. *arXiv preprint arXiv:2402.08567*, 2024.

Charlie Guo. Shell + Skills + Compaction: Tips for Long-Running Agents That Do Real Work. https://developers.openai.com/blog/skills-shell-tips, February 2026. OpenAI Developers Blog. Published February 11, 2026. Accessed: 2026-06-24.

Chuanzhe Guo, Jingjing Wu, Sijun He, Yang Chen, Zhaoqi Kuang, Shilong Fan, Bingjin Chen, Siqi Bao, Jing Liu, Hua Wu, Qingfu Zhu, Wanxiang Che, and Haifeng Wang. Menvagent: Scalable polyglot environment construction for verifiable software engineering. *arXiv preprint arXiv:2601.22859*, 2026a. URL https://arxiv.org/abs/2601.22859.

Jing Guo, Nan Li, Jianchuan Qi, Hang Yang, Ruiqiao Li, Yuzhen Feng, Si Zhang, and Ming Xu. Empowering working memory for large language model agents. *arXiv preprint arXiv:2312.17259*, 2023. URL https://arxiv.org/abs/2312.17259.

Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. SkillProbe: Security auditing for emerging agent skill marketplaces via multi-agent collaboration. *arXiv preprint arXiv:2603.21019*, 2026b. URL https://arxiv.org/abs/2603.21019.

Bhaskar Gurram. Evaluating tool-using language agents: Judge reliability, propagation cascades, and runtime mitigation in AgentProp-Bench. *arXiv preprint arXiv:2604.16706*, 2026. URL https://arxiv.org/abs/2604.16706.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. SWE-skills-bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*, 2026. URL https://arxiv.org/abs/2603.15401.

Harbor Framework. Harbor: A framework for running agent evaluations and creating and using RL environments, 2026. URL https://github.com/harbor-framework/harbor.

Jun He and Deying Yu. Sovereign agentic loops: Decoupling AI reasoning from execution in real-world systems. *arXiv preprint arXiv:2604.22136*, 2026. URL https://arxiv.org/abs/2604.22136.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.

Chiyeong Heo, Jaechang Kim, Junhyuk Kwon, Hoyoung Kim, Dongmin Park, Jonghyun Lee, and Jungseul Ok. MMTB: Evaluating terminal agents on multimedia-file tasks. *arXiv preprint arXiv:2605.10966*, 2026. URL https://arxiv.org/abs/2605.10966.

HKUDS. CLI-Anything: Making ALL software agent-native, 2026a. URL https://github.com/HKUDS/CLI-Anything.

HKUDS. OpenHarness, 2026b. URL https://github.com/HKUDS/OpenHarness.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275, 2024.

Junxing Hu, Ai Han, Haolan Zhan, Pu Wei, Zhiqian Zhang, Yuhang Guo, Jiawei Lu, Zhen Chen, Haoran Li, and Zicheng Zhang. HiMA-Ecom: Enabling joint training of hierarchical multi-agent e-commerce assistants. *arXiv preprint arXiv:2506.19846*, 2025. URL https://arxiv.org/abs/2506.19846.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024. URL https://arxiv.org/abs/2408.08435.

Zhanghao Hu, Qinglin Zhu, Runcong Zhao, Di Liang, Hanqi Yan, Yulan He, and Lin Gui. Beyond rag for agent memory: Retrieval by decoupling and aggregation. *arXiv preprint arXiv:2602.02007*, 2026. URL https://arxiv.org/abs/2602.02007.

Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From raw experience to skill consumption: A systematic study of model-generated agent skills, 2026. URL https://arxiv.org/abs/2605.23899.

Jacob Jackson, Ben Trapani, Nathan Wang, and Wanqi Zhu. Improving composer through real-time rl, March 2026. URL https://cursor.com/blog/real-time-rl-for-composer. Cursor research blog, accessed 2026-04-24.

Naman Jain et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Haonian Ji et al. Clawarena: Benchmarking AI agents in evolving information environments. *arXiv preprint arXiv:2604.04202*, 2026.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026a. URL https://arxiv.org/abs/2603.12056.

Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. SoK: Agentic skills — beyond tool use in LLM agents. *arXiv preprint arXiv:2602.20867*, 2026b.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.06770.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025a. URL https://arxiv.org/abs/2503.09516.

Chuanyang Jin, Jing Xu, Bo Liu, Leitian Tao, Olga Golovneva, Tianmin Shu, Wenting Zhao, Xian Li, and Jason Weston. The era of real-world human interaction: RL from user conversations. *arXiv preprint arXiv:2509.25137*, 2025b. URL https://arxiv.org/abs/2509.25137.

Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021. doi: 10.1561/2200000083.

Seth Karten, Joel Zhang, Tersoo Upaa, Jr., Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents, 2026. URL https://arxiv.org/abs/2605.09998.

KAT-Coder Team. KAT-Coder-V2 technical report. *arXiv preprint arXiv:2603.27703*, 2026. URL https://arxiv.org/abs/2603.27703.

Kimi Team. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025. URL https://arxiv.org/abs/2507.20534.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114. URL https://arxiv.org/abs/1612.00796.

Alexandre Lacoste, Nicolas Gontier, Oleh Shliazhko, Aman Jaiswal, Kusha Sareen, Shailesh Nanisetty, Joan Cabezas, Manuel Del Verme, Omar G. Younis, Simone Baratta, et al. Cube: A standard for unifying agent benchmarks. *arXiv preprint arXiv:2603.15798*, 2026. URL https://arxiv.org/abs/2603.15798.

Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. ComputerRL: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025. URL https://arxiv.org/abs/2508.14040.

Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in llm agents: Risks, mechanisms, and the stability and safety governed memory (ssgm) framework. *arXiv preprint arXiv:2603.11768*, 2026.

LangChain Team. LangGraph. LangChain Blog, January 2024. URL https://www.langchain.com/blog/langgraph. Official LangChain announcement blog; LangChain changelog also lists "Introducing LangGraph" in January 2024.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=5298fKGmv3.

Dongjun Lee, Ga-eun Bae, and Insu Yun. CTFusion: A CTF-based benchmark for LLM agent evaluation. *arXiv preprint arXiv:2605.11504*, 2026a. URL https://arxiv.org/abs/2605.11504.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026b. URL https://arxiv.org/abs/2603.28052.

Martin Legrand, Tao Jiang, Matthieu Feraud, Benjamin Navet, Yousouf Taghzouti, Fabien Gandon, Elise Dumont, and Louis-Félix Nothias. Mimosa framework: Toward evolving multi-agent systems for scientific research, 2026. URL https://arxiv.org/abs/2603.28986.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11401.

Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. *arXiv preprint arXiv:2506.12104*, 2025a.

Jiachun Li, Zhuoran Jin, Tianyi Men, Yupu Hao, Kejian Zhu, Lingshuai Wang, Dongqi Huang, Longxiang Wang, Shengjia Hua, Lu Wang, Jinshan Gao, Hongbang Yuan, Ruilin Xu, Kang Liu, and Jun Zhao. Agentic environment engineering for large language models: A survey of environment modeling, synthesis, evaluation, and application. *arXiv preprint arXiv:2606.12191*, 2026a. URL https://arxiv.org/abs/2606.12191.

Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Jianyang Gu, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Tianyang Wang, Jihun Hamm, Smita Krishnaswamy, Jun Huan, and Chandan K. Reddy. Agent harness engineering: A survey, May 2026b. URL https://openreview.net/forum?id=eONq7FdiHa. TMLR submission, under review; OpenReview forum eONq7FdiHa.

Sha Li and Naren Ramakrishnan. Experience as a compass: Multi-agent rag with evolving orchestration and agent prompts, 2026. URL https://arxiv.org/abs/2604.00901.

Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, volume 2, pages 429–450, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/file/1f5fe83998a09396ebe6477d9475ba0c-Paper.pdf.

Weizhen Li, Jianbo Lin, Zhuosong Jiang, Jingyi Cao, Xinpeng Liu, Jiayu Zhang, Zhenqiang Huang, Qianben Chen, Weichen Sun, Qiexiang Wang, et al. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic RL. *arXiv preprint arXiv:2508.13167*, 2025b. URL https://arxiv.org/abs/2508.13167.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026c. URL https://arxiv.org/abs/2602.12670.

Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail. *arXiv preprint arXiv:2601.04748*, 2026. URL https://arxiv.org/abs/2601.04748.

Xinze Li, Ziyue Zhu, Siyuan Liu, Yubo Ma, Yuhang Zang, Yixin Cao, and Aixin Sun. Emembench: Interactive benchmarking of episodic memory for vlm agents. *arXiv preprint arXiv:2601.16690*, 2026d.

Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026e. URL https://arxiv.org/abs/2604.13016.

Yixuan Li, Mingshu Cai, Ziyang Xiao, Wanyuan Wang, Yanchen Deng, and Bo An. MIND-Skill: Quality-guaranteed skill generation via multi-agent induction and deduction. *arXiv preprint arXiv:2605.08670*, 2026f. URL https://arxiv.org/abs/2605.08670.

Yu Li, Haoyu Luo, Yuejin Xie, Yuqian Fu, Zhonghao Yang, Shuai Shao, Qihan Ren, Wanying Qu, Yanwei Fu, Yujiu Yang, Jing Shao, Xia Hu, and Dongrui Liu. ATBench: A diverse and realistic agent trajectory benchmark for safety evaluation and diagnosis. *arXiv preprint arXiv:2604.02022*, 2026g.

Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning, 2026h. URL https://arxiv.org/abs/2603.16060.

Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, and Pan Lu. In-the-flow agentic system optimization for effective planning and tool use. *arXiv preprint arXiv:2510.05592*, 2025c. URL https://arxiv.org/abs/2510.05592.

Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. Skillnet: Create, evaluate, and connect AI skills. *arXiv preprint arXiv:2603.04448*, 2026a. URL https://arxiv.org/abs/2603.04448.

Zi Liang, Ronghua Li, Yanyun Wang, Qingqing Ye, and Haibo Hu. Can a single message paralyze the AI infrastructure? the rise of AbO-DDoS attacks through targeted mobius injection. *arXiv preprint arXiv:2605.11442*, 2026b. URL https://arxiv.org/abs/2605.11442.

Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation. *arXiv preprint arXiv:2605.27366*, 2026a. URL https://arxiv.org/abs/2605.27366.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Xuanjing Huang, Hang Yan, Zhenhua Han, and Tao Gui. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026b. URL https://arxiv.org/abs/2604.25850.

Yusong Lin, Haiyang Wang, Shuzhe Wu, Lue Fan, Feiyang Pan, Sanyuan Zhao, and Dandan Tu. CLI-Gym: Scalable CLI task generation via agentic environment inversion. *arXiv preprint arXiv:2602.10999*, 2026c. URL https://arxiv.org/abs/2602.10999.

Genglin Liu, Shijie Geng, Sha Li, Hejie Cui, Sarah Zhang, Xin Liu, and Tianyi Liu. Webcoach: Self-evolving web agents with cross-session memory guidance. *arXiv preprint arXiv:2511.12997*, 2025a.

Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. SafeAgent: A runtime protection architecture for agentic systems. *arXiv preprint arXiv:2604.17562*, 2026a. URL https://arxiv.org/abs/2604.17562.

Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, and Zhiyu Li. Skillsvote: Lifecycle governance of agent skills from collection, recommendation to evolution, 2026b. URL https://arxiv.org/abs/2605.18401.

Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026c.

Ruoqi Liu, Imran Q. Mohiuddin, Austin J. Schoeffler, Kavita Renduchintala, Ashwin Nayak, Prasantha L. Vemu, Shivam C. Vedak, Kameron C. Black, John L. Havlik, Isaac Ogunmola, Stephen P. Ma, Roopa Dhatt, and Jonathan H. Chen. PhysicianBench: Evaluating LLM agents in real-world EHR environments. *arXiv preprint arXiv:2605.02240*, 2026d. URL https://arxiv.org/abs/2605.02240.

Xiao Liu et al. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.03688.

Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint arXiv:2604.08618*, 2026e. URL https://arxiv.org/abs/2604.08618.

Yitao Liu et al. Contextual experience replay for self-improvement of language agents. *arXiv preprint arXiv:2506.06698*, 2025b. ACL 2025; WebArena verification.

Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings. *arXiv preprint arXiv:2604.04323*, 2026f. URL https://arxiv.org/abs/2604.04323.

Zeyuan Liu, Jeonghye Kim, Xufang Luo, Dongsheng Li, and Yuqing Yang. Exploratory memory-augmented LLM agent via hybrid on- and off-policy optimization. *arXiv preprint arXiv:2602.23008*, 2026g. URL https://arxiv.org/abs/2602.23008.

Zidong Liu, Zhuoyan Xu, Zhenmei Shi, and Yingyu Liang. Can language models compose skills in-context? *arXiv preprint arXiv:2510.22993*, 2025c. URL https://arxiv.org/abs/2510.22993.

Yunbo Long. Ai-supervisor: Autonomous ai research supervision via a persistent research world model, 2026. URL https://arxiv.org/abs/2603.24402.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017. URL https://arxiv.org/abs/1706.08840. Canonical replay-based CL baseline with transfer-oriented metrics.

Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. Deepseek-vl: Towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525*, 2024. URL https://arxiv.org/abs/2403.05525.

Shuo Lu, Kecheng Yu, Siru Jiang, Yinuo Xu, Bing Zhan, Yanbo Wang, Changxin Ke, Yuan Xu, Xin Xiong, Xinyun Zhou, Yihua Shao, Zhengbo Wang, Lijun Sheng, Aijing Yu, Haosen Yang, Yunpu Ma, Hao Tang, Nicu Sebe, Tat-Seng Chua, Philip Torr, Ran He, and Jian Liang. OpenClaw research: A systematic survey of large language model agents in open deployment, May 2026a. URL https://ykc1.github.io/OpenClaw_Survey_Web/. Version dated May 25, 2026.

Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. SKILL0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026b. URL https://arxiv.org/abs/2604.02268.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train ANY AI agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025. URL https://arxiv.org/abs/2508.03680.

Yang Luo, Zifeng Kang, Tiantian Ji, Xinran Liu, Yong Liu, Shuyu Li, and Lingyun Peng. ShadowMerge: A novel poisoning attack on graph-based agent memory via relation-channel conflicts. *arXiv preprint arXiv:2605.09033*, 2026. URL https://arxiv.org/abs/2605.09033.

Kexin Ma, Bojun Li, Yuhua Tang, Liting Sun, and Ruochun Jin. Cast: Character-and-scene episodic memory for agents. *arXiv preprint arXiv:2602.06051*, 2026a. URL https://arxiv.org/abs/2602.06051.

Wenquan Ma, Jiayan Nan, Wenlong Wu, and Yize Chen. What deserves memory: Adaptive memory distillation for llm agents. *arXiv preprint arXiv:2508.03341*, 2025. URL https://arxiv.org/abs/2508.03341.

Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026b. URL https://arxiv.org/abs/2604.08377.

Adyasha Maharana et al. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024. URL https://arxiv.org/abs/2402.17753.

Bulat Maksudov, Vladislav Kurenkov, Kathleen M. Curran, and Alessandra Mileo. ABRA: Agent benchmark for radiology applications. *arXiv preprint arXiv:2605.11224*, 2026. URL https://arxiv.org/abs/2605.11224.

Lance Martin, Gabe Cemaj, and Michael Cohen. Scaling managed agents: Decoupling the brain from the hands, April 2026. URL https://www.anthropic.com/engineering/managed-agents. Anthropic Engineering Blog, published April 8, 2026.

Stephen Martin. Most AI agent failures are harness failures, April 2026. URL https://www.martintechlabs.com/blog/most-ai-agent-failures-are-harness-failures. Martin Tech Labs blog, published April 21, 2026.

Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 2017. URL https://proceedings.mlr.press/v54/mcmahan17a.html.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025. URL https://arxiv.org/abs/2507.13334.

Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. *Preprints*, 2026a. doi: 10.20944/preprints202604.0428.v2. URL https://www.preprints.org/manuscript/202604.0428/v2.

Xiangcheng Meng, Shu Wang, and Yixiang Fang. SkillRAE: Agent skill-based context compilation for retrieval-augmented execution. *arXiv preprint arXiv:2605.10114*, 2026b. URL https://arxiv.org/abs/2605.10114.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. URL https://arxiv.org/abs/2601.11868.

Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric PPO for LLM agents. *arXiv preprint arXiv:2602.01869*, 2026. URL https://arxiv.org/abs/2602.01869.

MiniMax. MiniMax M2.7: Agent-native intelligence, April 2026. URL https://www.minimax.io/models/text/m27. MiniMax model release page, accessed 2026-05-20.

Model Context Protocol. Model Context Protocol Specification, 2025. URL https://github.com/modelcontextprotocol/modelcontextprotocol.

Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of LLM agents: A survey. *arXiv preprint arXiv:2507.21504*, 2025. doi: 10.1145/3711896.3736570. URL https://arxiv.org/abs/2507.21504.

Moonshot AI. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026. URL https://arxiv.org/abs/2602.02276.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Huyen Nguyen, Heidi Jiang, Xi Chen, Toki Kundu, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2022. URL https://arxiv.org/abs/2112.09332.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026. URL https://arxiv.org/abs/2603.25158.

Nous Research. Hermes Agent. Open-source agent project, February 2026. URL https://github.com/NousResearch/hermes-agent. Nous Research release page lists Hermes Agent on 2026-02-25 as an autonomous agent that lives on a server, remembers what it learns, and gets more capable the longer it runs.

OpenAI. Introducing codex, May 2025. URL https://openai.com/index/introducing-codex/. OpenAI product release, accessed 2026-05-12.

OpenAI. Introducing GPT-5.5, April 2026. URL https://openai.com/index/introducing-gpt-5-5/. OpenAI model release note, accessed 2026-05-20.

OpenClaw Contributors. OpenClaw, 2026. URL https://github.com/openclaw/openclaw.

Addy Osmani. Loop engineering, June 2026. URL https://addyo.substack.com/p/loop-engineering. Substack post on designing agentic coding loops.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents, 2026. URL https://arxiv.org/abs/2605.06614.

Akshay Pachaar. The anatomy of an agent harness. X/Twitter post, 2026. Posted by @akshay_pachaar on April 6, 2026; reports Anthropic/Claude Code engineering patterns including separation-of-concerns permission checks.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. *arXiv preprint arXiv:2412.21139*, 2024. URL https://arxiv.org/abs/2412.21139.

Wenbo Pan, Shujie Liu, Xiangyang Zhou, Shiwei Zhang, Wanlu Shi, Mirror Xu, and Xiaohua Jia. M*: Every task deserves its own memory harness, 2026. URL https://arxiv.org/abs/2604.11811.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

Atharv Singh Patlan, Ashwin Hebbar, Pramod Viswanath, and Prateek Mittal. Context manipulation attacks: Web agents are susceptible to corrupted memory. *arXiv preprint arXiv:2506.17318*, 2025.

Swarna Kamal Paul, Shubhendu Sharma, and Nitin Sareen. Gaama: Graph augmented associative memory for agents. *arXiv preprint arXiv:2603.27910*, 2026. URL https://arxiv.org/abs/2603.27910.

Xiaoxuan Peng, Kaiqi Zhang, Xinyu Lu, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Litecoder-terminal: Scaling long-horizon terminal environments for learning language agents. *arXiv preprint arXiv:2605.29559*, 2026a.

Yulin Peng, Xinxin Zhu, Chenxing Wei, Nianbo Zeng, Leilei Wang, Ying Tiffany He, and F. Richard Yu. Sage: Multi-agent self-evolution for llm reasoning, 2026b. URL https://arxiv.org/abs/2603.15255.

Hung Pham Van, Nguyen Manh Hieu, Khang Pham Tran Tuan, Nam Le Hai, Linh Ngo Van, Nguyen Thi Ngoc Diep, and Trung Le. Memorai: Memory organization and retrieval via adaptive graph intelligence for llm conversational agents. *arXiv preprint arXiv:2605.01386*, 2026. URL https://arxiv.org/abs/2605.01386.

Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025a. URL https://arxiv.org/abs/2504.13958.

Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jielin Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. xRouter: Training cost-aware LLMs orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025b. URL https://arxiv.org/abs/2510.08439.

Hongjin Qian and Zheng Liu. Metaagent: Toward self-evolving agent via tool meta-learning, 2025. URL https://arxiv.org/abs/2508.00271.

Rushi Qiang, Yuchen Zhuang, Yinghao Li, Dingu Sagar V K, Rongzhi Zhang, Changhao Li, Ian Shu-Hei Wong, Sherry Yang, Percy Liang, Chao Zhang, and Bo Dai. MLE-Dojo: Interactive environments for empowering LLM agents in machine learning engineering. *arXiv preprint arXiv:2505.07782*, 2025a. URL https://arxiv.org/abs/2505.07782.

Rushi Qiang, Yuchen Zhuang, Anikait Singh, Percy Liang, Chao Zhang, Sherry Yang, and Bo Dai. MLE-Smith: Scaling MLE tasks with automated multi-agent pipeline. *arXiv preprint arXiv:2510.07307*, 2025b. URL https://arxiv.org/abs/2510.07307.

Jingyang Qiao, Weicheng Meng, Yu Cheng, Zhihang Lin, Zhizhong Zhang, Xin Tan, Jingyu Gong, Kun Shao, and Yuan Xie. Memory intelligence agent, 2026. URL https://arxiv.org/abs/2604.04503.

Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2024. ICLR 2024 Spotlight; 16,464 real APIs.

Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual LLM agent refinement. *arXiv preprint arXiv:2601.22758*, 2026. URL https://arxiv.org/abs/2601.22758.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Harsh Raj, Niranjan Orkat, Suvrorup Mukherjee, Aritra Guha, Cheryl Flynn, and Subhabrata Majumdar. Consistency as a testable property: Statistical methods to evaluate AI agent reliability. *arXiv preprint arXiv:2605.10516*, 2026. URL https://arxiv.org/abs/2605.10516.

Recursive. First steps toward automated ai research, June 2026. URL https://www.recursive.com/articles/first-steps-toward-automated-ai-research. Recursive blog post on automated AI research loops.

Bowen Ren, Heyan Huang, Yinghao Li, and Yang Gao. Metaevo: A meta-optimization framework for experience-driven agent evolution. OpenReview, 2025. URL https://openreview.net/forum?id=1YcMHVY9cl. ICLR 2026 withdrawn submission.

Cursor Research, Aaron Chan, Ahmed Shalaby, Alexander Wettig, Aman Sanger, Andrew Zhai, Anurag Ajay, Ashvin Nair, Charlie Snell, et al. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*, 2026. URL https://arxiv.org/abs/2603.24477.

Taeyun Roh, Wonjune Jang, Junha Jung, and Jaewoo Kang. Clag: Adaptive memory organization via agent-driven clustering for small language model agents. *arXiv preprint arXiv:2603.15421*, 2026. URL https://arxiv.org/abs/2603.15421.

Xiaochan Xue Rong Xu, Yinxin Wan. Domusmind: A benchmark for evaluating lifelong smart home agents under drift. *OpenReview*, 2026. URL https://openreview.net/forum?id=dCBF23RZYJ. ICLR 2026 Workshop LLA.

Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. Aorchestra: Automating sub-agent creation for agentic orchestration, 2026. URL https://arxiv.org/abs/2602.03786.

Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020. URL https://aima.cs.berkeley.edu/.

Shoumik Saha, Kazem Faghih, and Soheil Feizi. Under the hood of SKILL.md: Semantic supply-chain attacks on AI agent skill registry. *arXiv preprint arXiv:2605.11418*, 2026. URL https://arxiv.org/abs/2605.11418.

Jürgen Schmidhuber. Goedel machines: Self-referential universal problem solvers making provably optimal self-improvements, 2003. URL https://arxiv.org/abs/cs/0309048.

Sahil Sen, Elias Lumer, Anmol Gulati, and Vamse Kumar Subbiah. Chronos: Temporal-aware conversational agents with structured event retrieval for long-term memory. *arXiv preprint arXiv:2603.16862*, 2026.

Ning Shang, Yifei Liu, Yi Zhu, Li Lyna Zhang, Weijiang Xu, Xinyu Guan, Buze Zhang, Bingcheng Dong, Xudong Zhou, Bowen Zhang, et al. rstar2-agent: Agentic reasoning technical report. *arXiv preprint arXiv:2508.20722*, 2025. URL https://arxiv.org/abs/2508.20722.

Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024. URL https://arxiv.org/abs/2410.06153.

Shanghai AI Lab. Frontier AI risk management framework in practice: A risk analysis technical report. *arXiv preprint arXiv:2507.16534*, 2025a. doi: 10.48550/arXiv.2507.16534.

Shanghai AI Lab. SafeWork-R1: Coevolving safety and intelligence under the AI-45° law. *arXiv preprint arXiv:2507.18576*, 2025b. doi: 10.48550/arXiv.2507.18576.

Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2actbench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. *arXiv preprint arXiv:2601.19935*, 2026a.

Zixuan Shen, Xiaowen Hu, Xiang Li, Tian Fang, Jia Li, and Sheng Zhang. World-model-augmented web agents with action correction. *arXiv preprint arXiv:2602.15384*, 2026b. URL https://arxiv.org/abs/2602.15384.

Yaorui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi Gu, Xunliang Cai, Xiang Wang, and An Zhang. Skill1: Unified evolution of skill-augmented agents via reinforcement learning, 2026. URL https://arxiv.org/abs/2605.06130.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Matthew Hausknecht, and Jesse Thomason. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.03768.

Akshey Sigdel and Rista Baral. ToolMisuseBench: An offline deterministic benchmark for tool misuse and recovery in agentic systems. *arXiv preprint arXiv:2604.01508*, 2026. URL https://arxiv.org/abs/2604.01508.

David Silver and Richard S. Sutton. Welcome to the era of experience, 2025. URL https://storage.googleapis.com/deepmind-media/Era-of-Experience%20/The%20Era%20of%20Experience%20Paper.pdf. DeepMind essay.

Joykirat Singh, Zaid Khan, Archiki Prasad, Justin Chih-Yao Chen, Akshay Nambi, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. Agent-BRACE: Decoupling beliefs from actions in long-horizon tasks via verbalized state uncertainty. *arXiv preprint arXiv:2605.11436*, 2026. URL https://arxiv.org/abs/2605.11436.

Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026. URL https://arxiv.org/abs/2604.00626.

Saksham Sahai Srivastava and Haoyu He. Memorygraft: Persistent compromise of llm agents via poisoned experience retrieval. *arXiv preprint arXiv:2512.16962*, 2025.

Hang Su, Jun Luo, Chang Liu, Xiao Yang, Yichi Zhang, Yinpeng Dong, and Jun Zhu. A survey on autonomy-induced security risks in large model-based agents. *arXiv preprint arXiv:2506.23844*, 2025a.

Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, et al. Toolorchestra: Elevating intelligence via efficient model and tool orchestration. *arXiv preprint arXiv:2511.21689*, 2025b. URL https://arxiv.org/abs/2511.21689.

Michael Sullivan, Mareike Hartmann, and Alexander Koller. Procedural environment generation for tool-use agents. *arXiv preprint arXiv:2506.11045*, 2025. URL https://arxiv.org/abs/2506.11045. Accepted at EMNLP 2025.

Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. SEAgent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025. URL https://arxiv.org/abs/2508.04700.

Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, and Julian McAuley. Generate, filter, control, replay: A comprehensive survey of rollout strategies for LLM reinforcement learning. *arXiv preprint arXiv:2605.02913*, 2026. URL https://arxiv.org/abs/2605.02913.

Hao Tang, Darren Key, and Kevin Ellis. Worldcoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2402.12275.

Yuheng Tang, Kaijie Zhu, Bonan Ruan, Chuqi Zhang, Michael Yang, Hongwei Li, Suyue Guo, Tianneng Shi, Zekun Li, Christopher Kruegel, Giovanni Vigna, Dawn Song, William Yang Wang, Lun Wang, Yangruibo Ding, Zhenkai Liang, and Wenbo Guo. Devops-gym: Benchmarking AI agents in software devops cycle. *arXiv preprint arXiv:2601.20882*, 2026. URL https://arxiv.org/abs/2601.20882.

Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models. *arXiv preprint arXiv:2404.14387*, 2024. URL https://arxiv.org/abs/2404.14387.

Hanling Tian, Zeyang Sha, Jingying Wang, Yuhang Liu, Zhehao Huang, and Xiaolin Huang. Injecmem: Memory injection attack on LLM agent memory systems. OpenReview submission to ICLR 2026, 2025. URL https://openreview.net/forum?id=QVX6hcJ2um.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 16022–16076. Association for Computational Linguistics, 2024. URL https://aclanthology.org/2024.acl-long.850/.

Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, and Dongbin Zhao. Dynamic dual-granularity skill bank for agentic RL. *arXiv preprint arXiv:2603.28716*, 2026. URL https://arxiv.org/abs/2603.28716.

Gido M. van de Ven and Andreas S. Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.

Nikhil Verma. Active context compression: Autonomous memory management in llm agents. *arXiv preprint arXiv:2601.07190*, 2026.

Bowen Wang, Dunjie Lu, Junli Wang, Tianyi Bai, Shixuan Liu, Zhipeng Zhang, Haiquan Wang, Hao Hu, Tianbao Xie, Shuai Bai, Dayiheng Liu, Que Shen, Junyang Lin, and Tao Yu. Cua-gym: Scaling verifiable training environments and tasks for computer-use agents. *arXiv preprint arXiv:2605.25624*, 2026a. URL https://arxiv.org/abs/2605.25624.

Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. SkillX: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026b. URL https://arxiv.org/abs/2604.04804.

Guanzhi Wang, Yuqi Xie, Zhenzhen Jiang, Ajay Mandlekar, Yuke Zhu, Anima Anandkumar, and Chelsea Fan. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents. *arXiv preprint arXiv:2604.10674*, 2026c. URL https://arxiv.org/abs/2604.10674.

Haoqing Wang, Xiang Long, Ziheng Li, Yilong Xu, Tingguang Li, and Yehui Tang. To mix or to merge: Toward multi-domain reinforcement learning for large language models. *arXiv preprint arXiv:2602.12566*, 2026d. URL https://arxiv.org/abs/2602.12566.

Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. SWE-dev: Building software engineering agents with training and inference scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3742–3761. Association for Computational Linguistics, July 2025a. doi: 10.18653/v1/2025.findings-acl.193. URL https://aclanthology.org/2025.findings-acl.193/. arXiv:2506.07636.

Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. Skillorchestra: Learning to route agents via skill transfer, 2026e. URL https://arxiv.org/abs/2602.19672.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025b. URL https://arxiv.org/abs/2512.17102.

Junjue Wang, Weihao Xuan, Heli Qi, Pengyu Dai, Kunyi Liu, Hongruixuan Chen, Zhuo Zheng, Junshi Xia, Stefano Ermon, and Naoto Yokoya. Can LLM agents respond to disasters? benchmarking heterogeneous geospatial reasoning in emergency operations. *arXiv preprint arXiv:2605.11633*, 2026f. URL https://arxiv.org/abs/2605.11633.

Juntong Wang, Haoyue Zhao, Guanghui Pan, Xiyuan Wang, Yanbo Wang, Qiyan Deng, and Muhan Zhang. Sage: A self-evolving agentic graph-memory engine for structure-aware associative memory. *arXiv preprint arXiv:2605.12061*, 2026g. URL https://arxiv.org/abs/2605.12061.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024. doi: 10.1007/s11704-024-40231-1.

Leye Wang, Zixing Wang, and Anjie Xu. Skilltester: Benchmarking utility and security of agent skills. *arXiv preprint arXiv:2603.28815*, 2026h. URL https://arxiv.org/abs/2603.28815.

Prince Zizhuang Wang and Shuli Jiang. Prime: Training free proactive reasoning via iterative memory evolution for user-centric agent, 2026. URL https://arxiv.org/abs/2604.07645.

Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner's guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025. URL https://arxiv.org/abs/2510.01132.

Xiaoxing Wang, Ning Liao, Shikun Wei, Chen Tang, and Feiyu Xiong. Autoagent: Evolving cognition and elastic memory orchestration for adaptive agents, 2026i. URL https://arxiv.org/abs/2603.09716.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025c.

Yanting Wang, Wei Zou, Runpeng Geng, and Jinyuan Jia. AgentWatcher: A rule-based prompt injection monitor. *arXiv preprint arXiv:2604.01194*, 2026j. URL https://arxiv.org/abs/2604.01194.

Yimeng Wang, Jiaxing Zhao, Hongbin Xie, Hexing Ma, Yuzhen Lei, Shuangxue Liu, Xuan Song, Zichen Zhang, and Haoran Zhang. Metagen: Self-evolving roles and topologies for multi-agent llm reasoning, 2026k. URL https://arxiv.org/abs/2601.19290.

Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. Openclaw-RL: Train any agent simply by talking, 2026l. URL https://arxiv.org/abs/2603.10165.

Yuchong Wang. SIP-Bench: An open protocol for longitudinal self-improvement evaluation, 2026. URL https://github.com/Yuchong-W/SIP_Bench. v0.1.0; supports SkillsBench , Evoagentbench and tau-bench environments.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026m. URL https://arxiv.org/abs/2602.10090. UNC-Chapel Hill & Snowflake AI Research; 1,000 executable MCP environments.

Zora Zhiruo Wang et al. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2025d. ICML 2025; WebArena +51% relative improvement.

Chuyang Wei, Maohang Gao, Zhixin Han, Kefei Chen, Yu Zhuang, Haoxiang Guan, Yanzhi Zhang, Yilin Cheng, Jiyan He, Huanhuan Chen, Jian Li, Yu Shi, Yitong Duan, and Shuxin Zheng. The world leaks the future: Harness evolution for future prediction agents, 2026. URL https://arxiv.org/abs/2604.15719.

Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-MemGuard: A proactive defense framework for LLM-based agent memory. *arXiv preprint arXiv:2510.02373*, 2025a.

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025b. URL https://arxiv.org/abs/2502.18449.

Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play SWE-RL. *arXiv preprint arXiv:2512.18552*, 2025c. URL https://arxiv.org/abs/2512.18552.

Ruoyao Wen, Hao Li, Chaowei Xiao, and Ning Zhang. Agentsys: Secure and dynamic LLM agents through explicit hierarchical memory management. *arXiv preprint arXiv:2602.07398*, 2026.

Zhaotian Weng, Antonis Antoniades, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing, 2026. URL https://arxiv.org/abs/2602.04837.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024a. URL https://arxiv.org/abs/2410.10813.

Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026a. URL https://arxiv.org/abs/2605.12493.

Jiangrong Wu, Yuhong Nan, Yixi Lin, Huaijin Wang, Yuming Xiao, Shuai Wang, and Zibin Zheng. SkillScope: Toward fine-grained least-privilege enforcement for agent skills. *arXiv preprint arXiv:2605.05868*, 2026b. URL https://arxiv.org/abs/2605.05868.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024b.

Xinle Wu, Rui Zhang, Mustafa Anis Hussain, and Yao Lu. Towards autonomous memory agents. *arXiv preprint arXiv:2602.22406*, 2026c. URL https://arxiv.org/abs/2602.22406.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. Isolategpt: An execution isolation architecture for LLM-based agentic systems. *arXiv preprint arXiv:2403.04960*, 2024c.

Yuhao Wu, Tung-Ling Li, and Hongliang Liu. Behavioral integrity verification for AI agent skills. *arXiv preprint arXiv:2605.11770*, 2026d. URL https://arxiv.org/abs/2605.11770.

Yuyang Wu, Yue Huang, Shuaike Shen, Xujian Wang, Shuhao Zhang, Qiyao Xue, Weichen Liu, Runtian Gao, Jian Ma, Xiangliang Zhang, and Olexandr Isayev. Can agents price a reaction? evaluating LLMs on chemical cost reasoning. *arXiv preprint arXiv:2605.07251*, 2026e. URL https://arxiv.org/abs/2605.07251.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023. URL https://arxiv.org/abs/2309.07864.

Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, Zuxuan Wu, Qi Zhang, Xuanjing Huang, and Yu-Gang Jiang. AgentGym-RL: Training LLM agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025. URL https://arxiv.org/abs/2509.08755.

Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025. URL https://arxiv.org/abs/2511.16043.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026a. URL https://arxiv.org/abs/2602.08234.

Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Metaclaw: Just talk – an agent that meta-learns and evolves in the wild. *arXiv preprint arXiv:2603.17187*, 2026b. URL https://arxiv.org/abs/2603.17187. Online LoRA plus skill evolution for deployed agents.

Zhishang Xiang, Chengyi Yang, Zerui Chen, Zhimin Wei, Yunbo Tang, Zongpei Teng, Zexi Peng, Zongxia Li, Chengsong Huang, Yicheng He, Chang Yang, Xinrun Wang, Xiao Huang, Qinggang Zhang, and Jinsong Su. A systematic survey of self-evolving agents: From model-centric to environment-driven co-evolution. *SSRN Electronic Journal*, 2026. doi: 10.2139/ssrn.6626878. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6626878.

Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. UI-Mem: Self-evolving experience memory for online reinforcement learning in mobile GUI agents. *arXiv preprint arXiv:2602.05832*, 2026. URL https://arxiv.org/abs/2602.05832.

Xiaomi LLM-Core Team. MiMo-VL technical report. *arXiv preprint arXiv:2506.03569*, 2025. URL https://arxiv.org/abs/2506.03569.

Xiaomi LLM-Core Team. MiMo-V2-Flash technical report. *arXiv preprint arXiv:2601.02780*, 2026. URL https://arxiv.org/abs/2601.02780.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal

agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. URL https://arxiv.org/abs/2404.07972.

Weiwei Xie, Shaoxiong Guo, Fan Zhang, Tian Xia, Xue Yang, Lizhuang Ma, Junchi Yan, and Qibing Ren. MemEvoBench: Benchmarking memory misevolution in LLM agents. *arXiv preprint arXiv:2604.15774*, 2026. URL https://arxiv.org/abs/2604.15774.

Haidong Xin, Xinze Li, Zhenghao Liu, Yukun Yan, Shuo Wang, Cheng Yang, Yu Gu, Ge Yu, and Maosong Sun. Metamem: Evolving meta-memory for knowledge utilization through self-reflective symbolic optimization, 2026. URL https://arxiv.org/abs/2602.11182.

Feng Xiong, Zengbin Wang, Yong Wang, Xuecai Hu, Jinghan He, Liang Lin, Yuan Liu, and Xiangxiang Chu. Ace-skill: Bootstrapping multimodal agents with prioritized and clustered evolution, 2026a. URL https://arxiv.org/abs/2605.08887.

Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs, 2026b. URL https://arxiv.org/abs/2602.07755.

Bin Xu. Ai agent systems: Architectures, applications, and evaluation. *arXiv preprint arXiv:2601.01743*, 2026a.

Bo Xu, Danny Zhang, and Rohit Arunachalam. From model to agent: Equipping the responses API with a computer environment, March 2026a. URL https://openai.com/index/equip-responses-api-computer-environment. OpenAI Engineering Blog, published March 11, 2026.

Chang Xu. The future of agents is open source (part 1 of 2), April 2026b. URL https://www.basisset.com/insights/the-future-of-agents-is-open-source-part-1-of-2. Basis Set blog, published April 17, 2026.

Duling Xu, Zheng Chen, Zaifeng Pan, Jiawei Guan, Dong Dong, Jialin Li, and Bangzheng Pu. SkillSmith: Compiling agent skills into boundary-guided runtime interfaces, 2026b. URL https://arxiv.org/abs/2605.15215.

Frank F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2025a. NeurIPS 2025; 175 long-horizon tasks.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026. URL https://arxiv.org/abs/2602.12430.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025b. URL https://arxiv.org/abs/2502.12110. Advances in Neural Information Processing Systems (NeurIPS 2025).

Zhenlin Xu, Xiaogang Zhu, Yu Yao, Minhui Xue, and Yiliao Song. From storage to steering: Memory control flow attacks on LLM agents. *arXiv preprint arXiv:2603.15125*, 2026c.

Tianci Xue, Zeyi Liao, Tianneng Shi, Zilu Wang, Kai Zhang, Dawn Song, Yu Su, and Huan Sun. Autonomous continual learning of computer-use agents for environment adaptation. *arXiv preprint arXiv:2602.10356*, 2026. URL https://arxiv.org/abs/2602.10356.

Zhiling Yan, Dingjie Song, Hanrong Zhang, Wei Liang, Yuxuan Zhang, Yutong Dai, Lifang He, Philip S. Yu, Ran Xu, Xiang Li, and Lichao Sun. Openskill: Open-world self-evolution for llm agents, 2026. URL https://arxiv.org/abs/2606.06741.

Chao Yang, Chaochao Lu, Yingchun Wang, and Bowen Zhou. Towards AI-45° law: A roadmap to trustworthy AGI. *arXiv preprint arXiv:2412.14186*, 2024a. doi: 10.48550/arXiv.2412.14186.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024b.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025a. URL https://arxiv.org/abs/2504.21798.

Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. Plugmem: A task-agnostic plugin memory module for llm agents. *arXiv preprint arXiv:2603.03296*, 2026a.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026b. URL https://arxiv.org/abs/2605.23904.

Yuqing Yang, Tengxiao Liu, Wang Bill Zhu, Taiwei Shi, Linxin Song, and Robin Jia. Self-evolving llm memory extraction across heterogeneous tasks, 2026c. URL https://arxiv.org/abs/2604.11610.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026d. URL https://arxiv.org/abs/2603.01145.

Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, et al. Nemotron-cascade 2: Post-training LLMs with cascade RL and multi-domain on-policy distillation. *arXiv preprint arXiv:2603.19220*, 2026e. URL https://arxiv.org/abs/2603.19220.

Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, Yanhao Li, Yue Liu, Zhenxing Hu, Kaitai Zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, Zhilin Yang, and Tianyu Liu. Kimi-Dev: Agentless training as skill prior for SWE-agents. *arXiv preprint arXiv:2509.23045*, 2025b. URL https://arxiv.org/abs/2509.23045.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

Shunyu Yao et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. ICLR 2025.

Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026a. URL https://arxiv.org/abs/2604.06132.

Chongrui Ye, Yuxiang Liu, Yu Wang, Haofei Yu, Yining Zhao, Ge Liu, Julian McAuley, and Jiaxuan You. Auto-dreamer: Learning offline memory consolidation for language agents. *arXiv preprint arXiv:2605.20616*, 2026b. URL https://arxiv.org/abs/2605.20616.

Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution, 2026c. URL https://arxiv.org/abs/2601.21557.

Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026d. URL https://arxiv.org/abs/2602.12275.

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of LLM-based agents. *arXiv preprint arXiv:2503.16416*, 2025. URL https://arxiv.org/abs/2503.16416.

Bo Yin, Qi Li, and Xinchao Wang. On-policy self-evolution via failure trajectories for agentic safety alignment. *arXiv preprint arXiv:2605.11882*, 2026. URL https://arxiv.org/abs/2605.11882.

Justin Young. Effective harnesses for long-running agents, November 2025. URL https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents. Anthropic Engineering Blog, published November 26, 2025.

Jiawei Yu, Yixiang Fang, Xilin Liu, and Yuchi Ma. H-mem: A novel memory mechanism for evolving and retrieving agent memory via a hybrid structure. *arXiv preprint arXiv:2605.15701*, 2026a.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026b. URL https://arxiv.org/abs/2601.01885.

Jiarui Yuan et al. SE-Bench: Benchmarking self-evolution with knowledge internalization. *arXiv preprint arXiv:2602.04811*, 2026.

Qianhao Yuan, Jie Lou, Zichao Li, Jiawei Chen, Yaojie Lu, Hongyu Lin, Le Sun, Debing Zhang, and Xianpei Han. MemSearcher: Training LLMs to reason, search and manage memory via end-to-end reinforcement learning. *arXiv preprint arXiv:2511.02805*, 2025. URL https://arxiv.org/abs/2511.02805.

Ling Yue, Kushal Raj Bhandari, Ching-Yun Ko, Dhaval Patel, Shuxin Lin, Nianjun Zhou, Jianxi Gao, Pin-Yu Chen, and Shaowu Pan. From static templates to dynamic runtime graphs: A survey of workflow optimization for llm agents. *arXiv preprint arXiv:2603.22386*, 2026.

Z.ai. GLM-5.1, May 2026. URL https://docs.z.ai/guides/llm/glm-5.1. Z.ai developer documentation, accessed 2026-05-20.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026a.

Kun Zeng, Yu Huo, Siyu Zhang, Zi Ye, Yuecheng Zhuo, Haoyue Liu, Yuquan Lu, Junhao Wen, and Xiaoying Tang. Group of skills: Group-structured skill retrieval for agent skill libraries. *arXiv preprint arXiv:2605.06978*, 2026b. URL https://arxiv.org/abs/2605.06978.

Sihang Zeng, Matthew Thompson, Ruth Etzioni, and Meliha Yetisgen. Traj-Evolve: A self-evolving multi-agent system for patient trajectory modeling in lung cancer early detection. *arXiv preprint arXiv:2606.02812*, 2026c. URL https://arxiv.org/abs/2606.02812.

Chiyu Zhang, Huiqin Yang, Bendong Jiang, Xiaolei Zhang, Yiran Zhao, Ruyi Chen, Lu Zhou, Xiaogang Xu, Jiafei Wu, Liming Fang, and Zhe Liu. LITMUS: Benchmarking behavioral jailbreaks of LLM agents in real OS environments. *arXiv preprint arXiv:2605.10779*, 2026a. URL https://arxiv.org/abs/2605.10779.

Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang. SkillEvolver: Skill learning as a meta-skill. *arXiv preprint arXiv:2605.10500*, 2026b. URL https://arxiv.org/abs/2605.10500.

Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet, 2025a. URL https://arxiv.org/abs/2502.04180.

Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, Yifan Zhou, Yang Chen, Chen Zhang, Yutao Fan, Zihu Wang, Songtao Huang, Francisco Piedrahita-Velez, Yue Liao, Hongru Wang, Mengyue Yang, Heng Ji, Jun Wang, Shuicheng Yan, Philip Torr, and Lei Bai. The landscape of agentic reinforcement learning for LLMs: A survey. *Transactions on Machine Learning Research*, 2026c. URL https://arxiv.org/abs/2509.02547.

Guilin Zhang, Wei Jiang, Xiejiashan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. Adaptive memory admission control for llm agents. *arXiv preprint arXiv:2603.04549*, 2026d.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*, 2025b. URL https://arxiv.org/abs/2410.02644.

Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026e. URL https://arxiv.org/abs/2604.01687.

Haozhen Zhang, Tao Feng, and Jiaxuan You. Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. *arXiv preprint arXiv:2506.09033*, 2025c. URL https://arxiv.org/abs/2506.09033.

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents, 2026f. URL https://arxiv.org/abs/2602.02474.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2025d. URL https://arxiv.org/abs/2505.22954.

Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents, 2026g. URL https://arxiv.org/abs/2603.19461.

Jiaquan Zhang, Chaoning Zhang, Shuxu Chen, Zhenzhen Huang, Pengcheng Zheng, Zhicheng Wang, Ping Guo, Fan Mo, Sung-Ho Bae, Jie Zou, Jiwei Wei, and Yang Yang. Lightweight llm agent memory with small language models. *arXiv preprint arXiv:2604.07798*, 2026h.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation, 2024a. URL https://arxiv.org/abs/2410.10762.

Linghao Zhang. Nurture-first agent development: Building domain-expert ai agents through conversational knowledge crystallization, 2026a. URL https://arxiv.org/abs/2603.10808.

Linghao Zhang et al. SWE-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025e. NeurIPS 2025 D&B.

Qi Zhang, Shen Huang, Chu Liu, Shouqing Yang, Junbo Zhao, Haobo Wang, and Pengjun Xie. Deltamem: Towards agentic memory management via reinforcement learning. *arXiv preprint arXiv:2604.01560*, 2026i.

Qiaohong Zhang, Weihao Ye, Jialong Chen, Yi Luo, BoYuan Li, Bowen Deng, Zibin Zheng, Jianhao Lin, Wei-Shi Zheng, and Chuan Chen. DataClaw: A process-oriented agent benchmark for exploratory real-world data analysis. *arXiv preprint arXiv:2605.02503*, 2026j. URL https://arxiv.org/abs/2605.02503.

Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026k. URL https://arxiv.org/abs/2601.03192.

Wentao Zhang. Autogenesis: A self-evolving agent protocol, 2026b. URL https://arxiv.org/abs/2604.15034.

Xiaoying Zhang, Zichen Liu, Yipeng Zhang, Xia Hu, and Wenqi Shao. Retroagent: From solving to evolving via retrospective dual intrinsic feedback. *arXiv preprint arXiv:2603.08561*, 2026l. URL https://arxiv.org/abs/2603.08561.

Xichen Zhang, Ziyi He, Yinghao Zhu, Sitong Wu, Shaozuo Yu, Meng Chu, Wenhu Zhang, Haoru Tan, and Jiaya Jia. Searchgym: Bootstrapping real-world search agents via cost-effective and high-fidelity environment simulation. *arXiv preprint arXiv:2601.14615*, 2026m. URL https://arxiv.org/abs/2601.14615.

Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. Live-evo: Online evolution of agentic memory from continuous feedback. *arXiv preprint arXiv:2602.02369*, 2026n. URL https://arxiv.org/abs/2602.02369.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024b. URL https://arxiv.org/abs/2404.13501.

Zhang Zhang, Shuqi Lu, Hongjin Qian, Di He, and Zheng Liu. Agentfactory: A self-evolving framework through executable subagent accumulation and reuse, 2026o. URL https://arxiv.org/abs/2603.18000.

Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-SafetyBench: Evaluating the safety of LLM agents. *arXiv preprint arXiv:2412.14470*, 2024c.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025. URL https://arxiv.org/abs/2505.03335. Zero-data self-play reasoning with verifier-grounded rewards.

Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yanyan Zhao, Yu Zhang, Yang Wu, Dandan Tu, Bing Qin, and Ting Liu. Rethinking experience utilization in self-evolving language model agents. *arXiv preprint arXiv:2605.07164*, 2026a. URL https://arxiv.org/abs/2605.07164.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m ChatGPT interaction logs in the wild. *arXiv preprint arXiv:2405.01470*, 2024. URL https://arxiv.org/abs/2405.01470.

Yang Zhao, Chengxiao Dai, Mengying Kou, and Yue Xiu. Memorepair: Barrier-first cascade repair in agentic memory. *arXiv preprint arXiv:2605.07242*, 2026b. URL https://arxiv.org/abs/2605.07242.

Junhao Zheng, Xidi Cai, Shengjie Qiu, and Qianli Ma. Spurious forgetting in continual learning of language models. *arXiv preprint arXiv:2501.13453*, 2025a.

Junhao Zheng et al. LifelongAgentBench: Evaluating LLM agents as lifelong learners. *arXiv preprint arXiv:2505.11942*, 2025b.

Yanzhao Zheng, Zhentao Zhang, Chao Ma, Yuanqiang Yu, Jihuan Zhu, Baohua Dong, and Hangcheng Zhu. SkillRouter: Retrieve-and-rerank skill selection for LLM agents at scale. *arXiv preprint arXiv:2603.22455*, 2026. URL https://arxiv.org/abs/2603.22455.

Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. SkillLearnBench: Benchmarking continual learning methods for agent skill generation on real-world tasks. *arXiv preprint arXiv:2604.20087*, 2026a. URL https://arxiv.org/abs/2604.20087.

Zhiqing Zhong, Zhijing Ye, Jiamin Wang, and Xiaodong Yu. An executable benchmarking suite for tool-using agents. *arXiv preprint arXiv:2605.11030*, 2026b. URL https://arxiv.org/abs/2605.11030.

Andy Zhou, Kevin Wu, Francesco Pinto, Zhaorun Chen, Yi Zeng, Yu Yang, Shuang Yang, Sanmi Koyejo, James Zou, and Bo Li. Autoredteamer: Autonomous red teaming with lifelong attack integration. *arXiv preprint arXiv:2503.15754*, 2025.

Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026a. URL https://arxiv.org/abs/2604.08224.

Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. Memento-Skills: Let agents design agents. *arXiv preprint arXiv:2603.18743*, 2026b. URL https://arxiv.org/abs/2603.18743.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. URL https://arxiv.org/abs/2307.13854.

Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu. Counterfactual trace auditing of LLM agent skills. *arXiv preprint arXiv:2605.11946*, 2026c. URL https://arxiv.org/abs/2605.11946.

Xiaolin Zhou, Aojie Yuan, Zheng Luo, Zipeng Ling, Xixiao Pan, Yicheng Gao, Haiyue Zhang, Jiate Li, Shuli Jiang, Prince Zizhuang Wang, Zixuan Zhu, Jinbo Liu, Ryan A. Rossi, Hua Wei, and Xiyang Hu. When simulation lies: A sim-to-real benchmark and domain-randomized RL recipe for tool-use agents. *arXiv preprint arXiv:2605.11928*, 2026d. URL https://arxiv.org/abs/2605.11928.

Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. Synthetic sandbox for training machine learning engineering agents. *arXiv preprint arXiv:2604.04872*, 2026e. URL https://arxiv.org/abs/2604.04872.

Zhaojiacheng Zhou. Proteus: A self-evolving red team for agent skill ecosystems. *arXiv preprint arXiv:2605.11891*, 2026. URL https://arxiv.org/abs/2605.11891.

Qiming Zhu, Shunian Chen, Rui Yu, Zhehao Wu, and Benyou Wang. From lossy to verified: A provenance-aware tiered memory for agents. *arXiv preprint arXiv:2602.17913*, 2026a. URL https://arxiv.org/abs/2602.17913.

Wenhui Zhu, Xiwen Chen, Zhipeng Wang, Jingjing Wang, Xuanzhao Dong, Minzhou Huang, Rui Cai, Hejian Sang, Hao Wang, Peijie Qiu, Yueyue Deng, Prayag Tiwari, Brendan Hogan Rappazzo, and Yalin Wang. Ariadnemem: Threading the maze of lifelong memory for llm agents. *arXiv preprint arXiv:2603.03290*, 2026b.

Mingchen Zhuge, Changsheng Zhao, Haozhe Liu, Zijian Zhou, Shuming Liu, Wenyi Wang, Ernie Chang, Gael Le Lan, Junjie Fei, Wenxuan Zhang, Yasheng Sun, Zhipeng Cai, Zechun Liu, Yunyang Xiong, Yining Yang, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Neural computers. *arXiv preprint arXiv:2604.06425*, 2026. URL https://arxiv.org/abs/2604.06425.

Hongyao Tang Zihang Ma, Jinyi Liu et al. Benchmarking continual agent memory for online learning, transfer, and forgetting. *OpenReview*, 2026. URL https://openreview.net/forum?id=MSXbrNExax. ICLR 2026 Workshop LLA.

Mingxi Zou, Zhihan Guo, Langzhang Liang, Zhuo Wang, Qifan Wang, Qingsong Wen, Irwin King, Lizhen Qu, and Zenglin Xu. Remember the decision, not the description: A rate-distortion framework for agent memory. *arXiv preprint arXiv:2605.10870*, 2026. URL https://arxiv.org/abs/2605.10870.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning, 2025. URL https://arxiv.org/abs/2504.16084.